STOPWATCH · SENSOR · UART

Basys-3 FPGA 시계 · 센서 · UART 통합 시스템 — 설계 · 검증

WTS SR04 DHT11 UART

2026. 11. 02

팀원 정보윤 · 이종혁 · 정안성 · 견다혜

Contents

Basys-3 FPGA 한 보드에 시계 · 센서 · UART
기능을 하나의 Top 으로 통합,
Top · 하위 모듈 테스트벤치로 전체 경로 검증

- **WTS**
  스톱워치 · 시계 · 타이머 (1/100 s)
- **SR04**
  초음파 거리 0 ~ 400 cm · 60 ms 주기
- **DHT11**
  온도 ℃ · 습도 %RH · 2 s 주기
- **UART**
  9600 bps · RX 8 / TX 64 FIFO

01 프로젝트 개요
요약 · 목적 · 스펙 · 보드 IO · UART 명령표

02 시스템 구조
Top 블록 다이어그램 · 명령별 흐름 · UART 동작

03 Top · 모듈 검증
Top 시뮬레이션 파형 · Top TB 검증 · UART sender 설계 · 검증

04 트러블 슈팅
센서 오류 표시 (B-2 · B-3) · TB 가 찾은 결함 8 건

05 시연 영상
보드 동작

06 결론 · 고찰
목표 달성 수치 · 남은 과제

+ 부록
UART · Top 파형 상세 · sender 파형 해설 · 결함 8 건 상세 · 시연 영상

● 01 프로젝트 개요 – 견다혜

프로젝트 요약 및 목표

목적 | **Basys3 FPGA 통합 기능을 UART 명령으로 제어 / 상태·센서 데이터를 PC에서 확인**

UART 제어

6 종 24 개

명령 종류 · 명령+옵션 조합

버튼·스위치와 함께 제어
1회 요청 · 1 Hz 자동 전송

센서 오류 처리

3 + 3 유형

SR04 3종 · DHT11 3종

오류를 상태 코드로 구분해
FND·PC에 Err1~3으로 전달

FPGA 자원·타이밍

7.1 % LUT

LUT 1,476 / 20,800 · FF 1.8%

WNS +1.442 ns @ 100 MHz
배치·배선 후 · 100 MHz 경로 기준

TB 검증

305 PASS

전체 TB 12개 · 검사 305개

발견 결함 8건 수정 ·
잔여 결함 0건

● 01 프로젝트 개요 – 견다혜

시스템 스펙

요약 100 MHz FPGA 하나에 PC (UART) · 센서 2 종 연결



Basys3 보드 IO 구성

요약 **PC 통신 1 · 센서 Pmod 2 · 버튼 5 · 스위치 8 · LED 16 · FND 4 자리 — 한 보드에 IO 배치**

JA5 · GND → SR04 GND

JA2 · L2 → SR04 ECHO (echo)
echo 5 V 출력이면 분압 저항으로 3.3 V

JA1 · J1 → SR04 TRIG (trig)

JXADC6 · PWR (3.3 V) → DHT11 VCC

JXADC5 · GND → DHT11 GND

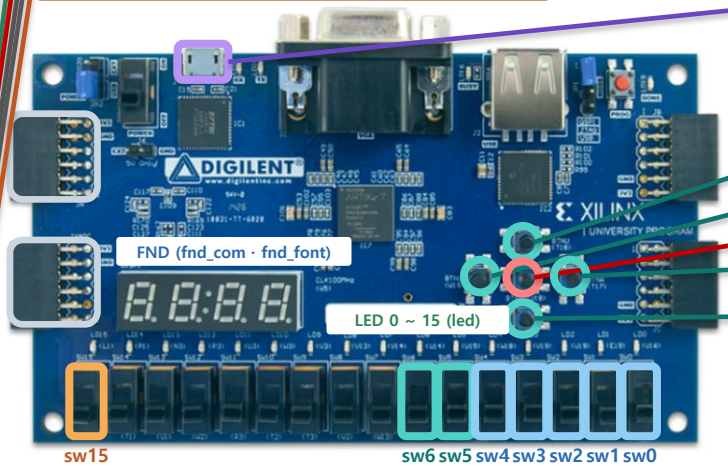
JXADC3 · M2 → DHT11 DATA (dht11_io)
PULLUP · 외부 5 kΩ 풀업 권장

- **sw0** 시계 / 스톱워치 · **sw1** 초.1/100초 / 시.분 · **sw2** 시계 시간 설정 **sw3** 12 / 24 시간 · **sw4** 타이머
- **sw5** 거리 (SR04) 화면 · **sw6** 온습도 (DHT11) 화면 (sw6 우선) UART 로 /mod 를 받은 뒤에는 sw0 ~ sw6 무시
- **sw15** 거리 막대 LED (LD0 ~ 7) 켜기
- **버튼** : 지금 화면의 모듈에만 들어감 (거리 · 온습도 : R 1 회 측정, U 자동 측정 켜기/끄기)

micro USB : 전원 + UART (9600 8N1)
rx = B18 (PC → FPGA) · tx = A18 (FPGA → PC)

JA12 : PWR — JA6 : PWR
JA11 : GND — JA5 : GND
JA10 : G3 — JA4 : G2
JA9 : H2 — JA3 : J2
JA8 : K2 — JA2 : L2
JA7 : H1 — JA1 : J1

JXAC12 : PWR — JXAC6 : PWR
JXAC11 : GND — JXAC5 : GND
JXAC10 : N1 — JXAC4 : N2
JXAC9 : M1 — JXAC3 : M2
JXAC8 : M3 — JXAC2 : L3
JXAC7 : K3 — JXAC1 : J3



BTNU · T18 → btn_U

BTNL · W19 → btn_L

BTNC · U18 → rst (리셋)

BTNR · T17 → btn_R

BTND · U17 → btn_D

● 01 프로젝트 개요 – 견다혜

UART 명령어 및 기능

요약 **명령 6 종 (/mod · /get · /btn · /auto · /set · /off) — PC 터미널로 화면 · 버튼 · 데이터 전송 제어**

명령어	option	설정 대상	기능	관련 IO	사용 예시
/mod	w_hm	시계, 시.분	화면 바꾸기 (이후 스위치 대신 UART 가 화면 결정)	④ FND	/mod dist → FND : 거리
	w_ms	시계, 초.1/100초			
	s_hm	스톱워치, 시.분			
	s_ms	스톱워치, 초.1/100초			
	t_hm	타이머 (분.초)			
	t_ms	타이머 (분.초)			
	dist	거리 (SR04)			
	temp	온습도 (DHT11)			
/get	–	지금 화면	한 줄 전송 (공백 뒤 Enter)	① UART	/get ⏎
/btn	R	버튼 R	버튼 한 번 (물리 버튼과 같음)	⑤ 버튼	/btn U 거리 화면 : SR04 자동 측정 켜기/끄기
	L	버튼 L			
	U	버튼 U			
	D	버튼 D			

명령어	option	설정 대상	기능	관련 IO	사용 예시
/auto	all	전체 (돌아가며)	1초마다 순환 전송	① UART	/auto all
	_sw	스톱워치	1초마다 한 줄 전송		/auto dis ← 045cm
	_tm	타이머			
	_wh	시계			
	dis	거리			
	tem	온습도			
	off	—	자동 전송 끄기		/auto off
/set	w_hm	시계 시간 설정	설정 켜기	⑦ SW2	/set w_hm
	24h	24시간 표시	+ Enter	⑦ SW3	/set 24h
	12h	12시간 표시 (기본)			
/off	w_h	시계 시간 설정	설정 끄기	⑦ SW2	/off w_h

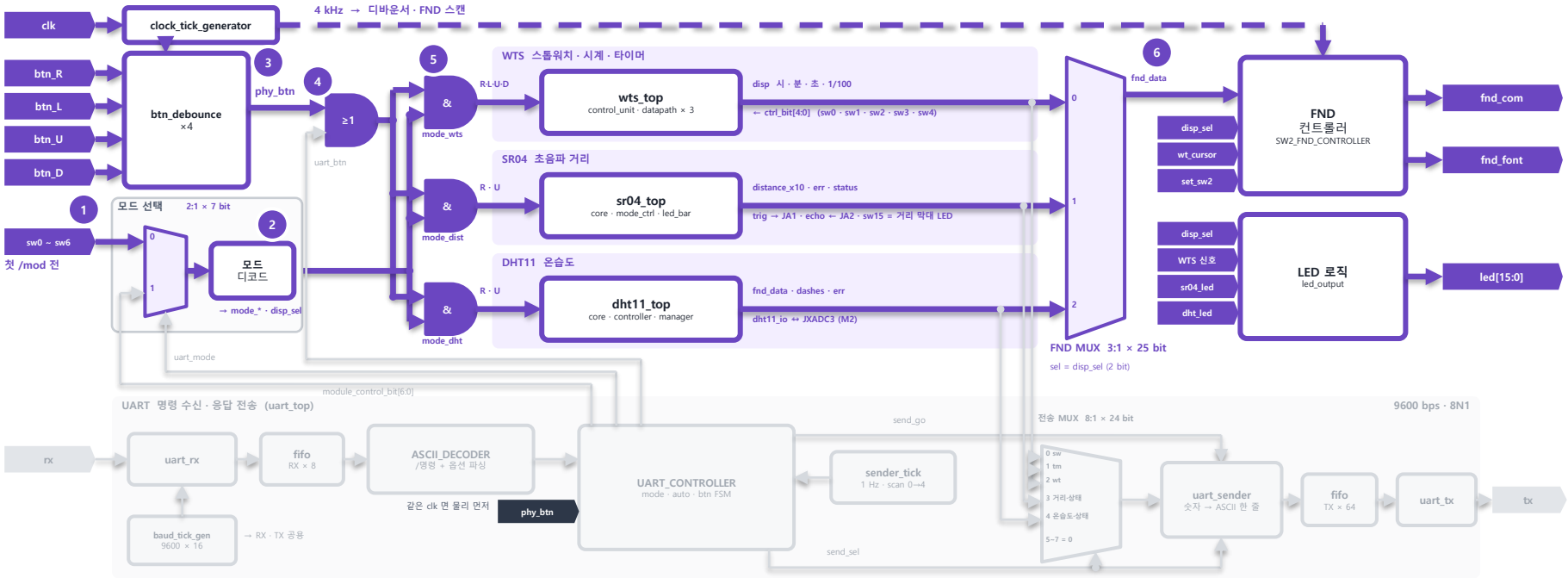
※ /set · /off 는 /mod 를 한 번 받은 뒤에만 동작
※ /get 은 자동 전송 중 무시

통신 **9600 bps · 8N1 · 줄 끝 CR 또는 LF (Enter)**
형식 **/<명령 3글자> <옵션>** · 명령은 소문자, 버튼만 대문자 R L U D · 형식 오류 → **ERROR**

02 시스템 구조 · 보드 입력 – 이종혁

스위치 · 버튼 흐름 — /mod 전에는 보드 입력으로 조작

요약 조작 방법 2 가지 (보드 스위치·버튼 / PC UART 명령) 중 첫 번째 · 리셋 직후 스위치로 화면 선택 · 버튼은 지금 화면 기능만



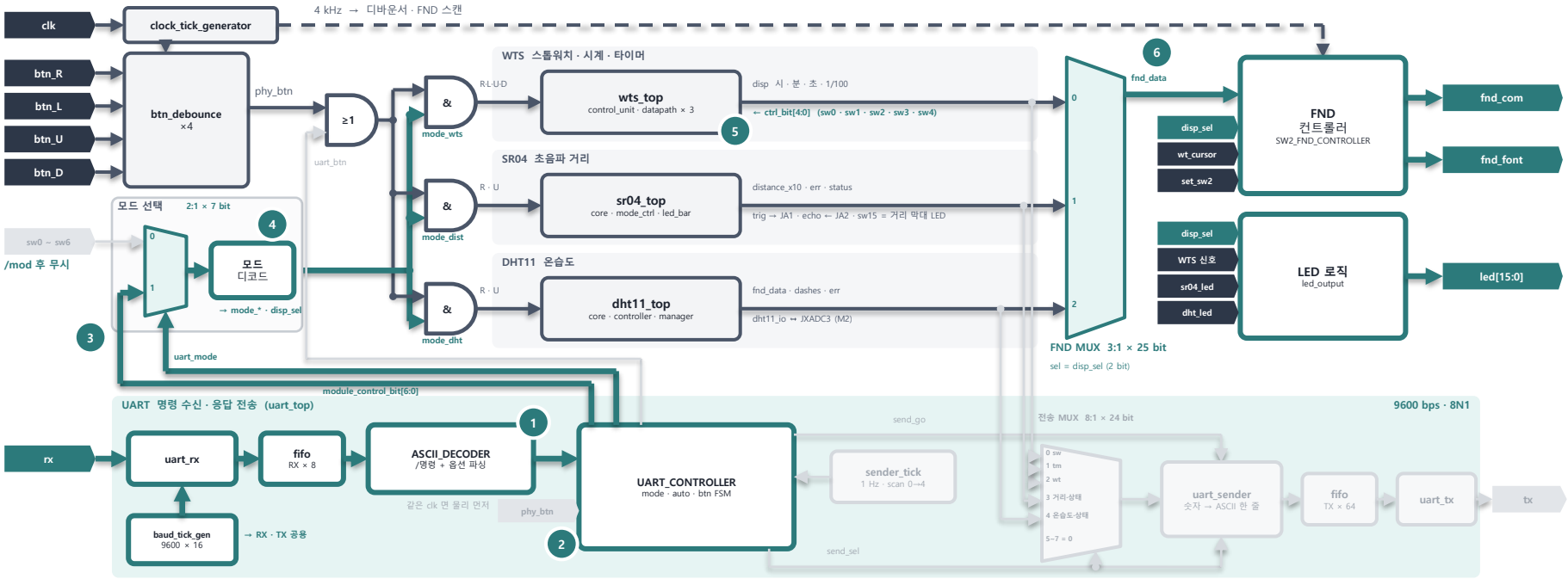
1. sw0 ~ sw6 → 모드 선택 MUX 로 입력 (스위치 쪽)
2. 모드 결정 : sw6 온습도 > sw5 거리 > 그 외 WTS (시계·스톱워치·타이머)
3. 버튼 4 개 → 디바운스 (누를 때 튀는 신호 제거)
4. ≥1 (OR) : 보드 버튼 + UART 버튼 → 하나로 합침
5. & (AND) : 버튼 4 개를 세 기능이 공유 → 지금 화면 기능만 통과
6. disp_sel → FND · LED 에 지금 화면 값 표시

경로 (예) sw6 = 1 · btn R → 모드 MUX → 모드 디코드 (온습도) → 버튼 디바운스 → OR → AND (온습도만) → dht11_top → FND 측정값 | 다음 : /mod

● 02 시스템 구조 · 명령별 흐름 – 이종혁

/mod 흐름 — 화면 · 버튼 대상 바꾸기

요약 **/mod 를 한 번 받으면 : 스위치 대신 UART 명령이 화면 · 버튼 대상 결정 (리셋 전까지)**



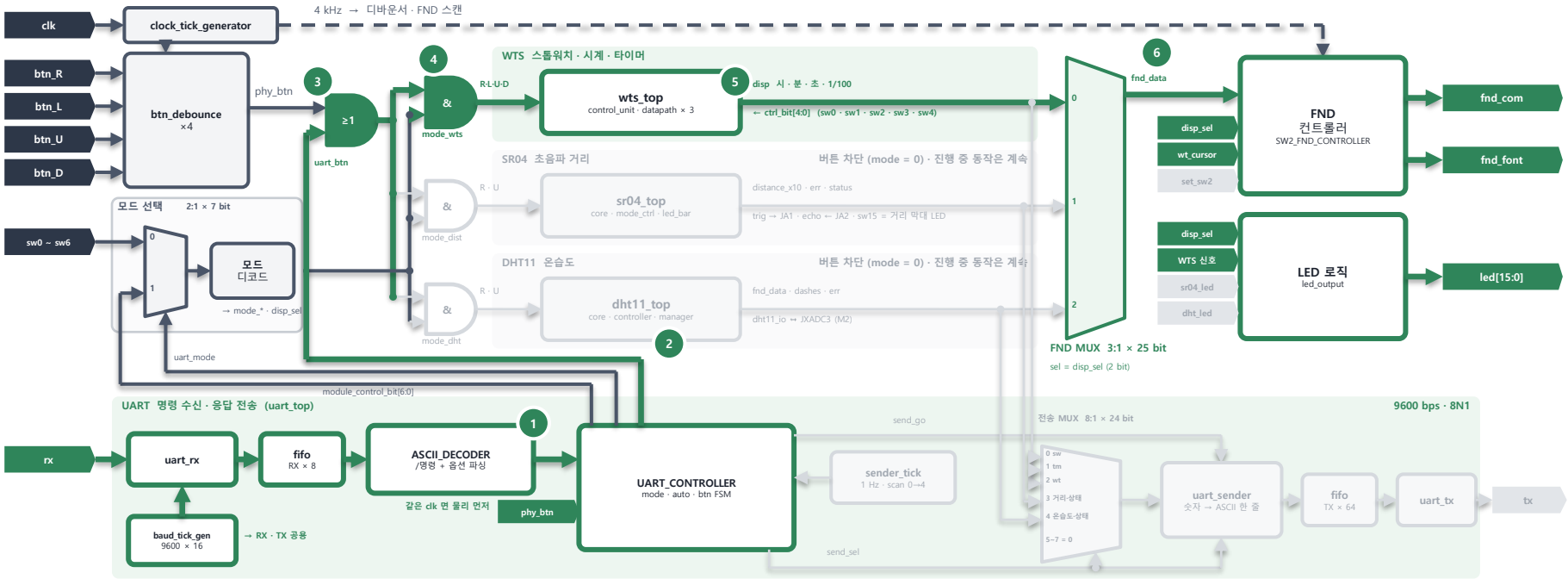
1. PC 입력 → 수신 → 디코더 : /mod + 화면 이름 4 글자
2. 컨트롤러 : 화면 이름 → 7 비트 모드 값으로 변환
3. uart_mode = 1 → MUX 가 스위치 대신 UART 값 선택
4. 모드 위 2 비트 → 버튼 받을 기능 결정
5. 모드 아래 5 비트 → 시계 · 스톱워치 · 타이머 세부 기능
6. disp_sel → FND · LED 화면도 같이 전환

경로 (예) PC /mod temp → 수신 · 디코더 → 컨트롤러 (7 비트) → 모드 MUX (UART 선택) → 모드 디코드 → 버튼 대상 · FND 화면 전환 | 다음 : /btn

● 02 시스템 구조 · 명령별 흐름 – 이종혁

/btn 흐름 ① WTS 화면 — /btn R · L · U · D

요약 /btn = PC 로 보드 버튼 누르기 · 먼저 /mod 로 고른 화면 기능만 반응 · 지금은 WTS 화면



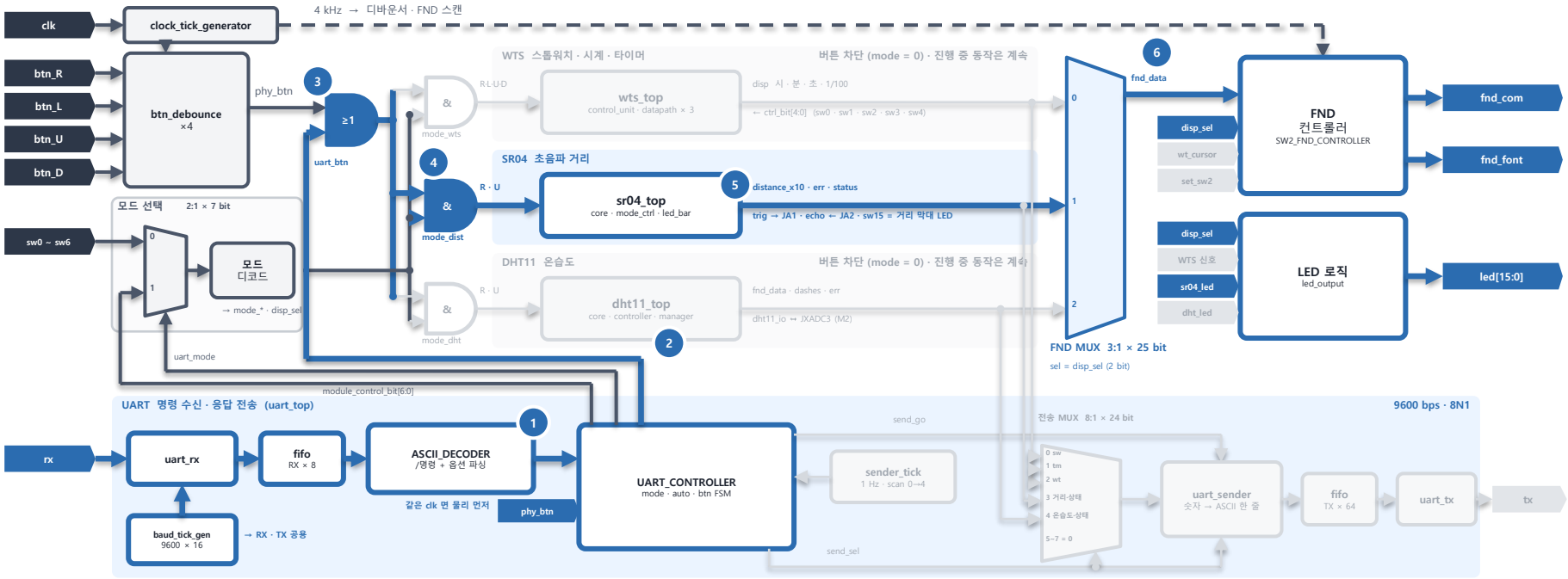
1. 디코더 : /btn + 버튼 이름 (R · L · U · D)
2. 컨트롤러 : 버튼 1 번 누른 것과 같은 펄스 생성
3. 보드 버튼과 합침 · 동시에 오면 보드 버튼 먼저
4. WTS 화면 → WTS 만 버튼 받음 · 거리 · 온습도 차단
5. 예 : R 스톱워치 시작 · L·R 시계 커서 · 타이머 +1 분
6. FND · LED : WTS 시간 표시

경로 (예) /mod s_ms 후 PC /btn R → 디코더 → 컨트롤러 (버튼 펄스) → OR → AND (WTS 만) → wts_top → FND 시간 | 다음 : 거리 화면

● 02 시스템 구조 · 명령별 흐름 – 이종혁

/btn 흐름 ② 거리 화면 — /btn R 측정 · /btn U 자동

요약 거리 화면 (/mod dist) 을 먼저 고른 뒤 같은 /btn → R · U 만 SR04 로 · 거리 0.1 cm 단위 표시 + 막대 LED



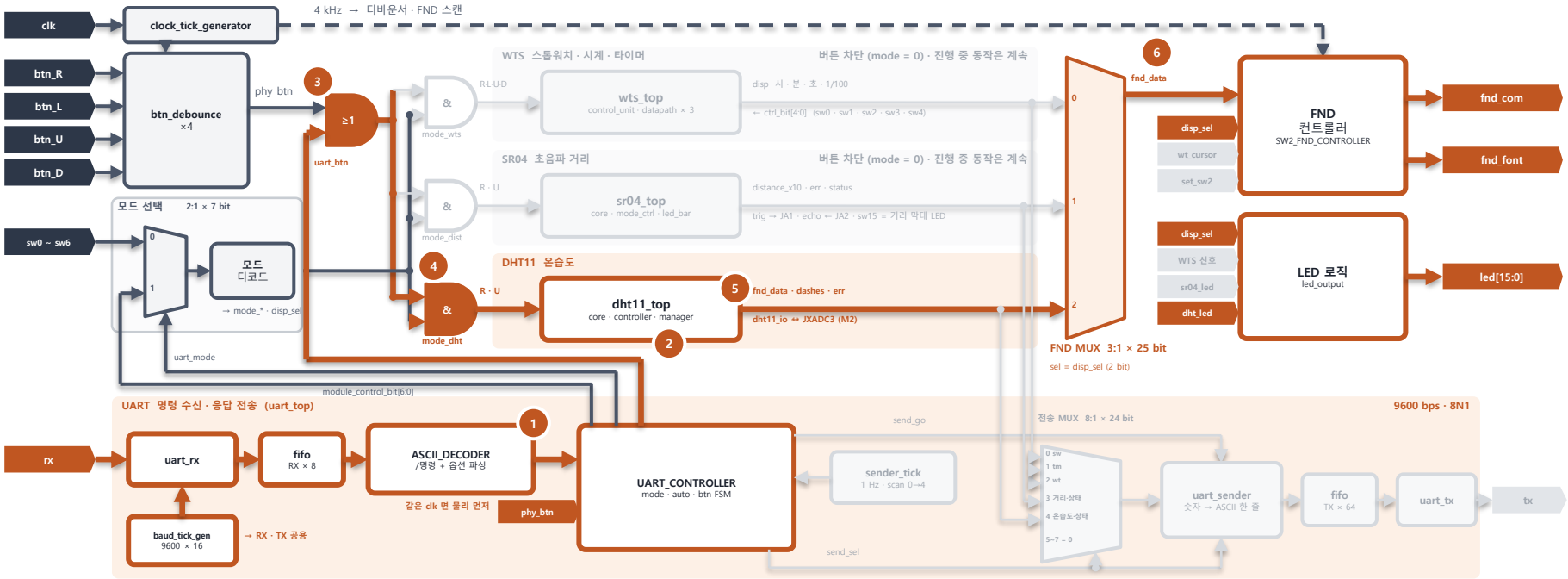
1. 디코더 : /btn R 또는 /btn U
2. 컨트롤러 : 버튼 1 번 펄스 생성
3. 보드 버튼과 합침 · 동시에 오면 보드 버튼 먼저
4. 거리 화면 → SR04 만 버튼 받음 · 나머지 차단
5. R = 1 회 측정 · U = 자동 (60 ms 마다) 켜기/끄기
6. FND : 352.7 cm · 실패 Err1~3 · LED 거리 막대

경로 (예) /mod dist 후 PC /btn R → 디코더 → 컨트롤러 (버튼 펄스) → OR → AND (거리만) → sr04_top → FND 352.7 · LED 막대 | 다음 : 온습도 화면

● 02 시스템 구조 · 명령별 흐름 – 이종혁

/btn 흐름 ③ 온습도 화면 — /btn R 측정 · /btn U 자동

요약 온습도 화면 (/mod temp) 을 먼저 고른 뒤 같은 /btn → R · U 만 DHT11 로 · 40 비트 받아 습도.온도 표시



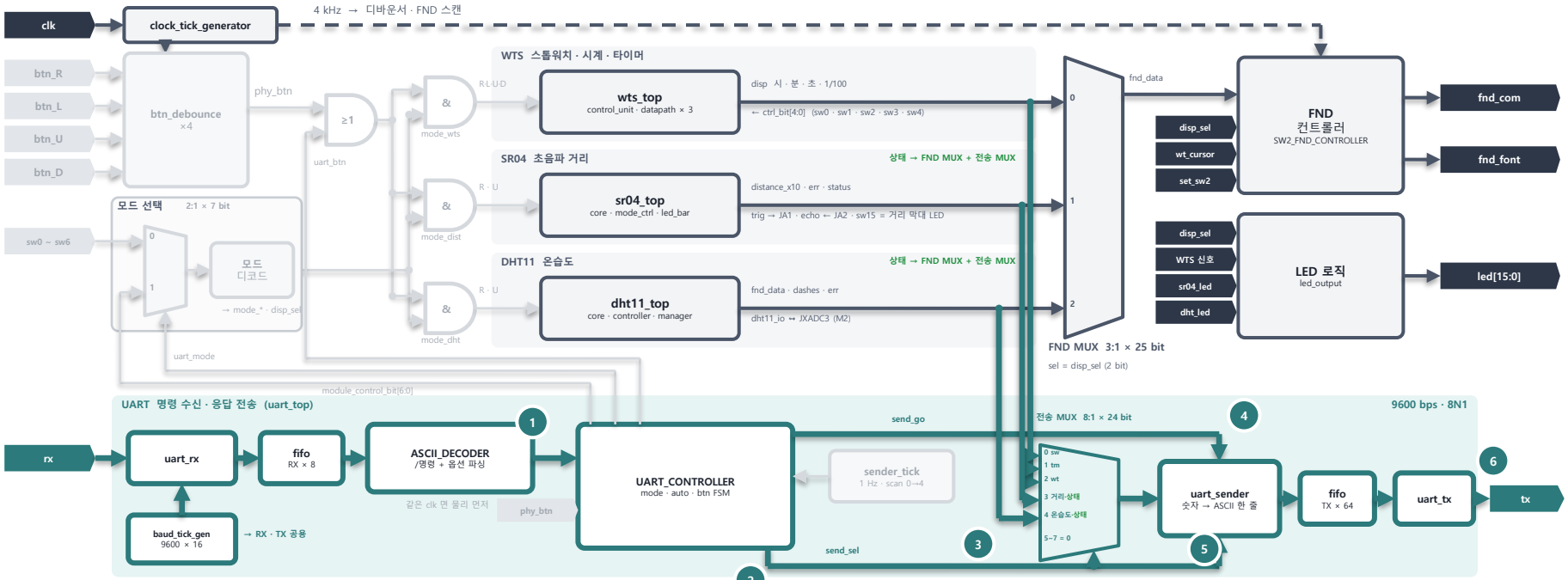
1. 디코더 : /btn R 또는 /btn U
2. 컨트롤러 : 버튼 1 번 펄스 생성
3. 보드 버튼과 합침 · 동시에 오면 보드 버튼 먼저
4. 온습도 화면 → DHT11 만 버튼 받음 · 나머지 차단
5. R = 1 회 측정 · U = 자동 (2 초마다) 켜기/끄기
6. FND : 55.24 (습도 55 % · 24 ℃) · 실패 Err

경로 (예) /mod temp 후 PC /btn R → 디코더 → 컨트롤러 (버튼 펄스) → OR → AND (온습도만) → dht11_top → FND 55.24 | 다음 : /get

● 02 시스템 구조 · 명령별 흐름 · v2 – 이종혁

/get 흐름 — 지금 화면 값 1 회 전송

요약 **/get = 지금 화면 값을 PC 로 한 줄 전송 · 센서 오류면 오류 글자도 전송**



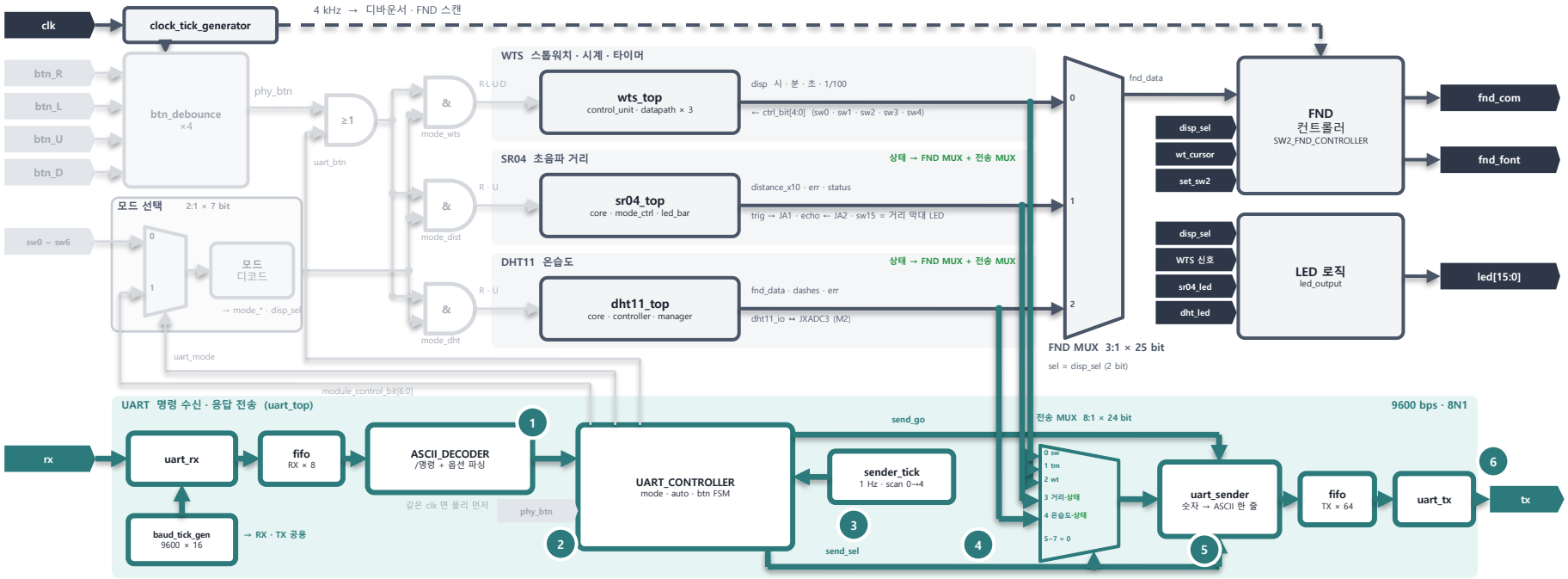
1. 디코더 : /get + Enter
2. 컨트롤러 : 지금 화면에 맞는 데이터 선택
3. 전송 MUX : 시간 · 거리 · 온습도 값 + 센서 상태
4. send_go → 그 순간 값을 붙잡아 둠
5. sender : 숫자 → 글자 (045cm · 55%24C · Err 2)
6. TX FIFO → uart_tx → PC · 자동 전송 중엔 무시

경로 (예) PC /get → 디코더 → 컨트롤러 (데이터 선택) → 전송 MUX → sender (글자 변환) → TX FIFO → PC 55%24C (오류 Err 2 · 측정 전 --%--C) | 다음 : /auto

● 02 시스템 구조 · 명령별 흐름 · v2 – 이종혁

/auto 흐름 — 1 초마다 자동 전송

요약 **/auto = 1 초마다 자동 전송 · 대상 하나 또는 all (5 종류 돌아가며)**



1 디코더 : /auto + 대상 3 글자

2 컨트롤러 : 자동 상태 · 1 초 tick 켜기

3 1 초마다 보낼 데이터 선택

4 전송 MUX → sender · 전송 중이면 저장 후 이어서

5 TX FIFO → uart_tx → PC 에 한 줄씩

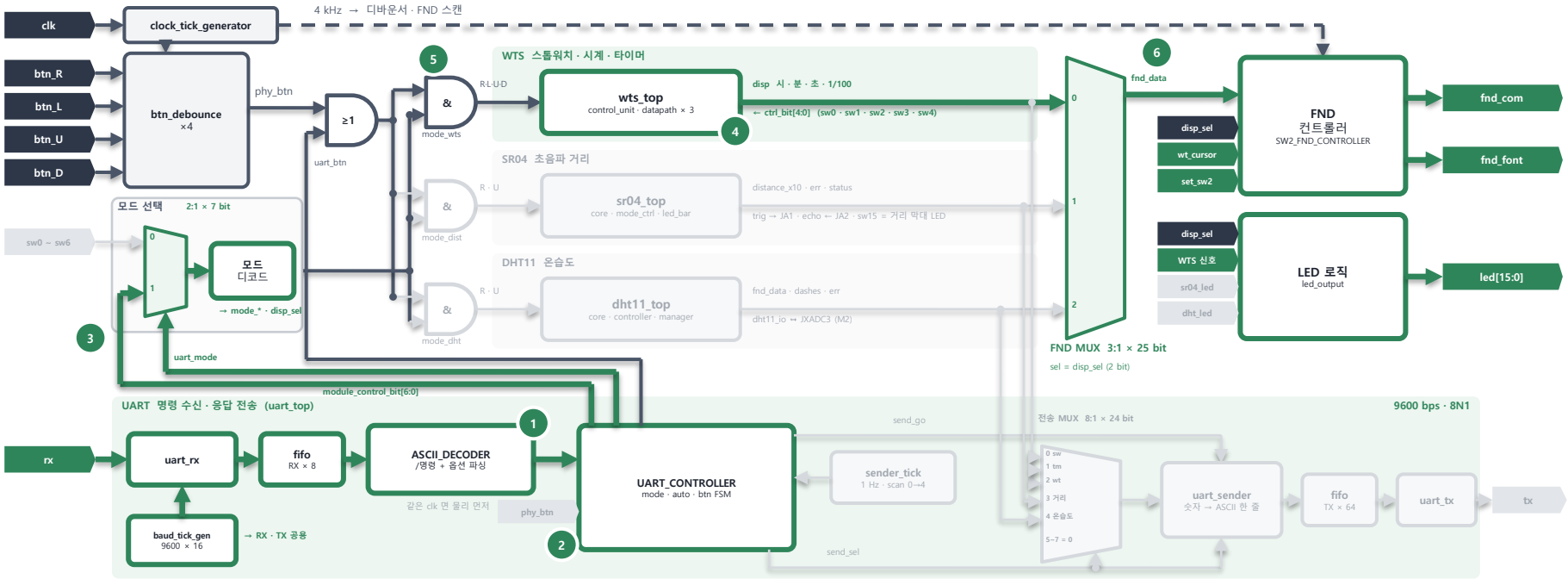
6 /auto off → 전송 멈춤 · FND 화면은 그대로

경로 (예) PC /auto all → 디코더 → 컨트롤러 (1 초 tick) → 전송 MUX → sender → TX FIFO → PC 에 한 줄씩 | 다음 : /set · /off

● 02 시스템 구조 · 명령별 흐름 – 이종혁

/set · /off 흐름 — 시계 시간 설정 · 12 / 24 시간

요약 /set · /off = 시계 설정 모드 켜기/끄기 · 12/24 시간 전환 · 숫자 변경은 /btn · /mod 받은 뒤에만 동작



1 디코더 : 명령 READ

2 /set w_hm → 설정 모드 켜기 · /off w_h → 끄기

3 /set 24h · 12h → 시간 표시 방식 (화면 바꿔도 유지)

4 모드 비트 → 시계 : 설정 모드 · 12/24 반영

5 /btn L · R = 커서 이동 · U · D = 숫자 ±

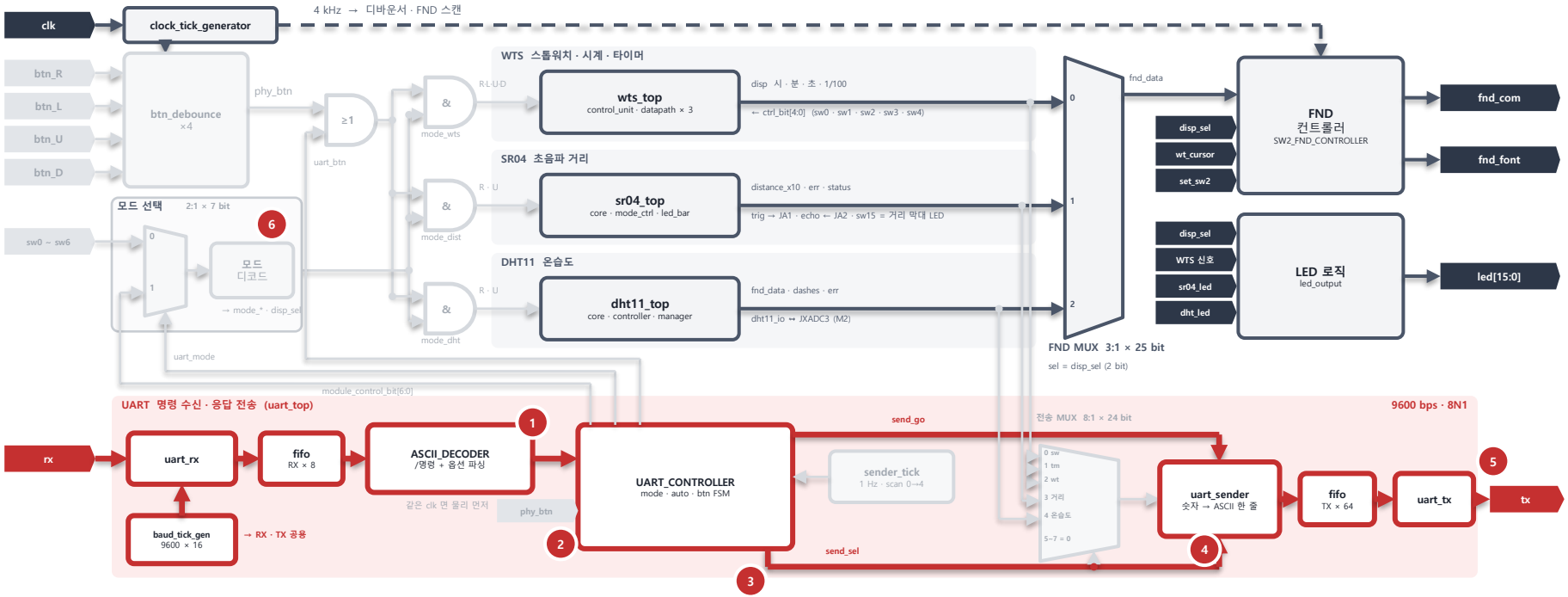
6 FND : 설정 중인 자리 0.5 초 깜빡 · LD15 = 오후

경로 (예) PC /set w_hm → 디코더 → 컨트롤러 (모드 비트) → wts_top 설정 모드 → /btn 으로 숫자 변경 → FND 깜빡 | 다음 : 오류

02 시스템 구조 · 명령별 흐름 – 이종혁

오류 흐름 — 형식이 틀리면 ERROR 응답

요약 형식이 틀린 명령 → 버리고 PC 에 ERROR 응답 · 보드 상태는 그대로



1. 틀린 형식 : 없는 명령 · 공백 없음 · 옵션 없음 → 버림
2. 컨트롤러가 오류 신호 기억
3. ERROR 를 가장 먼저 전송 · 전송 중이면 끝난 뒤
4. sender : ERROR 글자 만들기
5. TX FIFO → uart_tx → PC
6. 모드 · 버튼 · 화면 그대로 · 다음 / 부터 다시 받음

경로 (예) PC 틀린 명령 → 디코더 (버림 · 오류 신호) → 컨트롤러 (기억) → sender ERROR → TX FIFO → PC ERROR

● 02 시스템 구조 · UART 동작 – 정안성

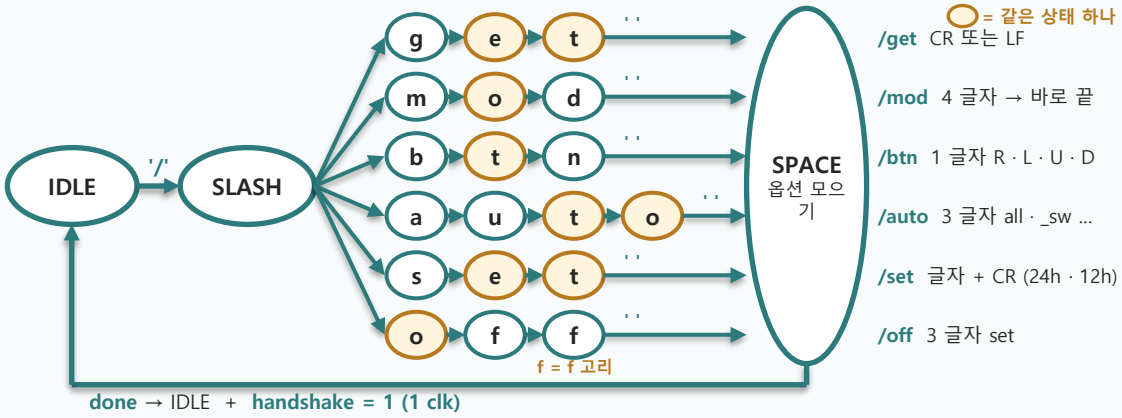
디코더 — 글자 하나에 상태 하나, '/' 부터 옵션까지 따라가며 명령을 판별

핵심

'/' → 명령 첫 글자 (g · m · b · a · s · o) 에서 **깃발** 1 개를 세움 → e · t · o 는 상태 하나를 같이 쓰고 깃발로 다음 갈래 결정
SPACE 에서 옵션 글자를 **data_reg 에 밀어 넣음** → 끝나면 **handshake 1 clk** (ascii 도 그 clk 만) · 틀린 글자면 **error 1 clk**

PC rx → uart_rx → FIFO RX → **DECODER** → CONTROLLER → SENDER → FIFO TX → uart_tx → PC tx

UART_ASCII_DECODER 상태도 (c_state) — 줄 = 명령별 길



= 같은 상태 하나

/get CR 또는 LF

/mod 4 글자 → 바로 끝

/btn 1 글자 R · L · U · D

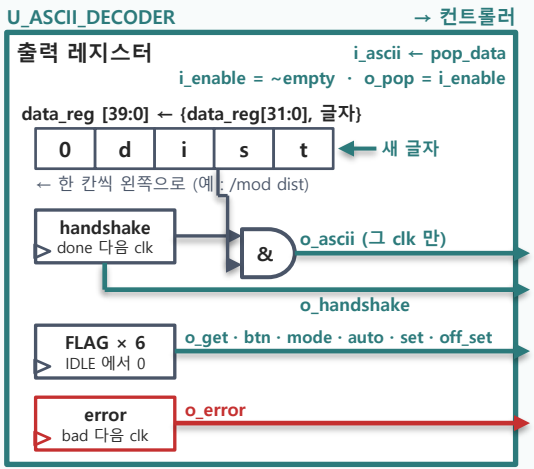
/auto 3 글자 all · _sw …

/set 글자 + CR (24h · 12h)

/off 3 글자 set

틀린 글자 (bad) → error = 1 (1 clk) · 깃발 · data_reg 지움 → 그 글자가 '/' 면 SLASH (바로 새 명령), 아니면 IDLE
IDLE 에서는 '/' 아닌 글자 (CR · LF 포함) 를 무시 — error 안 냄

U_ASCII_DECODER 출력 쪽 회로



예 /mod dist: / → SLASH · m → m · mode=1 · o → o · d → d · SP → SPACE · d → cnt 1 · i → cnt 2 · s → cnt 3 · t → done · handshake = 1

예 /mdo temp: / → SLASH · m → m · mode=1 · d → bad ('o' 자리) → error = 1 · IDLE · o SP t e m p → IDLE 이라 무시 (error 안 냄) → ERROR 응답 1 번

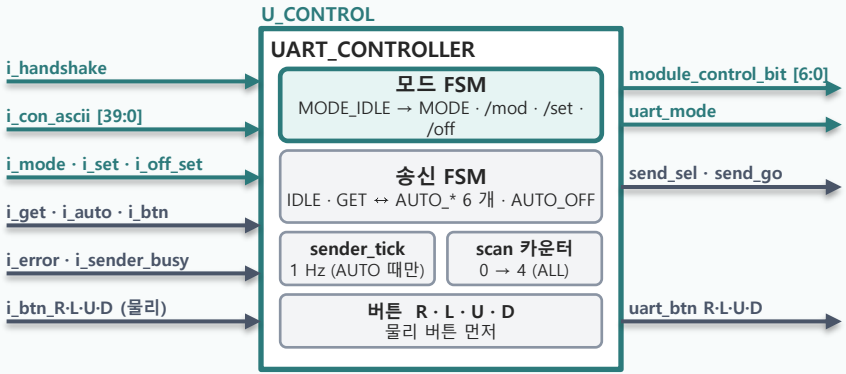
● 02 시스템 구조 · UART 동작 – 정안성

컨트롤러 · 모드 FSM — /mod 4 글자를 스위치 7 개 값으로 바꿈

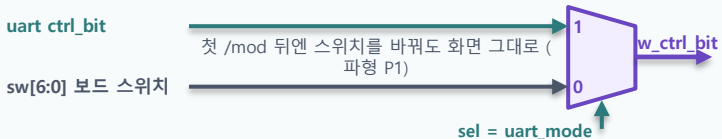
핵심

'/m' 을 받는 순간 **MODE_IDLE → MODE** (리셋 전까지 유지) · MODE 에서 handshake 마다 ascii 를 보고 **module_control_bit[6:0]** 를 정함
올바른 /mod 를 한 번 받으면 **uart_mode = 1** → Top 이 스위치 대신 이 값을 씀 · **bit3 (12/24 h) 는 /mod 로 안 바뀌고** /set 24h · 12h 만

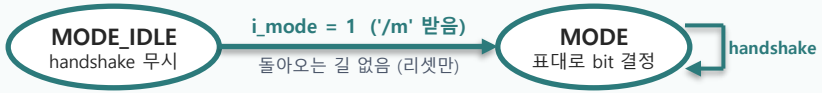
PC rx → uart_rx → FIFO RX → DECODER → **CONTROLLER** → SENDER → FIFO TX → uart_tx → PC tx



Top (merge_final_top) : 실제로 쓰는 모드 값



모드 FSM 상태도 (c_mode_state)



명령 · 옵션	6 온습도	5 거리	4 타이머	3 24h	2 설정	1 시:분	0 스톱	화면
/mod w_hm	0	0	0	유지	0	1	0	시계 시:분
/mod w_ms	0	0	0	유지	0	0	0	시계 분:초
/mod t_hm	0	0	1	유지	0	1	0	타이머 시:분
/mod t_ms	0	0	1	유지	0	0	0	타이머 분:초
/mod s_hm	0	0	0	유지	0	1	1	스톱워치 시:분
/mod s_ms	0	0	0	유지	0	0	1	스톱워치 분:초
/mod dist	0	1	0	유지	0	0	0	거리
/mod temp	1	0	0	유지	0	0	0	온습도
/set 24h · 12h				1 · 0				그 bit 만
/set (시계 화면)					1			시계 설정
/off set (설정 중)					0			설정 끝

module_control_bit = {dht_sw6, dist_sw5, timer_sw4, hour24_sw3, set_sw2, display_mode_sw1, mode_sw0} — 스위치 순서 그대로

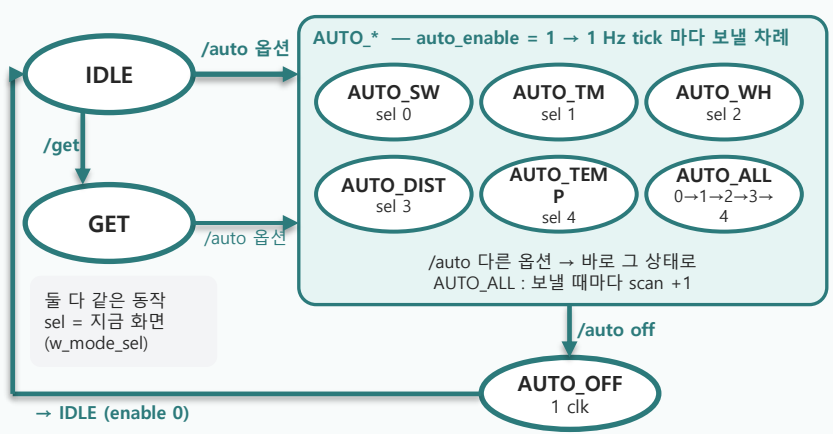
● 02 시스템 구조 · UART 동작 – 정안성

컨트롤러 · 송신 FSM — 무엇을 언제 보낼지, 겹치면 error 먼저

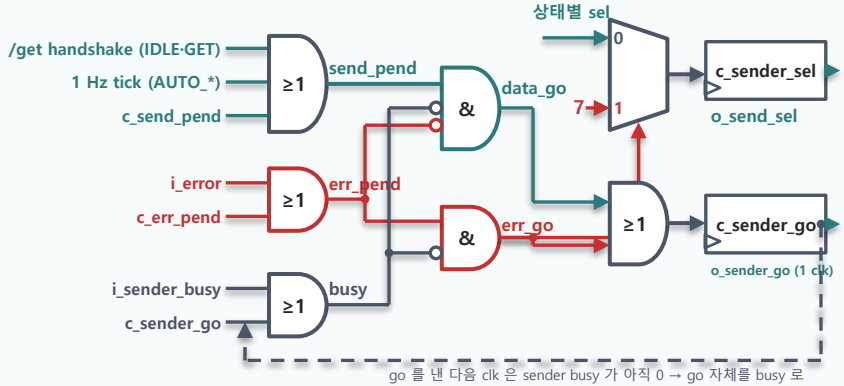
핵심 상태는 **무엇을** 보낼지 (IDLE · GET = 지금 화면, AUTO_* = 정한 데이터) · **pend 레지스터** 는 보낼 차례를 기억 · sender 가 비면 **send_go 1 clk** **error 가 최우선** (sel = 7, 데이터 차례는 pend 에 남음) · **busy = sender_busy | go** — go 다음 clk 에 한 번 더 쏘는 것을 막음

PC rx → uart_rx → FIFO RX → DECODER → **CONTROLLER** → SENDER → FIFO TX → uart_tx → PC tx

송신 FSM 상태도 (c_auto_state)



보낼지 판단 회로 (comb → c_sender_go · c_sender_sel FF)



보낸 clk 에 그 pend 는 0 으로 · 못 보내면 (busy) pend 유지 → busy 풀린 clk 에 바로

① error 최우선

data_go 조건에 ~err_pend
→ 같은 clk 이면 ERROR 먼저
데이터는 pend 에 남았다 이어서

② 두 번 쏘지 않기

busy = sender_busy | go
go 바로 다음 clk 까지 막음

③ AUTO_ALL 순환

실제로 보낸 clk 에 scan_tick
sel 0 → 1 → 2 → 3 → 4 → 0
(파형 P3 : 1 s 간격)

④ 버튼

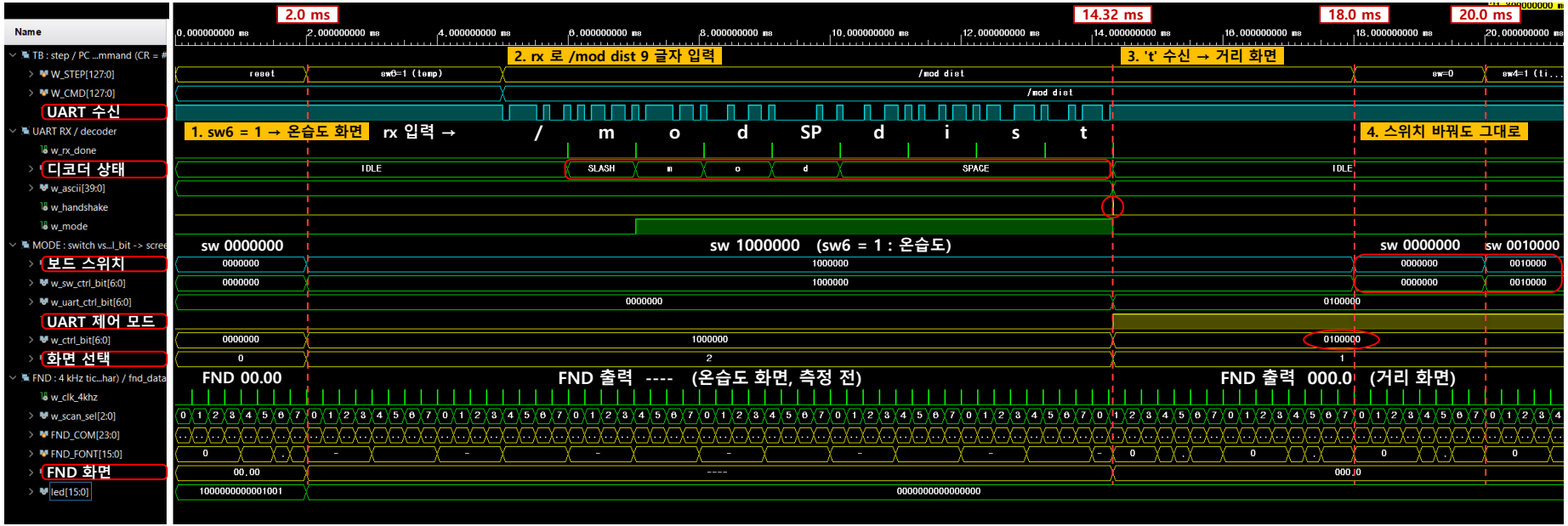
/btn R → c_btn_R = 1
물리 버튼 없는 clk 에 1 clk 출력
물리 버튼 있으면 기다림

● 03 Top · 시뮬레이션 ① 화면 제어 /mod – 정안성

P1 /mod dist — 스위치에서 UART 로 화면 제어권 전환

결론

첫 /mod 뒤 **w_uart_mode 0 → 1** → w_ctrl_bit = UART 값 **0100000** (거리 화면)
/mod dist 9 글자 → 마지막 글자 't' 수신 (**14.32 ms**) 에서 disp_sel **2 → 1**, FND **---- → 000.0**
그 뒤 스위치를 바꿔도 **w_ctrl_bit · disp_sel 그대로** → 리셋 전까지 UART 가 화면 결정

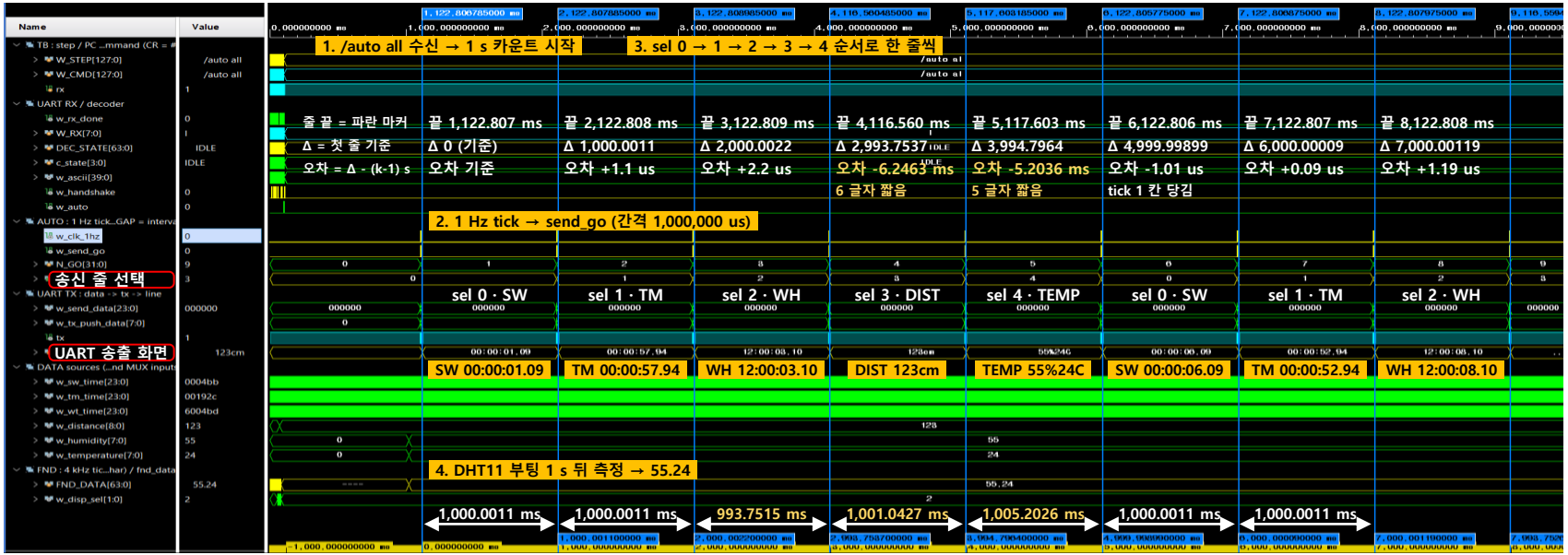


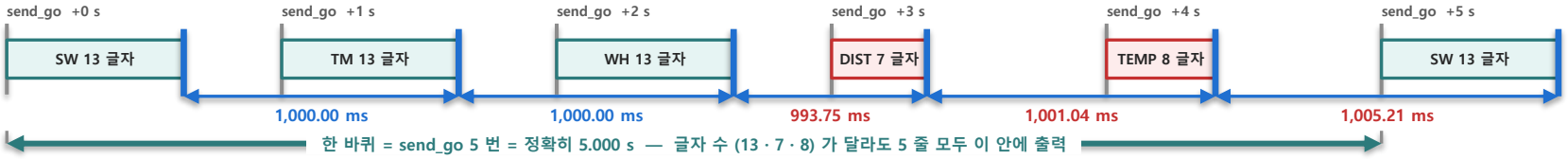
● 03 Top · 시뮬레이션 ② 송신 /auto – 정안성

P3 /auto all — 1 Hz 마다 전송 MUX 입력 5 개를 차례로

결론

send_go 간격 **100,000,000 clk = 1.000 000 s** (정확히 1 Hz), o_send_sel **0 → 1 → 2 → 3 → 4 → 0** — 한 바퀴 5 줄 = **5 s**
SW **00:00:01.09** → TM **00:00:57.94** → WH **12:00:03.10** → **123cm** → **55%24C** : 줄마다 1 s 흐른 값



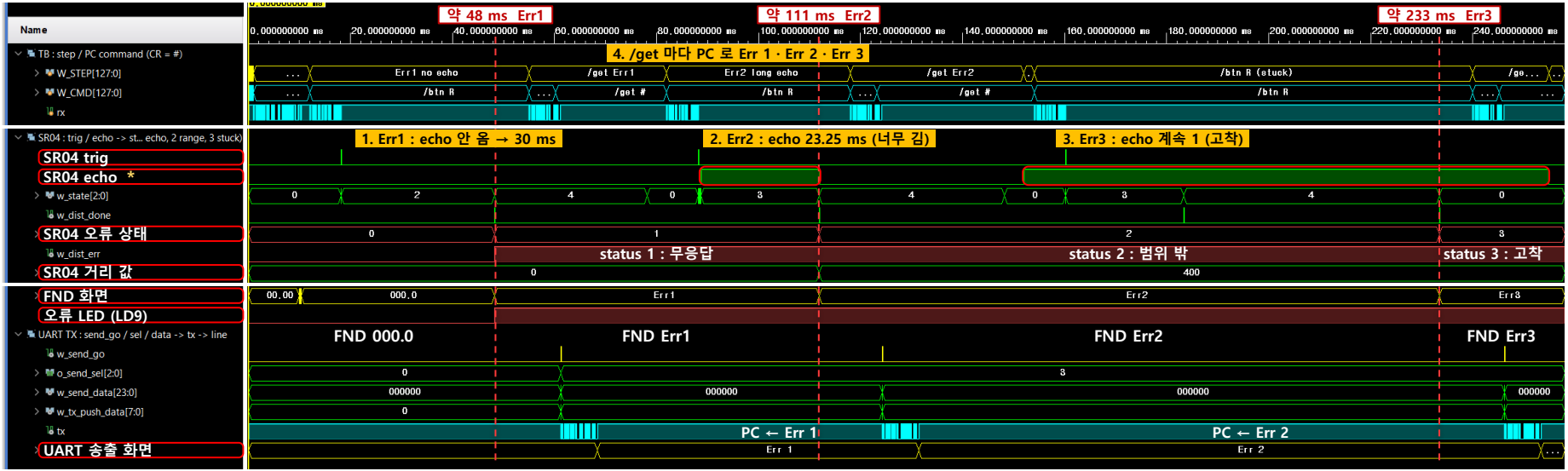


● 03 Top · 시뮬레이션 센서 오류 SR04 – 정안성

P5 SR04 Err1 · Err2 · Err3 — FND · LED · PC 가 같은 오류 번호

결론

Err1 무응답 : trig↓ + **30 ms** → status **1** · FND **Err1** · LD9 → /get **Err 1**
Err2 범위 밖 : echo 23.25 ms > **23.2 ms (400 cm)** → status **2** · distance 400 · FND **Err2** → /get **Err 2**
Err3 echo 고착 : 23.2 ms 에 Err2 → **73.2 ms** 에 status **3** · 강제 IDLE (w_state 0) → FND **Err3**



Err1 무응답
trig↓ + 30 ms 에 판정 · LD9 · PC Err 1

Err2 범위 밖
echo↑ + 23.2 ms (400 cm) 에 판정 · PC Err 2

Err3 echo 고착
trig↓ + 73.2 ms 에 판정 · 강제 IDLE · PC Err 3

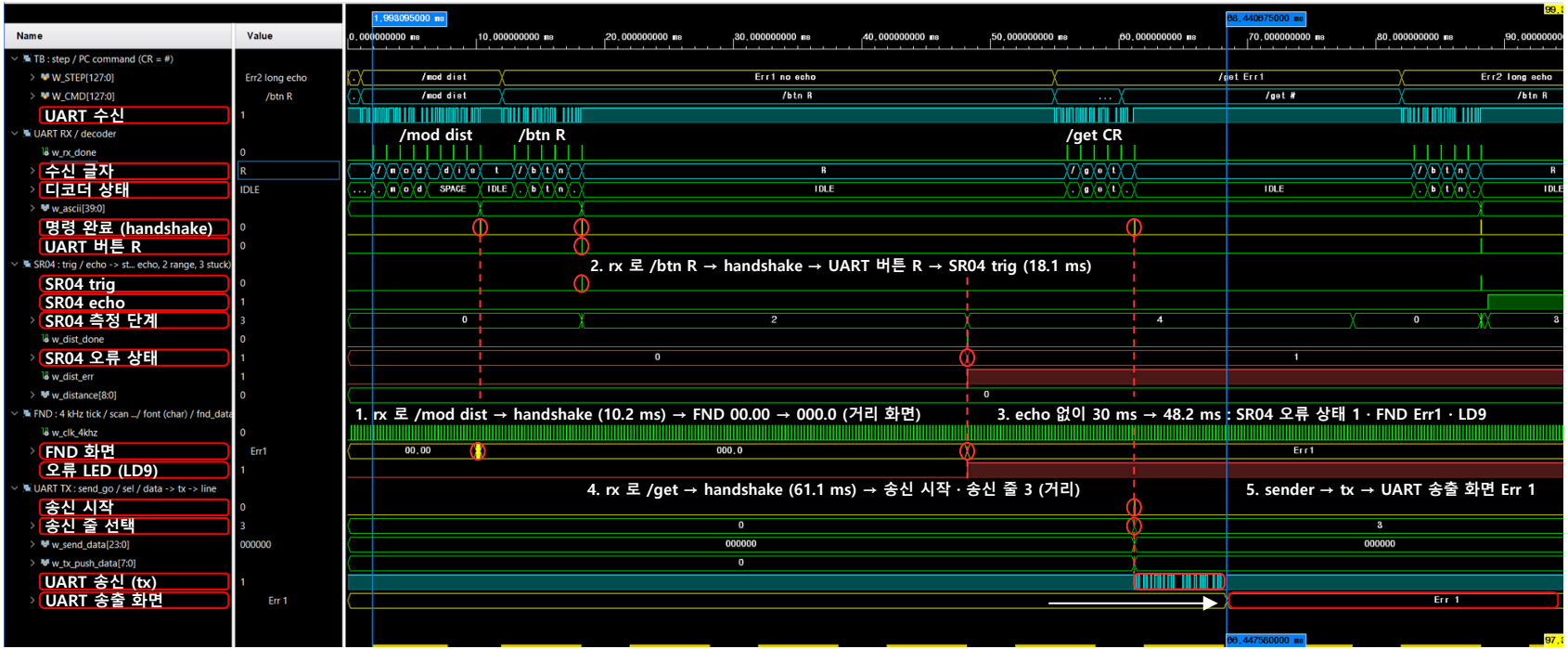
* echo 는 실제 센서 대신 TB 센서 모델 (tb_merge_harness) 이 만든 핀 파형 : trig↓ 뒤 0.5 ms 에 echo = 1 → 거리 × 58 us 동안 유지 (예 : 123 cm = 7,134 us)
Err1 = echo 를 안 줌 · Err2 = 폭 23,250 us 로 길게 · Err3 = TB 가 trig 전부터 echo = 1 로 고정 (152 ms) → 255 ms 에 풂

● 03 Top · 시뮬레이션 센서 오류 SR04 – 정안성

P5-1 SR04 Err1 무응답 — echo 가 안 오면 trig ↓ + 30 ms 에 Err1

결론

/btn R 로 측정 시작 → trig (**18.1 ms**) → echo 없이 **30 ms** 기다림 → **48.2 ms** 에 w_dist_status **1** · w_dist_err 1
같은 순간 FND 000.0 → **Err1** · LD9 켜짐 → 이어진 /get 응답 W_LINE = **Err 1** (68.4 ms)

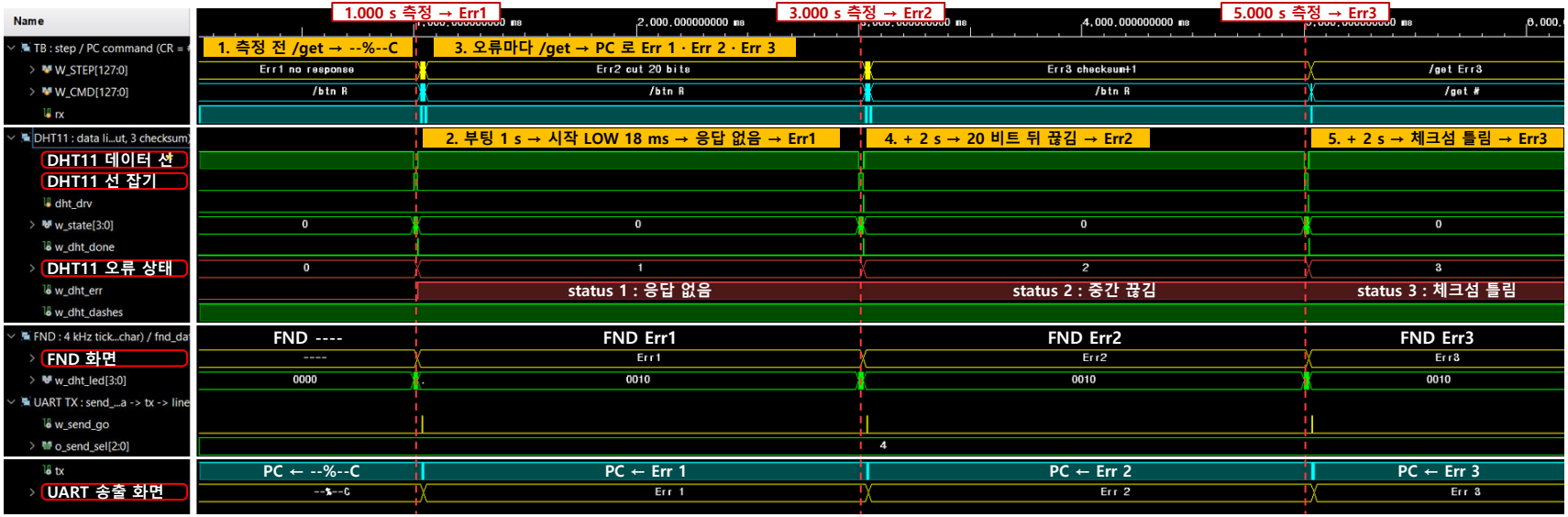


● 03 Top · 시뮬레이션 센서 오류 DHT11 – 정안성

P6 DHT11 Err1 · Err2 · Err3 — 2 s 간격 측정, 오류 번호 PC 전송

결론

부팅 **1 s** 뒤 첫 측정, 이후 이전 시작 + **2 s** 마다 (1.0 · 3.0 · 5.0 s) 시작 LOW 18 ms
응답 없음 → status **1** · 20 비트 뒤 끊김 → **2** · 체크섬 틀림 → **3** : FND **Err1 → Err2 → Err3**
PC 도 **--%--C → Err 1 → Err 2 → Err 3** (측정 전 · 오류 때 숫자 대신)



측정 전
0 ~ 1 s · PC --%--C

Err1 응답 없음
1.018 s · 선 놓은 뒤 응답 없음 · PC Err 1

Err2 중간 끊김
3.020 s · 20 비트 뒤 0.2 ms 끊김 · PC Err 2

Err3 체크섬
5.022 s · 40 비트 다 받았지만 체크섬 틀림 · PC Err 3

* dht11_io 의 센서 응답은 TB 센서 모델 (tb_merge_harness) 이 만든 파형 : FPGA 가 선을 18 ms LOW 로 잡았다 놓으면 30 us 뒤 LOW 80 · HIGH 80 us + 40 비트 (0 = HIGH 26 us, 1 = 70 us)
Err1 = 응답 안 함 · Err2 = 20 비트만 보내고 멈춤 · Err3 = 체크섬에 +1

● 03 Top · 시뮬레이션 검증 – 정안성

Top TB — 그룹 TB 12 개 · 92 시나리오 · 검사 305 개, FAIL 8

결론

그룹 TB 12 개 (A ~ L) : 검사 305 → **PASS 297 · FAIL 8** + 통합 TB (겹치지 않는 것) **292 / 292**
FAIL 8 = 스펙과 다르게 동작 → **고칠 곳** (오른쪽 표 — 나머지 297 검사는 모두 PASS)

대표 시나리오	기준 설정 근거	검사	검사 결과
전원 · 동작 중 리셋	해제 2 clk 뒤 초기값 · 리셋 즉시 핀 안전 상태	14	14 / 14
채터링 · 1.5 / 2.25 ms 경계 · 화면 게이팅	4 kHz × 8 샘플 = 2 ms → 1.5 무시 · 2.25 통과	14	14 / 14
스위치 우선순위 · /mod 8 종 · 틀린 /mod	README 화면 진리표 · /mod 표, 1 us 안 전환	30	30 / 30
시작 정확도 · 23:59:59 wrap · 랩	100 Hz tick → R 뒤 정확히 10.00 ms · 24 h wrap	15	14 / 15 · FAIL 1
12/24 경계 · 설정 커서 · 값 wrap	자정 = 12 AM · 자리 단위 wrap (캐리 없음)	23	22 / 23 · FAIL 1
10 초 알람 · 포화 · 다른 화면 알람	L 뒤 10.000 s (± 5 clk) · 최대 59:59	16	16 / 16
명령 24 개 · 형식 오류 10 종 · baud ± 2 %	README 명령 표 · ERROR 1 줄 · UART ± 2 %	52	50 / 52 · FAIL 2
/get · /auto 1 Hz · ERROR 우선 · /btn	9600 baud 13 B = 13.6 ms · 1 Hz 간격 ± 10 us	31	29 / 31 · FAIL 2
거리 5 점 · Err1 ~ 3 · 자동 60 ms	데이터시트 58 us/cm · 한계 23.2 ms · 60 ms	26	26 / 26
비트 경계 · 체크섬 · Err1 ~ 3 · 자동 2 s	데이터시트 비트 40 us 기준 · 체크섬 mod 256	30	29 / 30 · FAIL 1
거리 · 온습도 FND · LED 7 화면	README LED 표 (0x8009 ...) · 칸당 250 us 스캔	28	28 / 28
배경 동작 · 동시 부하 · 랜덤 리셋 20	화면 밖도 계속 · 누락 · hang 0 · 매번 초기값	26	25 / 26 · FAIL 1
통합 TB : 랜덤 명령 120 · MUX 100 번	v2 기대값과 매 줄 비교 (MISMATCH 0)	292	292 / 292

FAIL	상황	스펙	실제 동작
D7-2	랩 표시 중 /get	진행값	**랩 값 전송**
E6-1	시계 초 화면 + 설정 → U	분 그대로	**분 +1 (안 보임)**
G3-3	/set abcd (시.분 화면)	변화 없음	**설정 비트 켜짐**
G7-2	/off 오타 뒤 /off w_h	설정 꺼짐	**ERROR · 설정 그대로**
H7-2	/auto off 뒤 바로 /auto	첫 줄 1 주기 뒤	**0.54 주기 뒤**
H9-1	/get 20 번 연속	20 줄	**15 줄 (합쳐짐)**
J14-2	습도 ≥ 100 에서 /get	숫자 2 자리	**":0%24C"**
L6-1	캐리 clk 에 보낸 줄	00:00:01.00	**00:00:00.00**

03 Top · 시뮬레이션 검증 – 정안성

Top TB 신뢰성 — 버그 9 개를 넣어 보니 모두 FAIL

결론

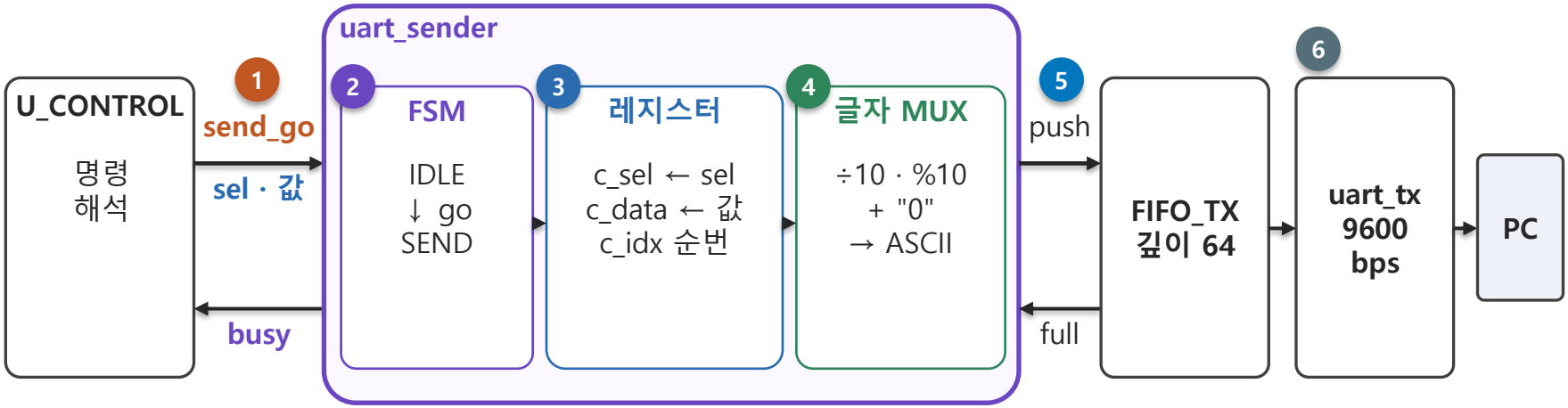
버그를 **하나씩 소스 사본에 넣고** 같은 Top TB 를 다시 돌림 → **9 / 9 검출** (원본에 없던 FAIL 이 새로 나옴)
처음 고른 TB 가 놓친 2 개 (B3 · B9) 는 **다른 TB 가 잡음** → 그룹 TB 끼리 서로 보완

넣은 버그	이유	결과 (어떻게 FAIL 이 났는지)	검출 결과
B1 디바운서 8 샘플 AND → OR (튐 통과)	채터링 거르기가 깨지면 잡는지	1.5 ms 짧은 누름에도 펄스 (기대 0 개) 누른 뒤 0.21 ms 에 펄스 (기대 1.75 ~ 2 ms)	**FAIL 6 검출**
B2 모드 MUX : /mod 무시 (스위치만)	UART 제어권 전환이 막히는 실수	/mod 7 종 뒤 화면 그대로 (ctrl 000_0000) 스위치가 계속 화면 · LED 결정	**FAIL 12 검출**
B3 스톱워치 · 타이머 tick 1 clk 짧음	10 ns 시간 오차도 잡는지	tb_D : 첫 tick 만 봐서 PASS tb_F : 10 초 누적 알람 9.990 ms ≠ 10.000 ms	**0 → FAIL 1** **tb_F 검출**
B4 12 시간 표시 : 0 시를 12 대신 00	자정 (12 시간 경계) 처리 실수	0 시 30 분 → "12:30" 대신 "00:30" 자정 넘어갈 때도 "00"	**FAIL 2 검출**
B5 SR04 400 cm 한계 100 us 늦음	범위 경계 1.7 cm 차이도 잡는지	echo 23,250 us → Err2 대신 "400cm" Err2 · Err3 10 us 늦음 (2.330 ≠ 2.320 ms)	**FAIL 3 검출**
B6 DHT11 체크섬 검사 빠짐	깨진 데이터를 정상 표시하는 실수	체크섬 틀린 응답 → Err3 대신 FND "55.24" UART 도 "55%24C"	**FAIL 2 검출**
B7 FND 거리 소수점 안 켜짐	화면 글자 한 칸 (점) 차이	"000.0" → "0000" · "352.7" → "3527" (점 없음)	**FAIL 4 검출**
B8 시계 LED 에 스톱워치 신호 연결	LED 배선 한 비트 실수	시계 화면 LED 8009 → 8008 스톱워치 화면 LED 000a → 000b	**FAIL 4 검출**
B9 UART 거리 백의 자리 계산 오류	송신 글자 자릿값 계산 실수	tb_H : /get 거리 0 cm 라 PASS tb_I : "110cm" ≠ "010cm" · 100 cm → ":00cm"	**0 → FAIL 4** **tb_I 검출**

버그 하나씩 FINAL/sim_file/mutants/<ID> 사본에 넣고 run_mutant.bat 로 그 그룹 TB 전체 실행 · 숫자 = 원본에 없던 새 FAIL 수 (원본 FAIL 8 개는 뺌) · FAST = 1 (시간 단축) 그대로

UART sender — send_go 한 번에 응답 한 줄을 PC 로

요약 **PC 터미널은 글자 (ASCII) 만 읽음 → 2 진수 값을 글자 한 줄로 바꿔 넘기는 블록이 필요**



1 send_go
U_CONTROL → sel · 값 24

2 FSM
IDLE → SEND

3 레지스터
sel · 값 저장

4 글자 MUX
숫자 → ASCII

5 push
1 클럭 1 글자 → FIFO

6 uart_tx
9600 bps → PC

왜 필요한가

① **sender :** 보드 값은 2 진수 (37 초 = 100101) → PC 에 "37" 로 보이려면 글자 "3" · "7" 로 바꿔야 함
② **FIFO_TX :** sender 는 한 줄을 130 ns 에 넣지만 선으로는 13.5 ms → 그 사이 글자를 받아 둠
③ **uart_tx :** 한 글자를 PC 와 약속한 9600 bps 8N1 로 1 비트씩 내보냄

sender 규칙 3 가지 — 응답 줄이 섞이거나 빠지지 않게

요약 **새 명령이 끼어들 때 · FIFO 가 찰 때 · 보낼 값이 바뀔 때**

1 보내는 중 새 send_go 가 오면

→ 무시 → 다른 줄이 끼어들지 않음

구현 : FSM (SEND 상태에서 go 무시) · 확인 : edge case ① 파형

2 FIFO_TX 가 꽉 차면

→ 그 글자에서 멈췄다 이어서 → 손실 0

구현 : push = SEND & ~full · 확인 : edge case ② 파형 (부록)

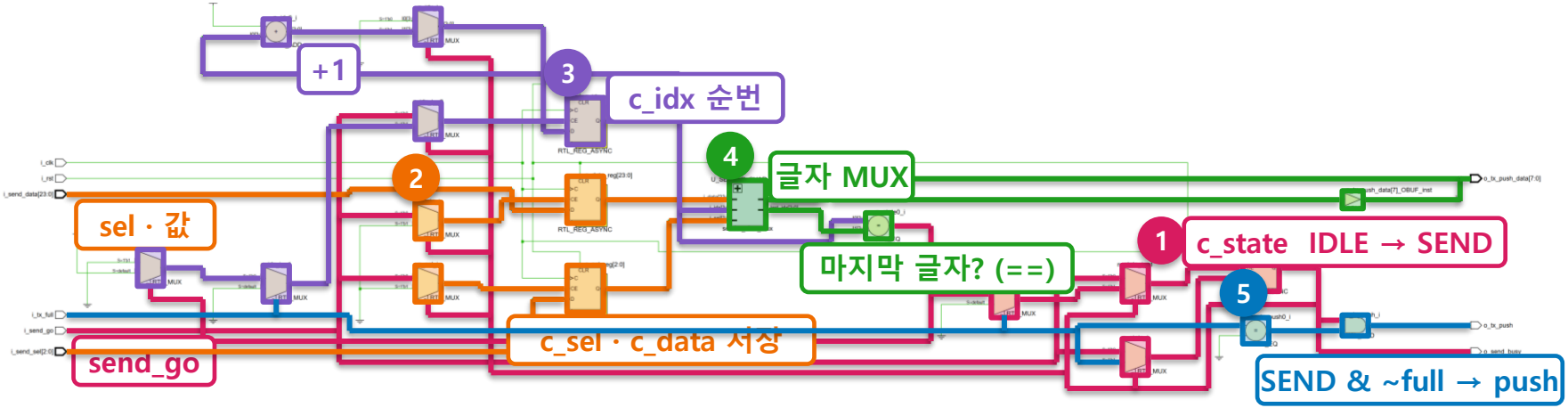
3 보내는 도중 바깥 값이 바뀌면

→ go 순간 저장한 값으로 끝까지 → "12:00" + ":00.00" 처럼 섞인 줄 없음

구현 : 레지스터 c_data · 확인 : uart_sender TB (보내는 중 값 바꾸기)

sender RTL — uart_sender : 번호 순서 = 데이터 흐름

요약 send_go → 상태 SEND · 값 저장 → c_idx 로 글자 하나씩 → 글자 MUX → SEND & ~full 일 때 push



1 상태 c_state
go → SEND · 마지막 글자 → IDLE

2 sel · data
go 때만 저장

3 c_idx
몇 번째 글자 (클럭마다 +1)

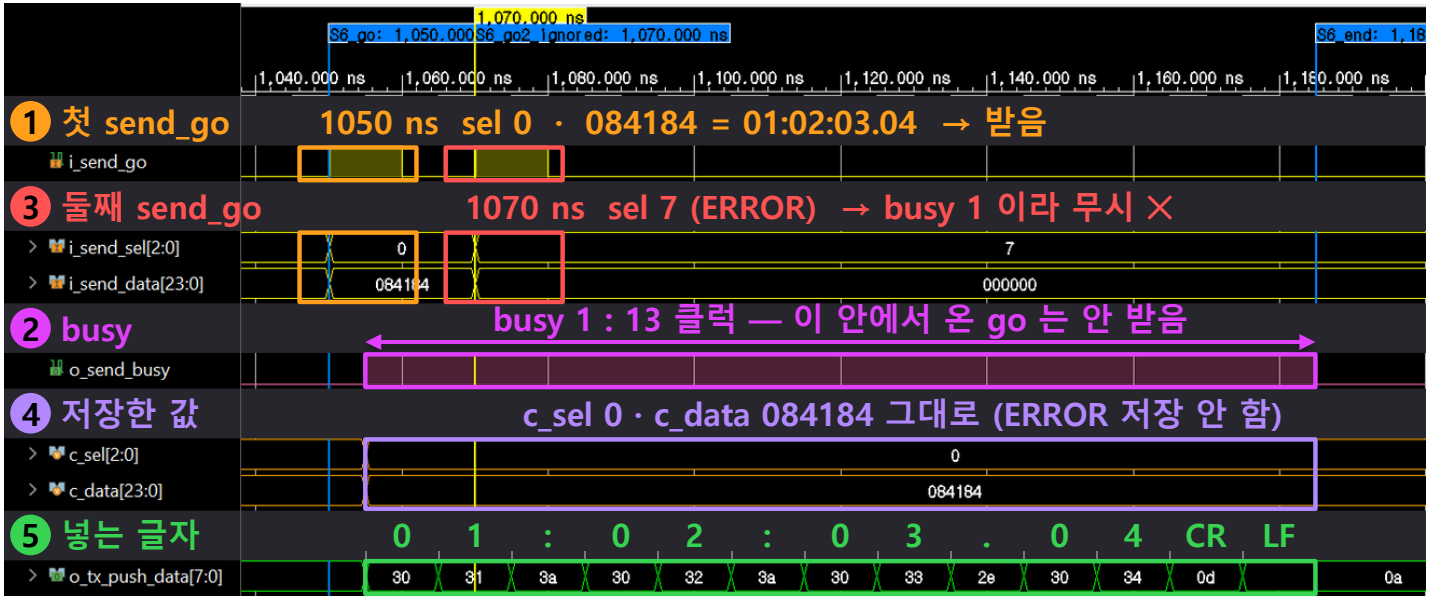
4 글자 MUX
글자 · 마지막 글자 비교

5 o_tx_push
SEND & ~full

Vivado RTL Schematic · 색 선 = 그 번호의 데이터가 지나는 선 · 레지스터 4 개 모두 비동기 rst · 래치 0

sender edge case ① — 보내는 중 (busy 1) 에 온 send_go 는 무시

요약 **busy 1 동안 온 send_go (sel 7 ERROR) 는 무시 → 01:02:03.04 글자 13 개 그대로**



1 첫 go
1050 ns · sel 0

2 busy 1
13 클럭 동안

3 둘째 go
1070 ns sel 7 무시

4 저장한 값
084184 유지

5 글자
01:02:03.04

결과
13 / 13 · ERROR 안 섞임

sender TB — 글자 MUX 부터 팀 Top 까지 모두 불일치 0

요약 **글자 MUX 만 → sender → + FIFO · uart_tx → 팀 Top 전체 순으로 넓혀 가며 검증**

검증 대상	시나리오	기준 설정 근거	결과
sender_char_mux	시간 · 거리 · 온습도 11 줄	최소 · 최대값에서 자리 · 0 채움	**11 / 11 줄**
	쓰지 않는 sel 5 · 6 · 7	정의 안 된 형식이 오면	**ERROR 3 / 3**
uart_sender	형식마다 한 줄 (4 종)	기본 동작 : 형식마다 다른 글자 수	**busy = 글자 수**
	FIFO 꽉 참 20 클럭	규칙 ② : PC 가 느려 FIFO 가 찰 때	**손실 0**
	보내는 중 새 send_go	규칙 ① : 명령이 겹칠 때	**줄 안 섞임**
	보내는 중 바깥 값 바뀜	규칙 ③ : 보내는 중 시계가 넘어갈 때	**go 순간 값**
	5 종 연달아	가장 긴 연속 전송 (54 글자)	**54 / 54**
	첫 글자 전 · 마지막 글자 전 꽉 참	꽉 참이 줄 처음 · 끝에 걸리는 경계	**안 빠짐**
	마지막 글자 클럭에 send_go	busy 가 내려가는 바로 그 클럭	**무시**
	보내는 중 리셋	리셋 뒤 멈춤 · 남은 글자 없는지	**바로 IDLE**
	무작위 200 줄 + 무작위 꽉 참	사람이 미리 못 떠올린 조합	**불일치 0**
sender_tx_top	한 줄 · 5 종 연달아	FIFO · uart_tx 까지 붙인 실제 출력	**67 / 67**
merge_final_top	시나리오 20 개 (랜덤 명령 120)	팀 모듈과 연결된 실제 명령 · 센서 오류	**558 / 558**

xsim 100 MHz · 시간 0 · 23:59:59.99, 거리 0 · 400 cm, 습도 · 온도 0 · 99 포함

sender TB 신뢰성 — 버그 8 개를 일부러 넣어 보니 모두 FAIL

요약 **단위 TB = Top 에선 안 생기는 경계 상황 · Top TB = 실제 연결 · 실제 입력 → 서로 못 잡는 걸 보완**

#	일부러 넣은 버그	깨뜨린 기능 / 이유	단위 TB	Top TB	잡은 쪽
B1	busy 중 send_go 를 받음	규칙 ① 겹친 명령 무시	FAIL 9	0	단위
B2	FIFO 꽉 차도 push	규칙 ② 꽉 차면 멈춤	FAIL 538	0	단위
B3	go 순간 값을 저장 안 함	규칙 ③ 값 저장	FAIL 16	FAIL 73	둘 다
B4	마지막 글자 하나 덜 보냄	줄 끝 판단	FAIL 465	FAIL 113	둘 다
B5	새 줄 글자 번호 안 돌림	다음 줄 시작	FAIL 143	FAIL 108	둘 다
B6	거리 백의 자리 계산 오류	숫자 → 글자 변환	FAIL 2 · 32	FAIL 10	둘 다
B7	거리 오류 상태 무시	트러블슈팅 1 수정	0	FAIL 4	Top
B8	온습도 측정 전 표시 무시	트러블슈팅 1 수정	0	FAIL 4	Top

1 단위 TB 만 잡음 (B1 · B2)

Top 에선 안 생기는 상황 (busy 중 go · FIFO 꽉 참) 을 단위 TB 는 만들 수 있음

2 Top TB 만 잡음 (B7 · B8)

센서 상태 비트는 단위 TB 에 시험 줄 없음 → 실제 연결의 Top TB 가 잡음

단위 TB : DUT = uart_sender · sender_char_mux / Top TB : DUT = merge_final_top (팀 전체) / 숫자 = 불일치 수

트러블 슈팅 ① 현상 — 보드 검증 중 발견

① 현상 **FND 는 Err2 · ---- 인데 PC 는 400cm · 00%00C → PC 쪽은 오류를 정상 값으로 오해**

B-2 거리 — 수정 전



상황	FND	PC
범위 밖 (천장)	**Err2**	**400cm**
ECHO 선 뽑음	**Err3**	**400cm**

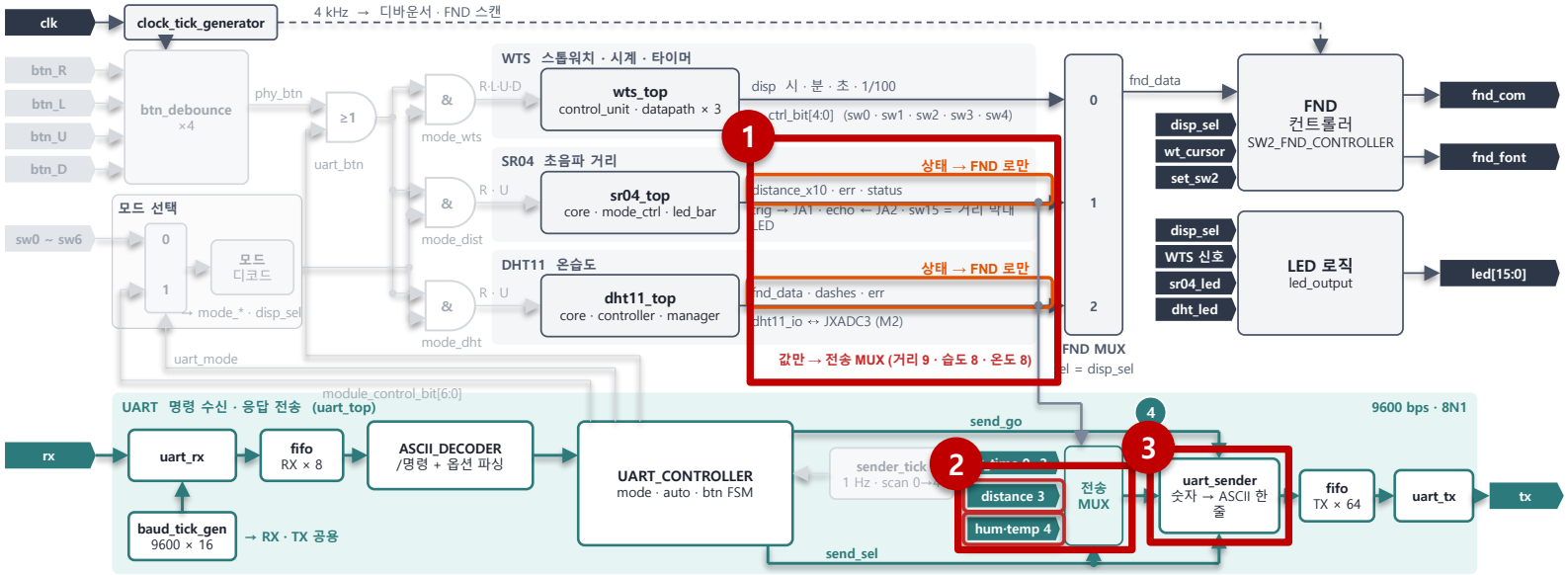
B-3 온습도 — 수정 전



상황	FND	PC
리셋 뒤 첫 측정 전	**----**	**00%00C**
DATA 선 뽑음	**Err1**	**32%24C**

트러블 슈팅 ② 원인 — 상태 선이 송신 MUX 에 없음

② 원인 **FND MUX 는 값 + 상태, 송신 MUX 는 값만 → uart_sender 는 숫자 글자만 만듦**



디코더 : "/get " + Enter

컨트롤러 : /mod 모드 → send_sel (온습도 4 · 거리 3 · ...)

전송 MUX : *_time · distance · hum·temp 중 선택

1 상태 선

FND MUX 로만 감

2 송신 MUX 입력

값만 (상태 없음)

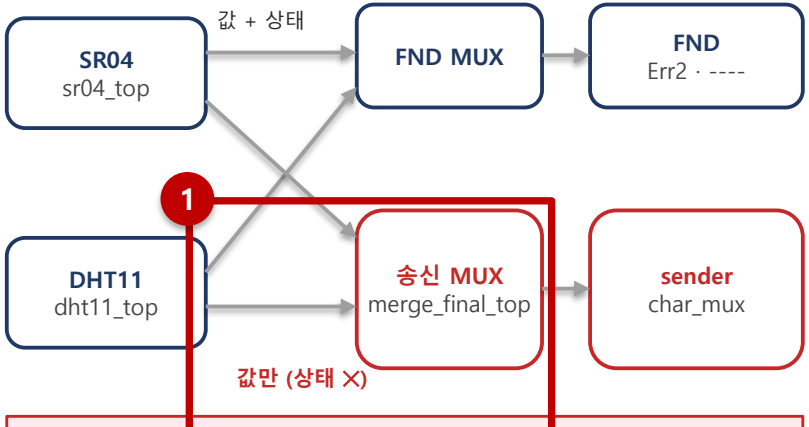
3 uart_sender

숫자만 → 400cm

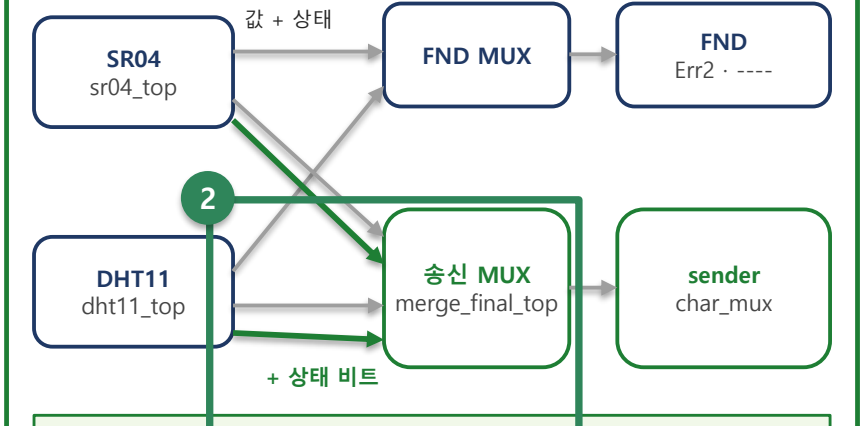
트러블 슈팅 ③ 해결 요약 — 수정 전 / 수정 후 한 장 비교

③ 해결 **바뀐 곳은 송신 MUX 입력 하나 → PC 도 FND 와 같게 Err 2 · --%--C**

수정 전



수정 후



1 수정 전 송신 MUX
값만 → PC 400cm

2 수정 후 송신 MUX
값 + 상태 → Err 2

3 그대로
블록 · 포트 · 24 비트

트러블 슈팅 ④ 적용 결과 — B-2 거리 (보드)

④ 적용 결과 같은 조작에서 PC 400cm · 400cm → Err 2 · Err 3

수정 전



수정 후



단계	조작	FND	수정 전 PC	수정 후 PC
T5	천장으로 → /get	**Err2**	**400cm**	**Err 2**
T6	ECHO 선 뽑기 → /get	**Err3**	**400cm**	**Err 3**

트러블 슈팅 2 ① 현상 · ② 원인 — TB 가 찾은 결함 8 건

① 현상 · ② 원인 **그룹 TB 305 검사 중 8 건이 목표 스펙과 다르게 동작 (PASS 297) → 원인 8 곳**

ID	상황 (목표 스펙과 다른 동작)	원인
G3	/set abcd 에도 시계 설정 켜짐	옵션 비교 없음
H9	/get 20 번 → 응답 15 줄	요청을 1 비트로 기억
J14	습도 100 → 99%24C 대신 :0%24C	두 자리만 가정
G7	/off 오타 뒤 /off w_h 도 ERROR	오류 때 카운터 안 지움
H7	/auto 다시 켜면 첫 줄이 일찍	1 초 카운터 안 지움
D7	랩 중 /get → 랩 값 전송	PC 값 = 화면 값
E6	초 화면 설정 → 안 보이는 분 +1	화면 조건 없음
L6	올림 순간 /get → 1 초 늦은 값	올림 신호 1 클럭 늦음

그룹 TB = 팀원이 짠 TB 12 개 · 목표 스펙 = 팀이 정한 명령 · 동작 표

트러블 슈팅 2 ③ 해결 · ④ 적용 결과 — 8 곳 수정

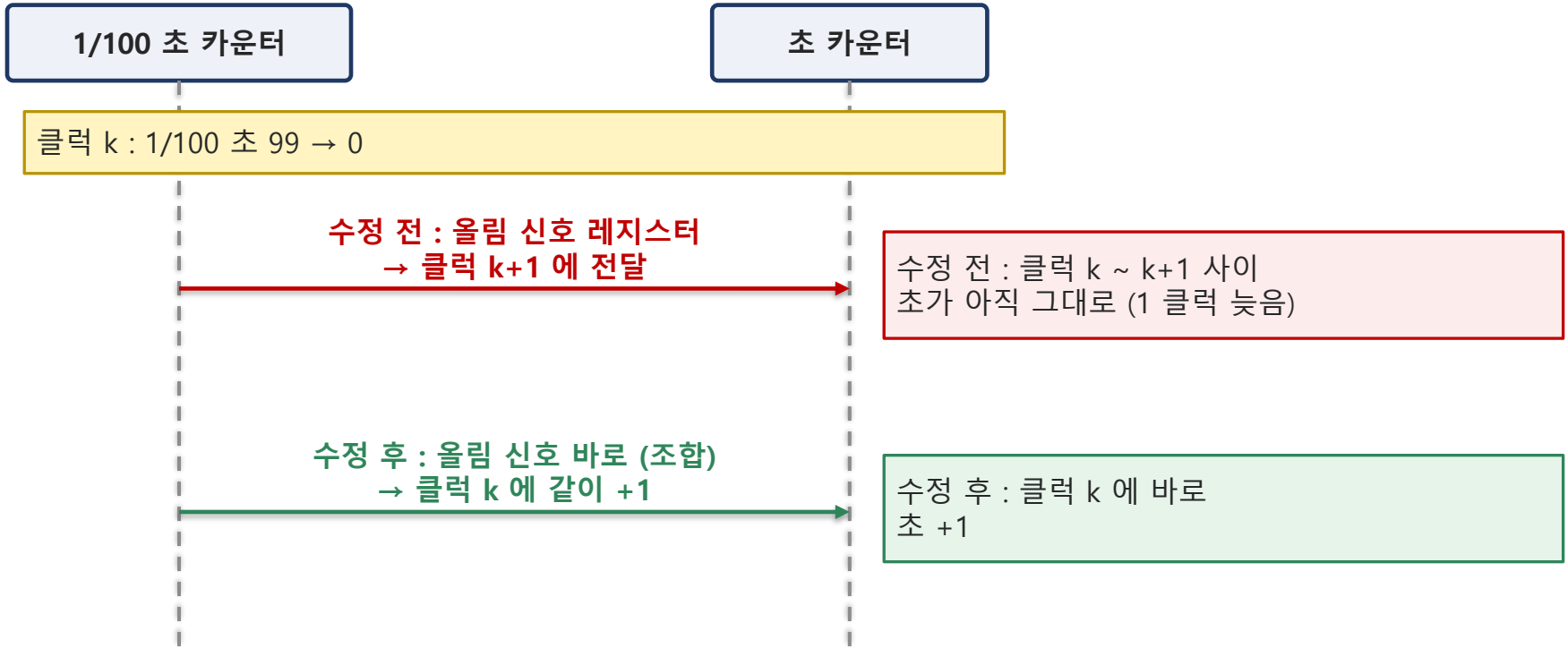
③ 해결 · ④ 결과 **파일 5 개 수정 → 그룹 TB 305 / 305 PASS · 타이밍 여유 +0.320 → +1.442 ns**

ID	고친 내용
G3	옵션이 w_hm 일 때만 설정
H9	/get 요청을 개수로 기억
J14	습도 · 온도 99 상한
G7	오류 때 /off 카운터도 0
H7	끌 때 1 초 카운터도 0
D7	PC 는 흐르는 값 (FND 는 랩)
E6	시.분 화면에서만 설정
L6	올림 신호를 같은 클럭에

	수정 전	수정 후
그룹 TB PASS	297	305
목표 스펙과 다름	8	0
통합 TB 불일치	0	0
LUT	1,489	1,476
WNS	+0.320 ns	+1.442 ns

L6 — 초가 넘어가는 바로 그 클럭에 /get → 1 clk 늦은 값

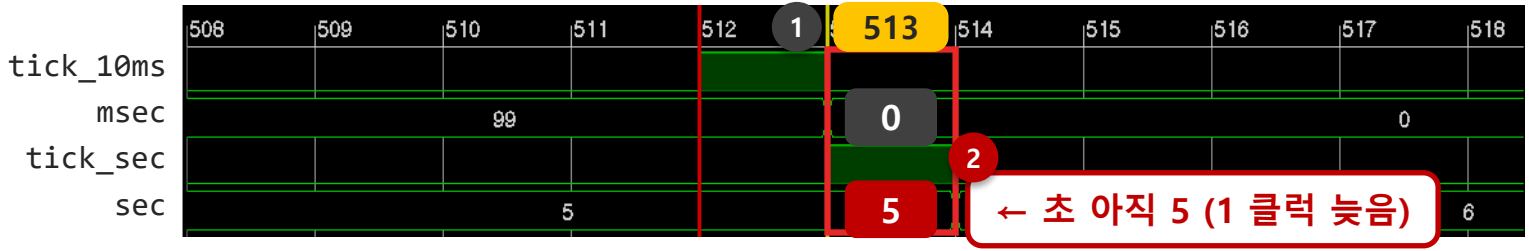
③ 해결 **담당 코드 : 스톱워치 카운터 (WTS)**



트러블 슈팅 2 ④ 보드 — L6 ILA 로 본 자리 올림

④ 적용 결과 **1/100 초가 0 이 되는 클럭 (샘플 513) 에서 수정 전은 초 5 그대로, 수정 후는 초 6 으로 같이 바뀜**

수정 전



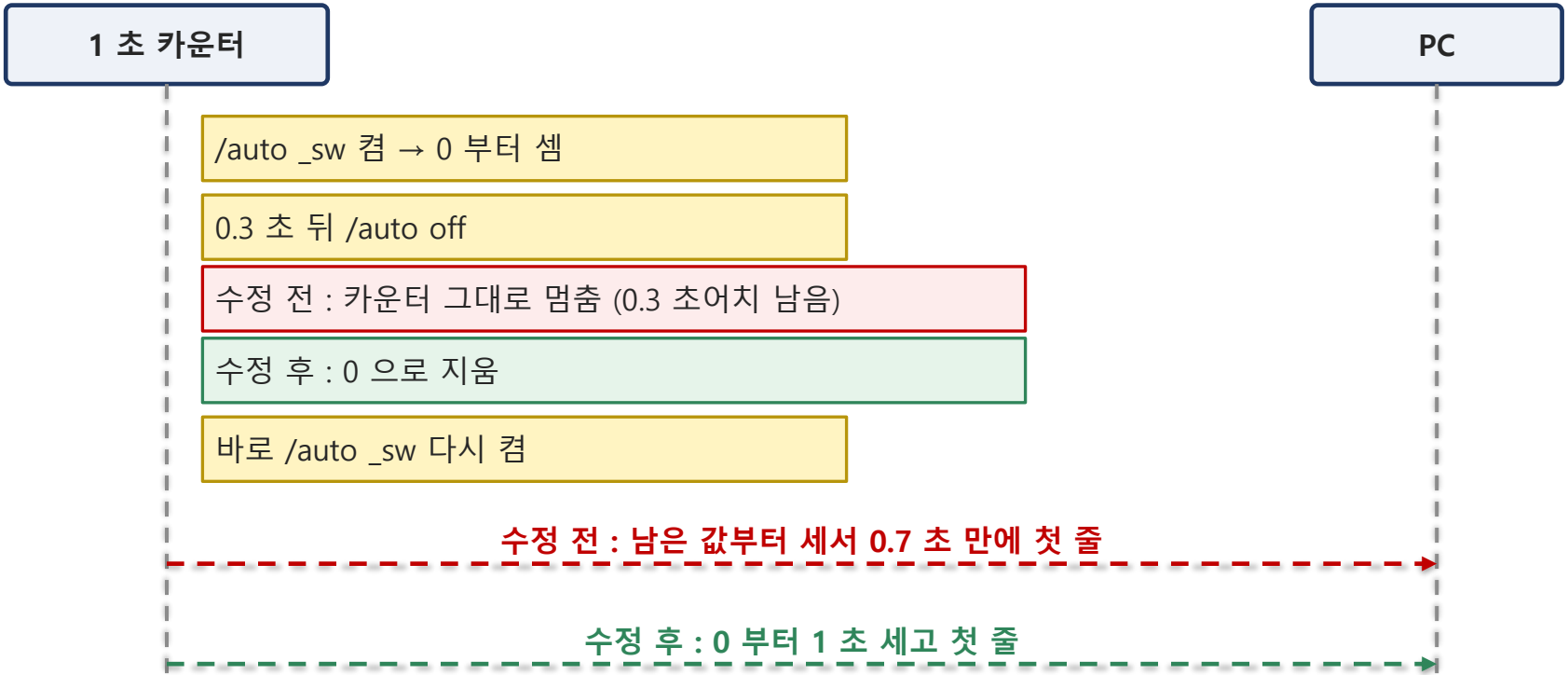
수정 후



Vivado ILA = 보드 칩 안 신호를 찍는 도구 · 샘플 1 개 = 1 클럭 = 10 ns (100 MHz)

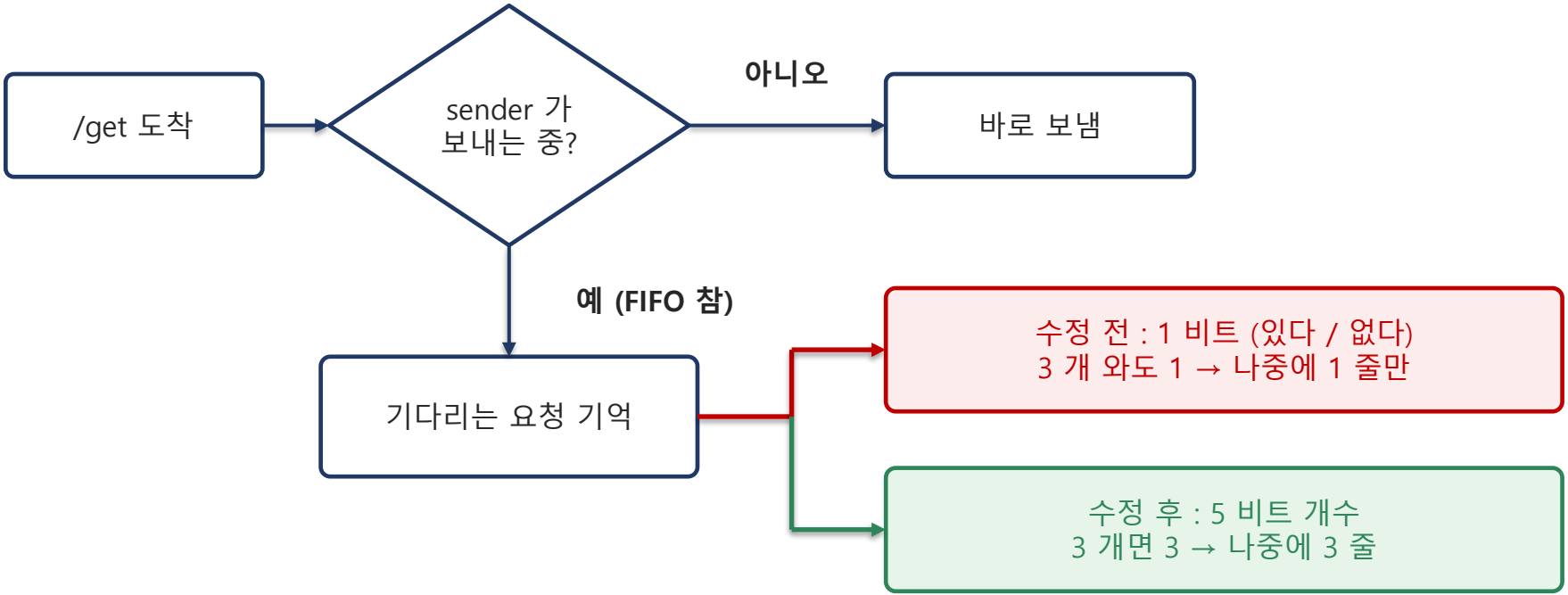
H7 — /auto 를 껐다 바로 켜면 첫 줄이 1 초보다 일찍 옴

③ 해결 **담당 코드 : UART 컨트롤러**



H9 — /get 을 쉬지 않고 20 번 보내면 15 줄만 옴

③ 해결 **담당 코드 : UART 컨트롤러**



트러블 슈팅 2 ④ 보드 — H7 · H9 PC 로거로 시간 · 줄 수

④ 적용 결과 **H7 다시 켠 첫 줄 730 → 1,048 ms (스펙 1,000) · H9 /get 20 번 → 15 → 20 줄**

	스펙	수정 전	수정 후
H7 /auto 다시 켠 뒤 첫 줄	1,000 ms	**730 ms**	**1,048 ms**
H9 /get 20 번 → 받은 줄	20 줄	**15 줄**	**20 줄**

```
C:\work\merge_fix4>py -3.10 uart_timing.py h7 COM7
       0.3 ms  SEND  /mod w_hm
     515.6 ms  SEND  /auto _sw
     825.0 ms  SEND  /auto off
     855.4 ms  SEND  /auto _sw
    1585.0 ms  RECV  00:03:58.76
    2576.6 ms  RECV  00:03:59.76
    3368.2 ms  SEND  /auto off

=> 다시 켠 뒤 첫 줄까지 730 ms
```

```
C:\work\merge_fix4>py -3.10 uart_timing.py h7 COM7
       0.2 ms  SEND  /mod w_hm
     505.5 ms  SEND  /auto _sw
     818.8 ms  SEND  /auto off
     848.6 ms  SEND  /auto _sw
    1894.4 ms  RECV  00:02:39.05
    2888.9 ms  RECV  00:02:40.05
    3354.0 ms  SEND  /auto off

=> 다시 켠 뒤 첫 줄까지 1048 ms
```

```
     693.2 ms  RECV  12:04:40.63
     709.1 ms  RECV  12:04:40.64
     710.0 ms  RECV  12:04:40.65
     725.1 ms  RECV  12:04:40.67

=> /get 20 번에 받은 줄 15 줄
```

```
     759.4 ms  RECV  12:03:22.07
     774.1 ms  RECV  12:03:22.09
     789.7 ms  RECV  12:03:22.10
     805.0 ms  RECV  12:03:22.12

=> /get 20 번에 받은 줄 20 줄
```

PC 프로그램 (uart_timing.py) 이 보낸 시각 · 받은 시각을 ms 로 기록

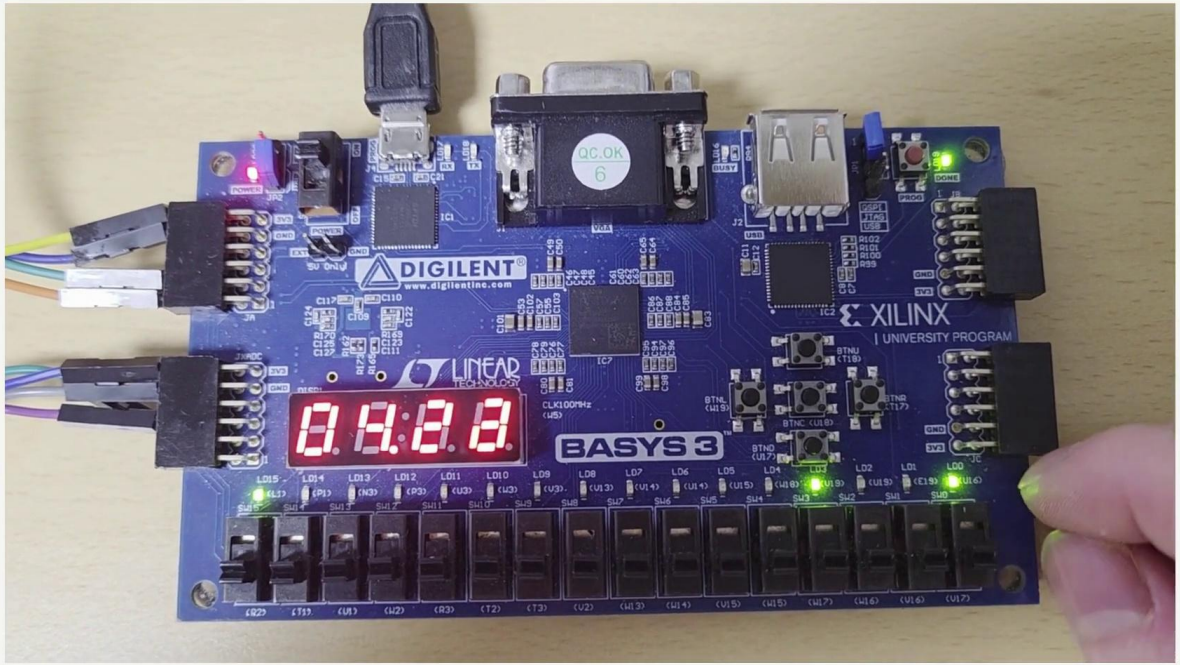
05 시연 영상 – 정보윤

시연 영상 — UART 명령으로 시계 · 스톱워치 · 타이머 · 센서 제어

시연 **9 개 시연 (A ~ I) 에서 핵심 장면만 이어 붙임 (83 초) · 전체 영상은 부록 C**

A. 스위치 모드 (UART 명령 보내기 전) — 리셋 직후

A3 **sw1 = 0, sw0 = 1** 스톱워치 화면 00.00



A 스위치 모드
B 시계 /get
C 스톱워치 · 랩
D 타이머 알람
E 거리 Err 2
F 온습도 Err 1
G 자동 전송 (1 초마다)
H ERROR
I 리셋

결론 · 고찰

결론 **목표 두 가지 모두 동작 — PASS 596 · FAIL 0 · TB 가 찾은 결함 8 건 수정**

목표	시뮬레이션	보드
① UART 명령으로 통합 기능 제어	그룹 TB 305 PASS	시연 A ~ F · H
② 상태 · 센서 데이터 PC 확인	통합 TB 558 일치	시연 B ~ G · I

결론 수치

- **TB 검증 :** PASS 596 · FAIL 0 (그룹 305 + 통합 291)
- **TB 신뢰성 :** sender 버그 8 개 주입 → 8 / 8 검출
- **트러블슈팅 1 :** 오류 · 측정 전도 FND 와 PC 같은 표시
- **트러블슈팅 2 :** TB 가 찾은 결함 8 건 수정 → 305 / 305
- **자원 · 타이밍 :** LUT 1,476 (7.1 %) · WNS +1.442 ns @ 100 MHz

고찰

① 설계 : 결과가 FND · PC 두 곳으로 나가는데 FND 만 챙김
② 검증 : 내 통합 TB 는 결함 8 건에서 FAIL 을 못 냄 (트러블슈팅 2 표의 상황을 시나리오에 안 넣음) → 목표 스펙대로 짠 그룹 TB 는 FAIL 로 잡음

개선 방향

① FND 와 PC 가 같은 상태인지 TB 가 서로 비교 (교차 검사)
② 시나리오 · 기대값을 DUT 가 아니라 목표 스펙 표에서 만들기

부록

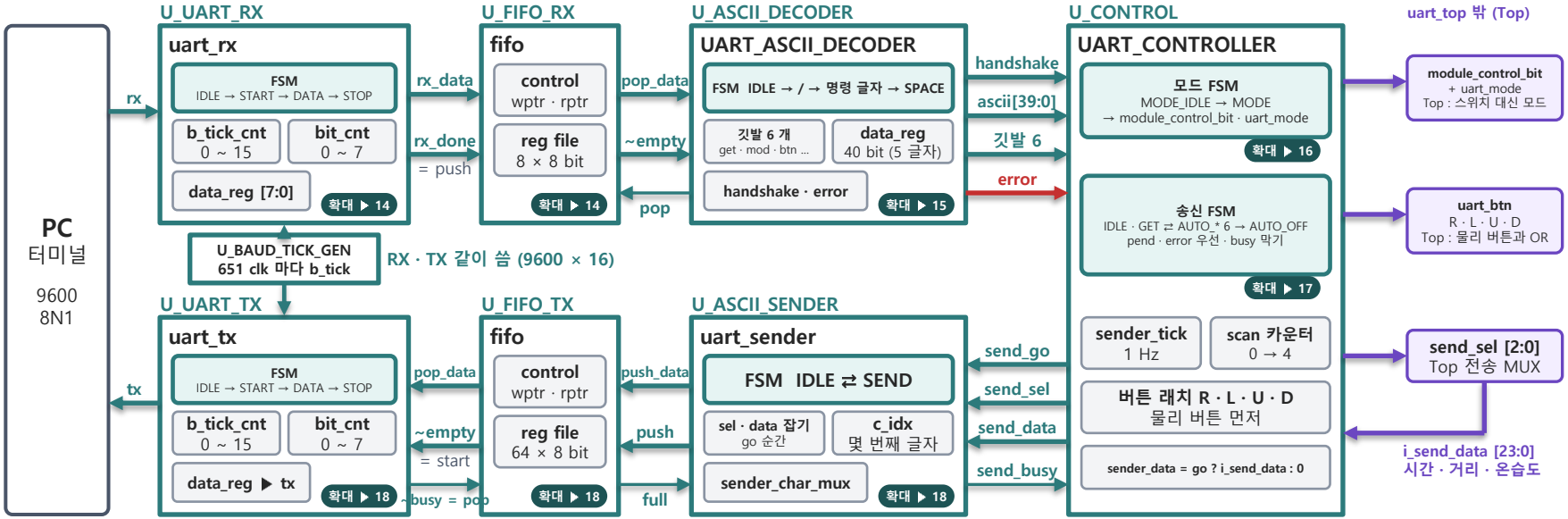
02 UART 동작 — UART 내부 · 수신 · 송신 · 명령 하나의 길 (4 장)
03 Top 시뮬레이션 — P1 ~ P6 확대 · 보충 (12 장)
03 UART sender — 응답 형식 · 시험용 Top · 글자 MUX · edge case ② · A 파형 해설 24 장
04 트러블 슈팅 — B-2 · B-3 해결 · 결과 표 · B 결함 8 건 상세 · 보드 확인
05 시연 영상 — C 전체 A ~ I (9 개)

● 02 시스템 구조 · UART 동작 – 정안성

UART 내부 — PC 명령이 RX → 디코더 → 컨트롤러 → 송신 → TX 로 도는 길

핵심

rx 한 글자 = **10 비트 ≈ 1.04 ms** → FIFO → 디코더가 **글자마다 상태 이동** → 명령 끝에 **handshake 1 clk** → 컨트롤러가 모드 · 버튼 · 전송을 결정
전송은 **send_sel** 로 Top 의 데이터 MUX 를 고르고 **send_go 1 clk** → sender 가 한 clk 에 한 글자씩 TX FIFO → uart_tx → PC



9600 bps · 8N1
시작 1 + 데이터 8 + 정지 1 = 10 비트

651 clk = b_tick
100 MHz ÷ (9600 × 16) · 16 tick = 1 비트 104 µs

1 글자 ≈ 1.04 ms
/mod dist CR 10 글자 ≈ 10.4 ms 동안 받음

rx_done → tx 0 : 7 clk
모듈 지날 때마다 1 ~ 2 clk · 합 70 ns (19 쪽)

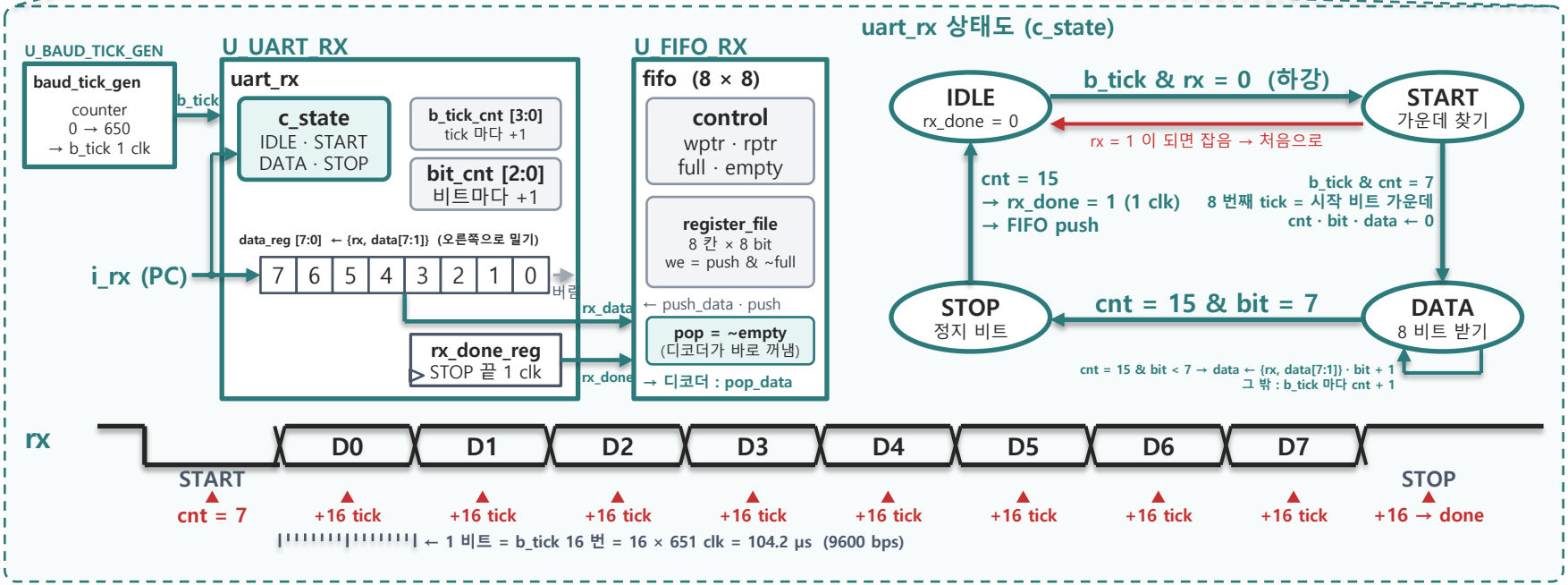
● 02 시스템 구조 · UART 동작 – 정안성

수신 — uart_rx 는 16 배로 쪼개 보고 비트 가운데에서 읽음

핵심

b_tick (651 clk) 16 번 = 1 비트 · START 에서 **8 번째 tick (cnt = 7)** = 시작 비트 가운데 → DATA 는 그 뒤 **16 tick 마다** 한 비트 (LSB 먼저)
STOP 끝에 **rx_done 1 clk** = FIFO push · 디코더는 FIFO 가 비지 않으면 **그 clk 에 바로 pop** (pop = ~empty)

PC rx → **uart_rx** → **FIFO RX** → DECODER → CONTROLLER → SENDER → FIFO TX → uart_tx → PC tx



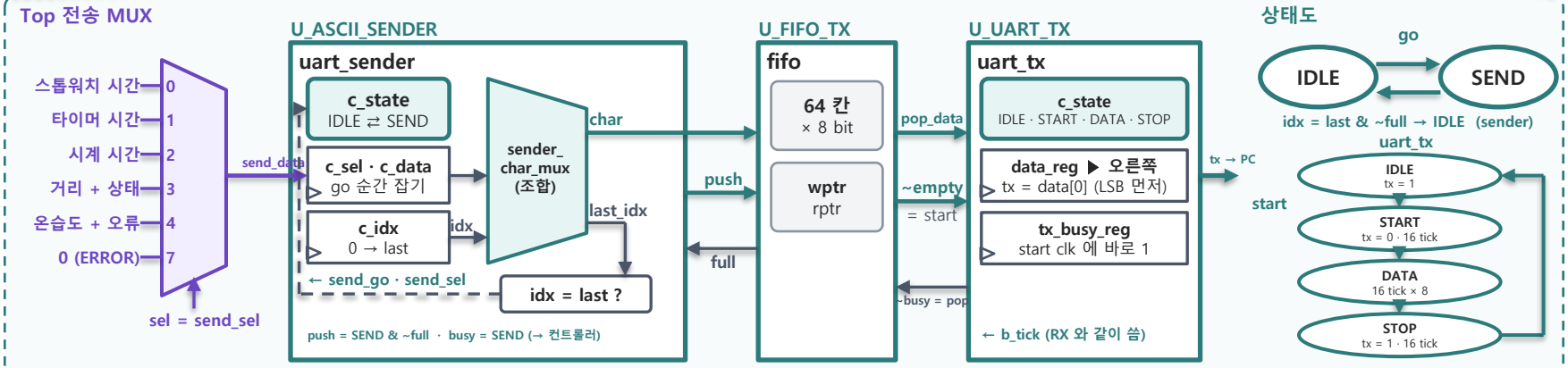
● 02 시스템 구조 · UART 동작 – 정안성

확대 ⑤ 송신 — sender 가 숫자를 글자로 바꿔 한 clk 에 한 글자씩, uart_tx 가 1 글자 1.04 ms 로

핵심

send_go 순간 **sel · data 를 잡아 둠** (보내는 동안 값이 바뀌어도 그 줄은 그대로) → **sender_char_mux** 가 (sel, idx) 자리 글자를 냄
SEND 에서 **idx = last_idx** 이면 끝 · TX FIFO 64 칸에 먼저 다 넣고 → **uart_tx 는 busy 가 0 일 때만 pop** (한 글자 한 번)

PC rx → uart_rx → FIFO RX → DECODER → CONTROLLER → **SENDER** → **FIFO TX** → **uart_tx** → PC tx



send_sel	데이터	보내는 글자 (CR LF = 줄 바꿈)	글자 수	last_idx	선 위 시간
0 · 1 · 2	스톱워치 · 타이머 · 시계	12:34:56.78 CR LF	13	12	≈ 13.5 ms
3	거리 / 센서 오류	123cm CR LF / Err 2 CR LF	7	6	≈ 7.3 ms
4	온습도 / 측정 전 / 오류	45%25C CR LF / --%--C CR LF / Err 1 CR LF	8 / 8 / 7	7 / 7 / 6	≈ 8.3 ms
7	디코더 error	ERROR CR LF	7	6	≈ 7.3 ms

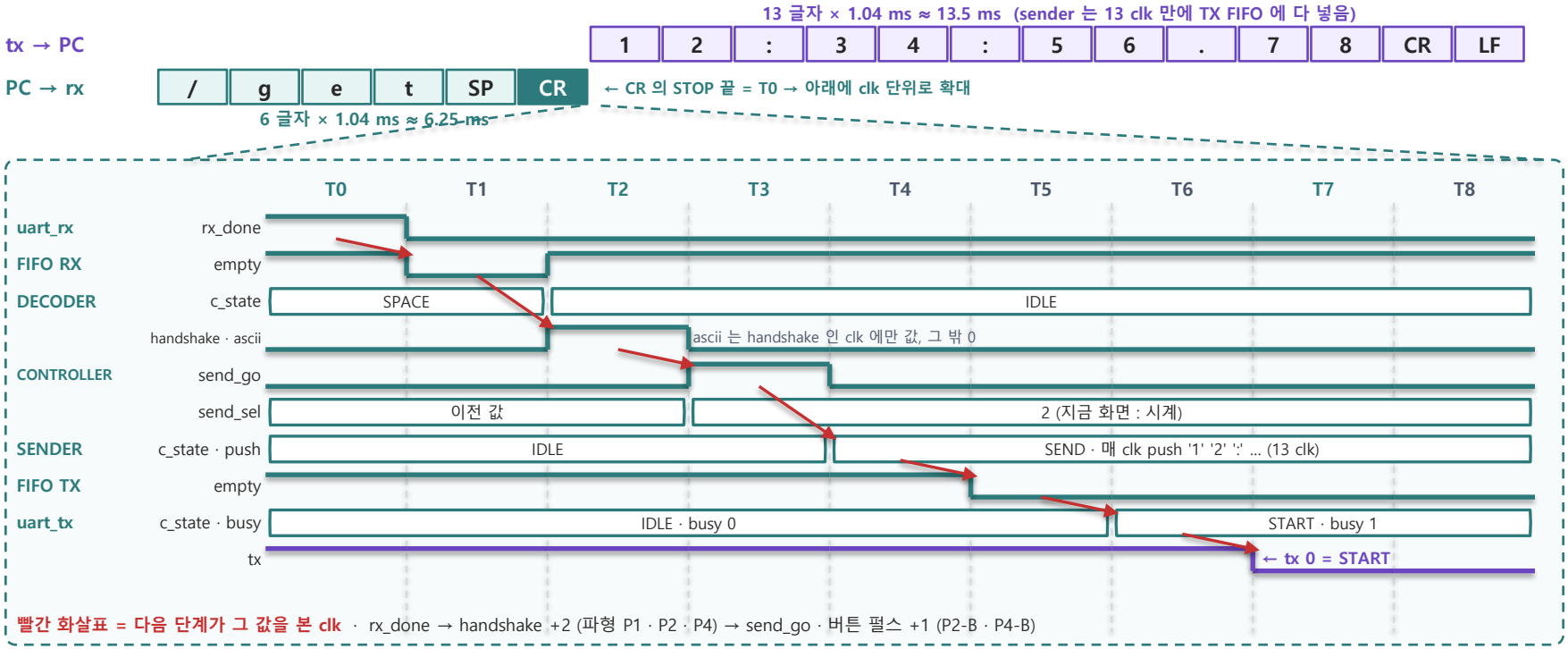
온습도 99 보다 크면 99 (FND 와 같은 규칙)

● 02 시스템 구조 · UART 동작 – 정안성

명령 하나의 길 — /get CR 마지막 글자에서 TX 시작까지 7 clk

핵심

ms 단위 : 6 글자 받기 ≈ 6.25 ms, 대답 13 글자 ≈ 13.5 ms → **clk 단위** 로 보면 모듈마다 레지스터 1 ~ 2 개
rx_done 에서 tx = 0 까지 7 clk (70 ns) · 파형 장의 rx_done → handshake +2 → go · 버튼 펄스 +1 = 여기 T0 → T2 → T3

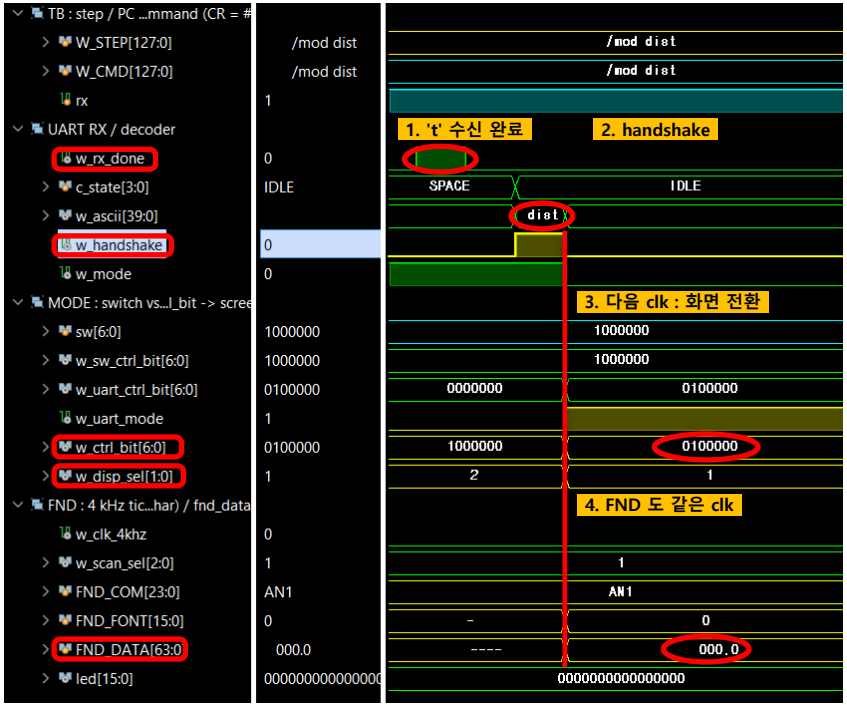


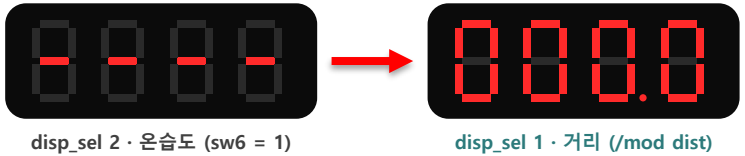
● 03 Top · 시뮬레이션 ① 화면 제어 /mod – 정안성

P1 확대 — 마지막 글자 수신 → 다음 clk 에 화면 전환

결론

rx_done → 디코더 handshake (w_ascii = **"dist"**) → **다음 clk** 에 w_ctrl_bit · disp_sel · FND 동시에 변경
sw 는 **1000000** (온습도) 그대로인데 화면은 **거리 000.0** → 스위치 대신 UART 값





disp_sel 2 · 온습도 (sw6 = 1)　　disp_sel 1 · 거리 (/mod dist)

1. **w_rx_done** — 마지막 글자 't' 수신 완료 → RX FIFO → 디코더
2. **handshake** — c_state SPACE → IDLE, w_ascii = "dist", w_handshake 1 clk 펄스
3. **다음 clk** — w_uart_ctrl_bit 0100000 · w_uart_mode 1 → w_ctrl_bit 1000000 → 0100000, disp_sel 2 → 1
4. **FND** — FND_FONT '-' → '0', FND_DATA ---- → 000.0 (같은 clk)

● 03 Top · 시뮬레이션 ③ 버튼 /btn – 정안성

P2 /btn R — UART 버튼으로 SR04 거리 측정

결론

/btn R handshake → **w_uart_btn_R → w_btn_R → SR04 i_btn_R → trig** (약 18.2 ms)
echo **7.134 ms** → distance **123** · x10 **1230** → FND **123.0**

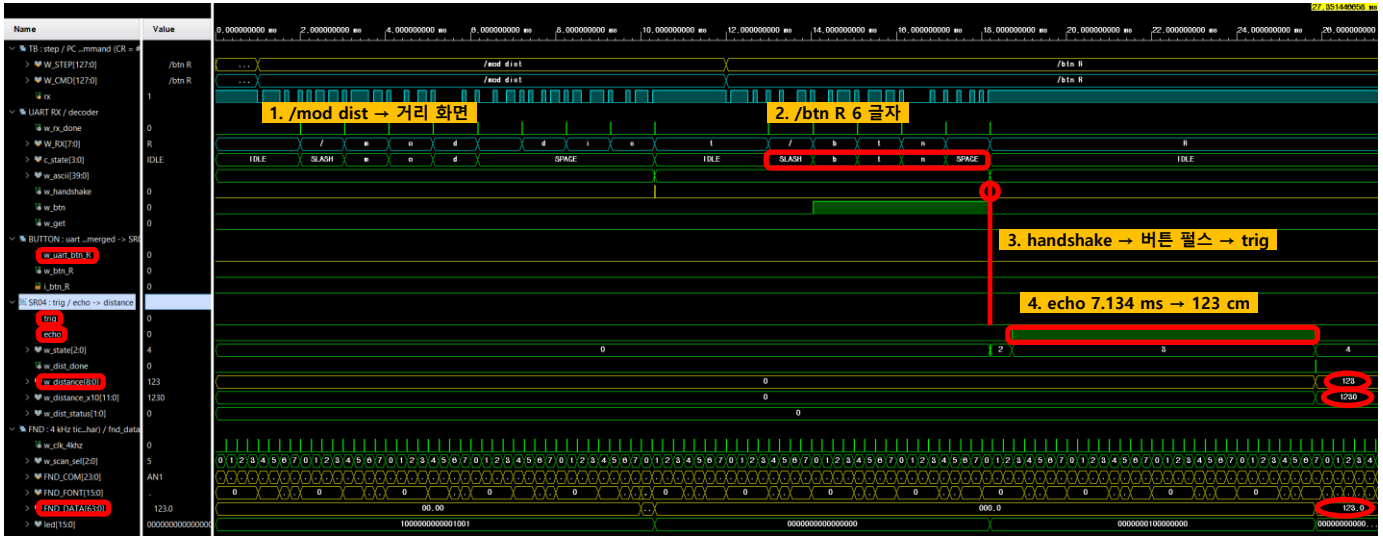
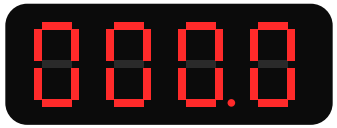
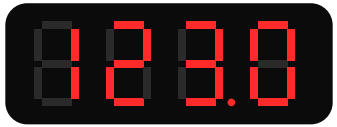


Figure. /btn R 전체 0 ~ 27 ms — 아래 UART TX 줄은 이 구간에 변화가 없어 잘라 냄



/btn R 전 000.0



측정 뒤 123.0 cm

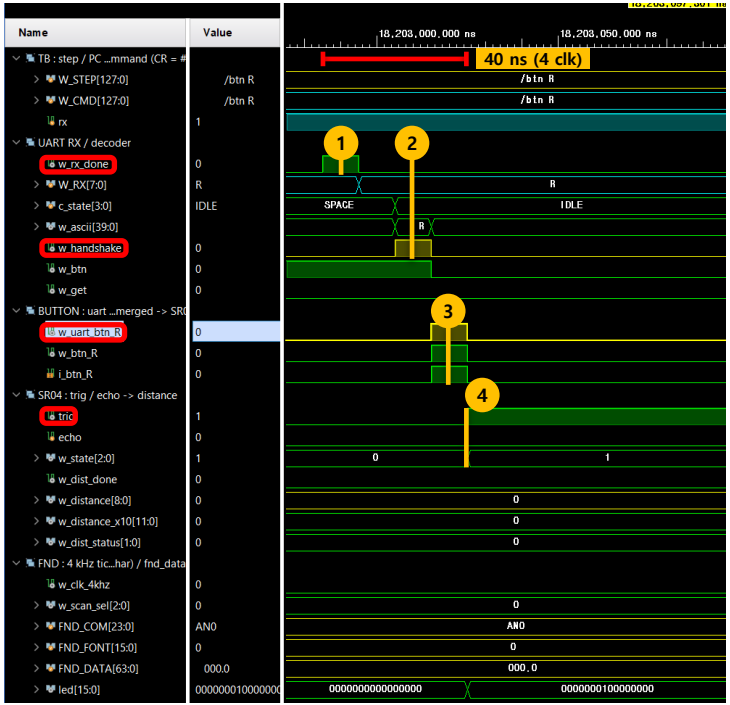
echo 7,134 us
÷ 58 us = 123 cm

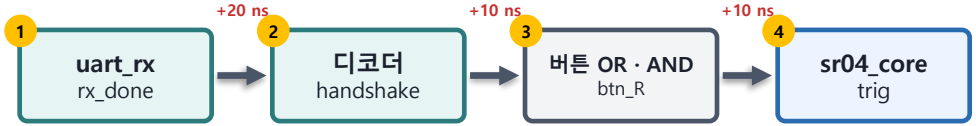
● 03 Top · 시뮬레이션 ③ 버튼 /btn – 정안성

P2 확대 — /btn R 수신에서 trig 상승까지

결론

rx_done → handshake **+2 clk** → 버튼 펄스 **+1 clk** → trig **+1 clk** : 합계 **4 clk (40 ns)**
UART 버튼도 물리 버튼과 같은 **OR → 화면 AND** 경로 → 지금 화면 (거리) 의 SR04 에만 들어감





1 **t + 0 ns** w_rx_done — 마지막 글자 'R' 수신 완료 (W_RX = R)

2 **t + 20 ns** c_state SPACE → IDLE, w_ascii = "R", w_handshake 1 clk

3 **t + 30 ns** w_uart_btn_R → w_btn_R (물리 버튼과 OR) → U_SR04 i_btn_R, 1 clk 펄스

4 **t + 40 ns** trig 1 · w_state 0 → 1 (START), trig 는 11 us 유지

1 clk = 10 ns (100 MHz) · t = rx_done 이 뜬 clk

● 03 Top · 시뮬레이션 ② 송신 /auto – 정안성

P3 보충 — 줄 끝 간격이 1 s 가 아닌 이유 : 데이터마다 글자 수가 다름

결론

sender 는 1 s 마다 정확히 출발 (send_go), 그런데 줄 길이가 sel 마다 다름 : 시간 **13** 글자 · 거리 **7** · 온습도 **8** 글자
마커는 줄 '끝' → **간격 = 1 s + (다음 줄 - 앞 줄) 글자 수 × 1.0416 ms** 예) 시계 → 거리 = 1 s - 6 글자 = **993.75 ms**

sender_char_mux 가 sel 마다 만드는 줄

sel	데이터	보내는 줄 (예)	글자 수	o_last_idx	줄 길이
0 · 1 · 2	스톱워치 · 타이머 · 시계	00:00:01.09 + CR LF	13	12	13.54 ms
3	거리	123cm + CR LF	7	6	7.29 ms
4	온습도	55%24C + CR LF	8	7	8.33 ms

1 글자 = start 1 + data 8 + stop 1 = 10 bit × 104.16 us = 1.0416 ms

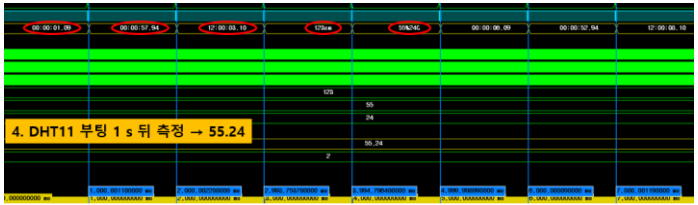
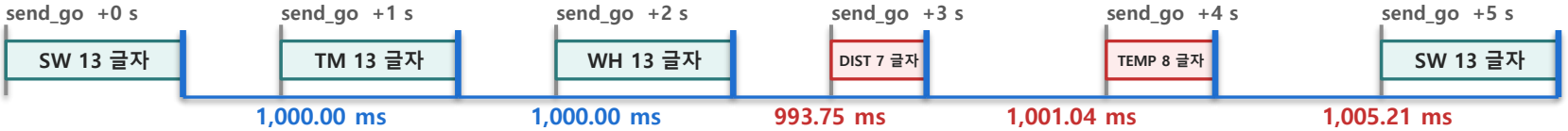


Figure. 줄 끝 마커 (아래 = 첫 마커 기준 시각)

send_go 는 1 s 마다 같은 자리에서 출발 → 막대 (= 줄) 길이만 달라서 끝 (마커) 이 앞뒤로 움직임 (막대 길이 = 글자 수, 크게 그림)



구간	앞 줄 → 다음 줄	글자 수	글자 차	계산 : 1 s + 글자 차 × 1.0416 ms	측정 (마커 간격)	남는 차
1 → 2	SW → TM	13 → 13	0	1,000.0000 ms	1,000.0011 ms	+1.1 us
2 → 3	TM → WH	13 → 13	0	1,000.0000 ms	1,000.0011 ms	+1.1 us
3 → 4	WH → DIST	13 → 7	-6	993.7504 ms	993.7515 ms	+1.1 us
4 → 5	DIST → TEMP	7 → 8	+1	1,001.0416 ms	1,001.0427 ms	+1.1 us
5 → 6	TEMP → SW	8 → 13	+5	1,005.2080 ms	1,005.2026 ms	-5.4 us

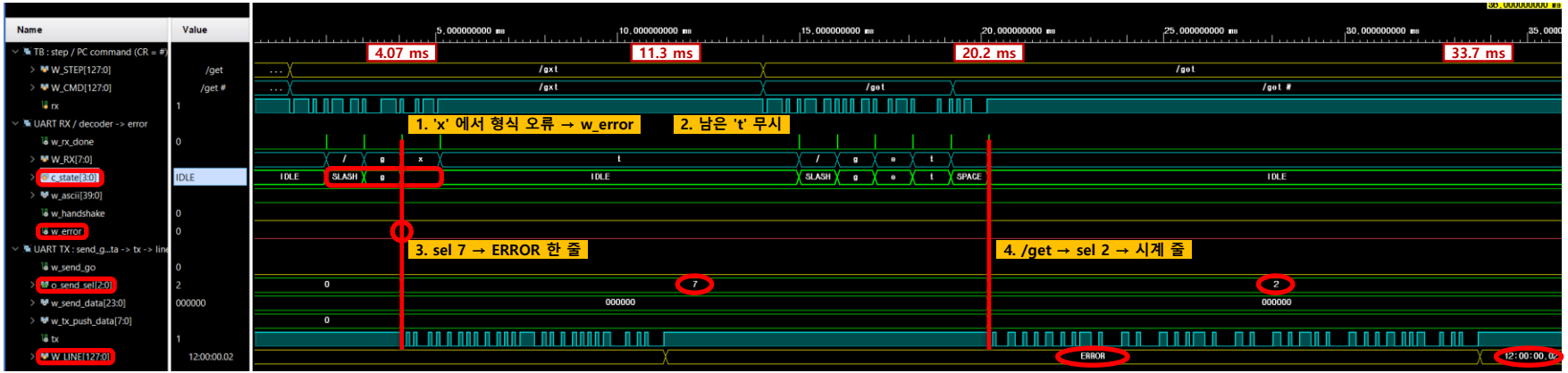
남는 수 us = baud tick 흔들림
tx 는 6.51 us 간격 tick 에 맞춰
출발 → 줄마다 +1.1 us 씩 밀리다
한 칸 (6.51 us) 차면 당겨짐
(+1.1 - 6.51 = -5.4 us)

● 03 Top · 시뮬레이션 ④ 오류 ERROR – 정안성

P4 /gxt — 형식 오류 → ERROR 한 줄, 다음 명령은 정상

결론

/gxt 의 3 번째 글자 **x** 에서 w_error (약 **4.07 ms**) → 전송 MUX sel **7** → **ERROR** 한 줄
남은 글자 t 는 무시 (c_state IDLE), 이어진 /get 은 정상 → sel **2** 시계 줄 **12:00:00.02** (약 20.2 ms)



PC 터미널 · 9600 bps 8N1

1.0 ms	> /gxt	보냄 · 형식 오류
11.3 ms	ERROR	받음 · sel 7
14.0 ms	> /get	보냄
33.7 ms	12:00:00.02	받음 · sel 2 시계

시간 순서

4.07 ms	'x' rx_done → w_error → send_go (sel 7)
4.1 ~ 11.3 ms	tx ERROR 7 글자 — 줄 하나만
14.0 ~ 20.2 ms	/get 6 글자 → handshake
20.2 ~ 33.7 ms	tx 시계 줄 13 글자

● 03 Top · 시뮬레이션 ④ 오류 ERROR – 정안성

P4 확대 — 'x' 수신에서 ERROR 송신 시작까지

결론 rx_done ('x') → **+2 clk** w_error → **+1 clk** send_go · sel 0 → 7 → 글자 E R R O R 를 TX FIFO 로
tx start bit 까지 **70 ns (7 clk)** — 오류는 다른 전송보다 먼저 (sender 가 비어 있으면 바로)

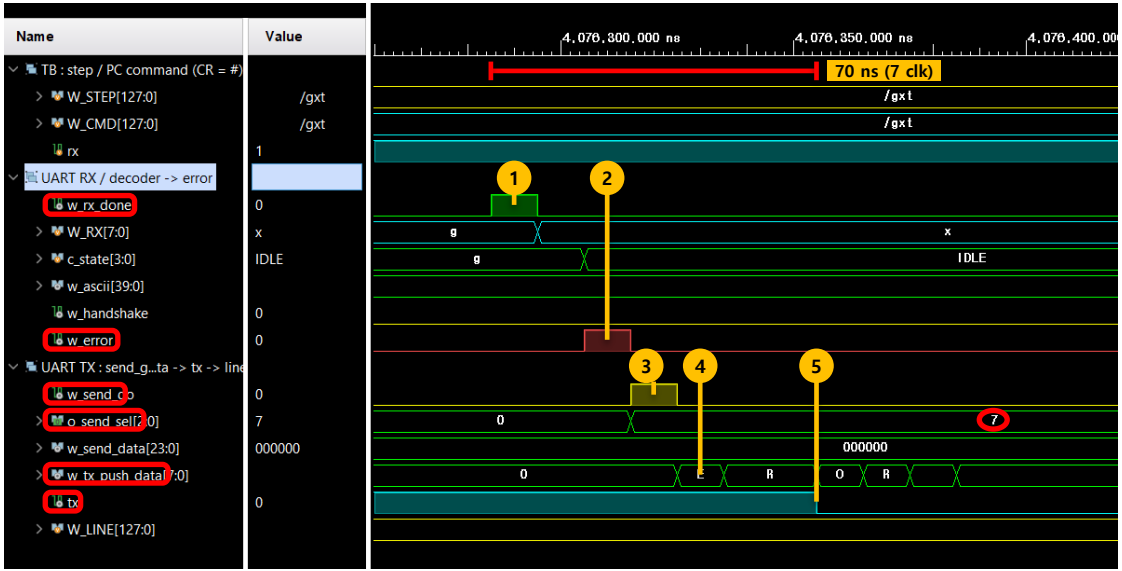


Figure. 4,076,285 ns (= 약 4.0763 ms) 부근 확대 — 1 clk = 10 ns

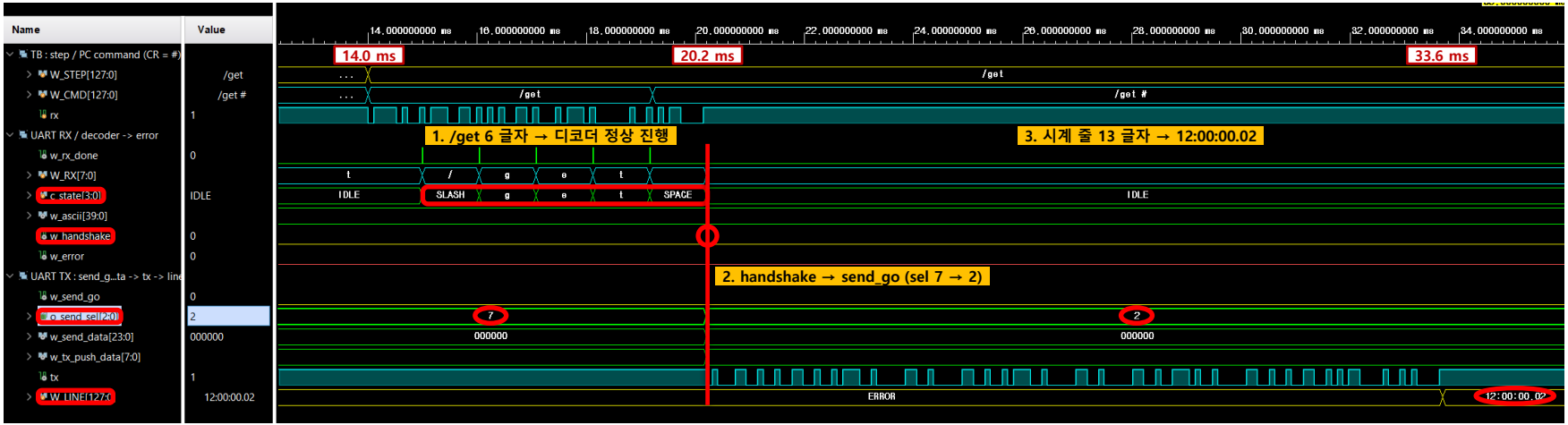
1. **t + 0 ns** — w_rx_done — 'x' 수신 완료 (W_RX = x)
2. **t + 20 ns** — c_state g → IDLE, w_error 1 clk
3. **t + 30 ns** — w_send_go · o_send_sel 0 → 7 (ERROR)
4. **t + 40 ns** — w_tx_push_data E → R → R → O → R
5. **t + 70 ns** — tx 1 → 0 (start bit) — 송신 시작

● 03 Top · 시뮬레이션 ④ 오류 ERROR – 정안성

P4 ERROR 뒤 /get — 다음 명령은 정상 처리

결론

디코더는 IDLE 에서 다시 시작 → /get 6 글자 **SLASH → g → e → t → SPACE** → handshake (약 **20.2 ms**)
/mod 전 (스위치 모드) 이라 sel **2** 시계 → **12:00:00.02** 13 글자 송신 (20.2 ~ 33.6 ms)



PC 터미널 · 9600 bps 8N1

11.3 ms	ERROR	앞 명령 /gxt 의 오류
14.0 ms	> /get	보냄
33.6 ms	12:00:00.02	받음 · sel 2 시계

시간 순서

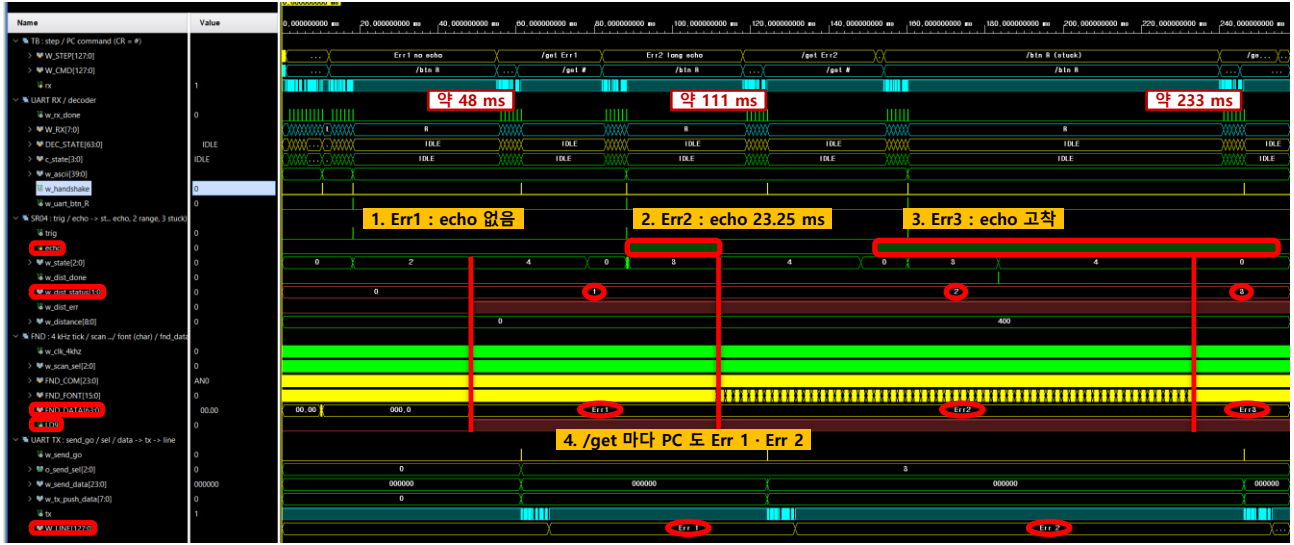
14.0 ms	/get 수신 시작 (6 글자 × 1.04 ms)
20.2 ms	handshake → send_go · sel 7 → 2
20.2 ~ 33.6 ms	tx 13 글자 → W_LINE 12:00:00.02

● 03 Top · 시뮬레이션 센서 오류 SR04 – 정안성

P5 SR04 Err1 · Err2 · Err3 — FND · LED · PC 가 같은 오류 번호

결론

Err1 무응답 : trig↓ + **30 ms** → status **1** · FND **Err1** · LD9 → /get **Err 1**
Err2 범위 밖 : echo 23.25 ms > **23.2 ms (400 cm)** → status **2** · distance 400 · FND **Err2** → /get **Err 2**
Err3 echo 고착 : 23.2 ms 에 Err2 → **73.2 ms** 에 status **3** · 강제 IDLE (w_state 0) → FND **Err3**





Err1 무응답
trig↓ + 30 ms 에 판정
echo 없음 · LD9 켜짐



Err2 범위 밖
echo↑ + 23.2 ms 에 판정
400 cm 넘음 · distance 400



Err3 echo 고착
trig↓ + 73.2 ms 에 판정
echo 고착 → 강제 IDLE

Figure. P5 전체 0 ~ 258 ms — 빨간 세로선 = 오류 판정 (w_dist_done)

● 03 Top · 시뮬레이션 센서 오류 DHT11 – 정안성

P6 측정 전 /get — 숫자 대신 --%--C

결론

측정 전 w_dht_dashes = **1** → FND **----** · PC 도 숫자 대신 **--%--C** (v1 은 00%00C)
/get handshake (약 **18.2 ms**) → sel **4** 온습도 → 8 글자 송신 (18.2 ~ 26.5 ms)

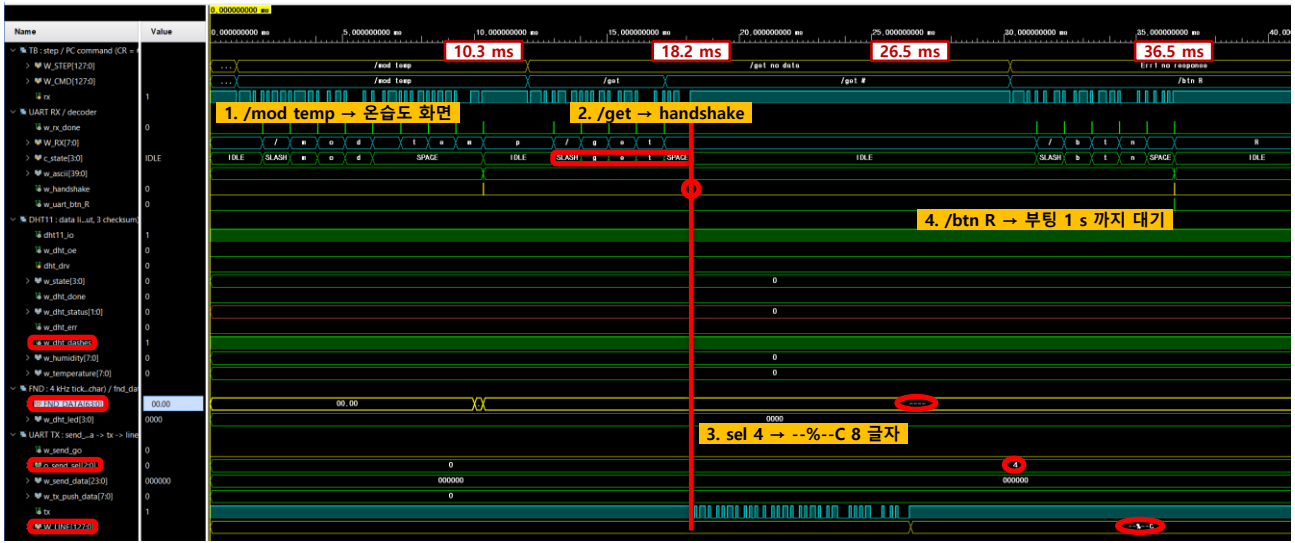


Figure. P6 시작 0 ~ 40 ms



FND ---- (dashes 1)

PC 터미널 · 9600 bps 8N1

12.0 ms	> /get	보냄
26.5 ms	--%--C	sel 4

10.3 ms	/mod temp → 화면 온습도
18.2 ms	/get handshake
26.5 ms	--%--C 도착
36.5 ms	/btn R → 1 s 에 측정

● 03 Top · 시뮬레이션 센서 오류 DHT11 – 정안성

P6 Err1 — 시작 LOW 18 ms 뒤 센서 응답 없음

결론

부팅 1 s 에 w_dht_oe 1 (선 LOW) → **18.0 ms** 뒤 선 놓음 → 응답 없음 → 약 **0.2 ms** 안에 status **1**
FND ---- → **Err1** · LED 0001 → 0010 → /get (약 1,036 ms) → PC **Err 1**

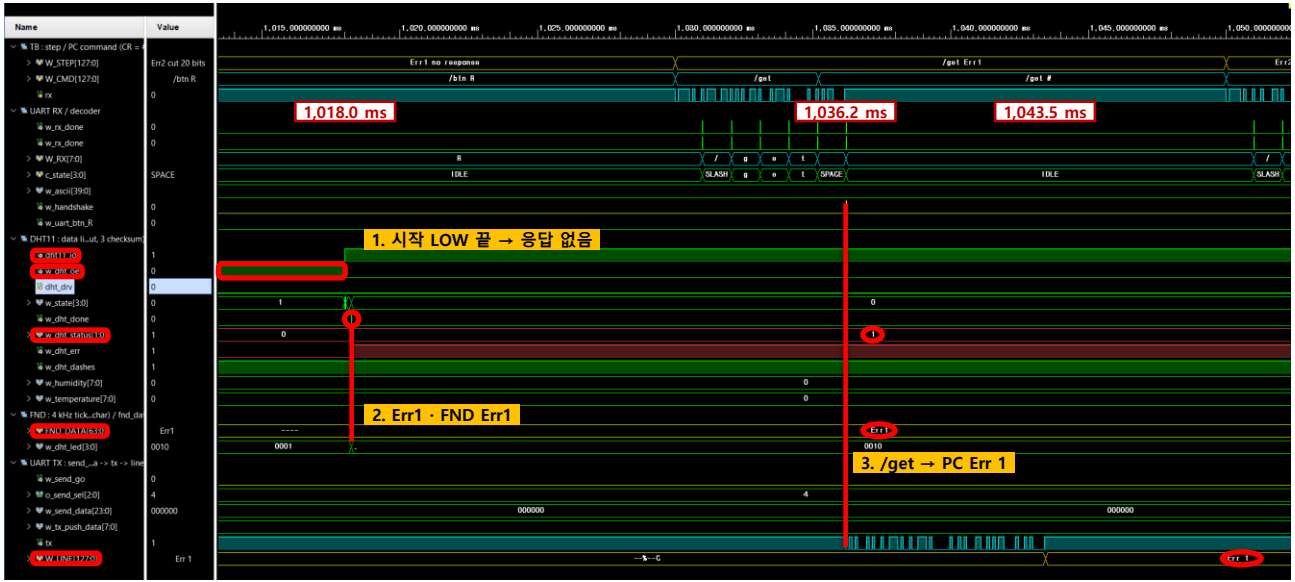


Figure. 1,013 ~ 1,052 ms 확대



측정 전
LED 0001 (측정 중)



Err1 응답 없음
1,018.2 ms · LED 0010

PC 터미널 · 9600 bps 8N1

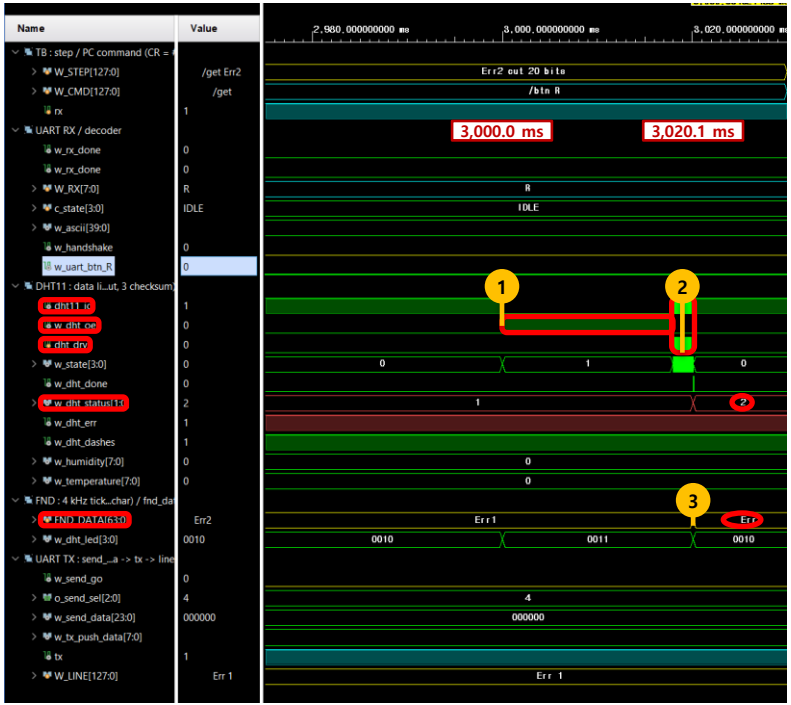
```
1,030 ms   > /get      보냄
1,043.5 ms Err 1       sel 4
```

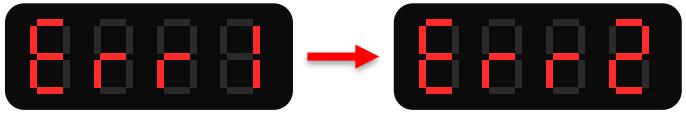
● 03 Top · 시뮬레이션 센서 오류 DHT11 – 정안성

P6 Err2 — 20 비트 뒤 데이터가 끊김

결론

이전 시작 + **2 s** (3,000 ms) 시작 LOW → 3,018 ms 응답 + 20 비트 → 끊김 → 약 **0.2 ms** 뒤 status **2**
FND Err1 → **Err2** · LED 0011 (측정 중) → 0010, w_state 0 복귀 → 다음 측정 가능





1 s 의 Err1 → 3.020 s Err2

1. **3,000 ms** — 이전 시작 + 2 s → w_dht_oe 1, 선 LOW 18 ms (LED 0011)
2. **3,018 ms** — 선 놓음 → 센서 응답 + 20 비트 (dht_drv 토글)
3. **약 3,020 ms** — 마지막 엣지 + 0.2 ms → w_dht_done, status 1 → 2, FND Err2

● 03 Top · 시뮬레이션 센서 오류 DHT11 – 정안성

P6 Err3 — 40 비트를 다 받았지만 체크섬이 틀림

결론

5,000 ms 시작 LOW → 5,018 ms 부터 40 비트 → 체크섬 불일치 → 약 **5,021.6 ms** 에 status **3**
습도 · 온도는 갱신 안 함 (**0 · 0** 그대로) → FND **Err3** → /get (약 5,036 ms) → PC **Err 3**

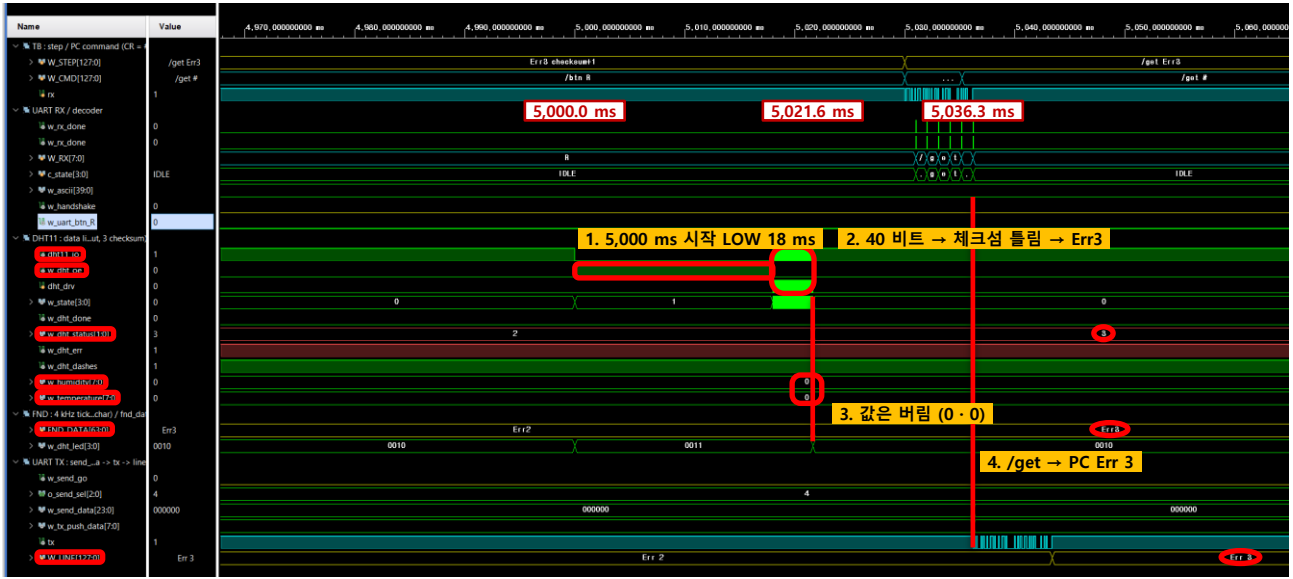


Figure. 4,966 ~ 5,063 ms 확대



3 s 의 Err2
LED 0011 (측정 중)



Err3 체크섬
5,021.6 ms · LED 0010

PC 터미널 · 9600 bps 8N1

5,030 ms	> /get	보냄
5,043.5 ms	Err 3	sel 4

응답 형식 → FIFO 깊이 64 · sender 자원

요약 **가장 긴 경우 5 종 연달아 = 54 글자 → 2 의 거듭제곱 깊이 64**

sel 별 응답 형식 (줄 끝 CR LF 포함)

sel	형식	예	글자
0 · 1 · 2	시간	12:34:56.78	13
3	거리	045cm	7
4	온습도	55%24C	8
7	오류	ERROR	7

1 FIFO 깊이 (depth) 계산

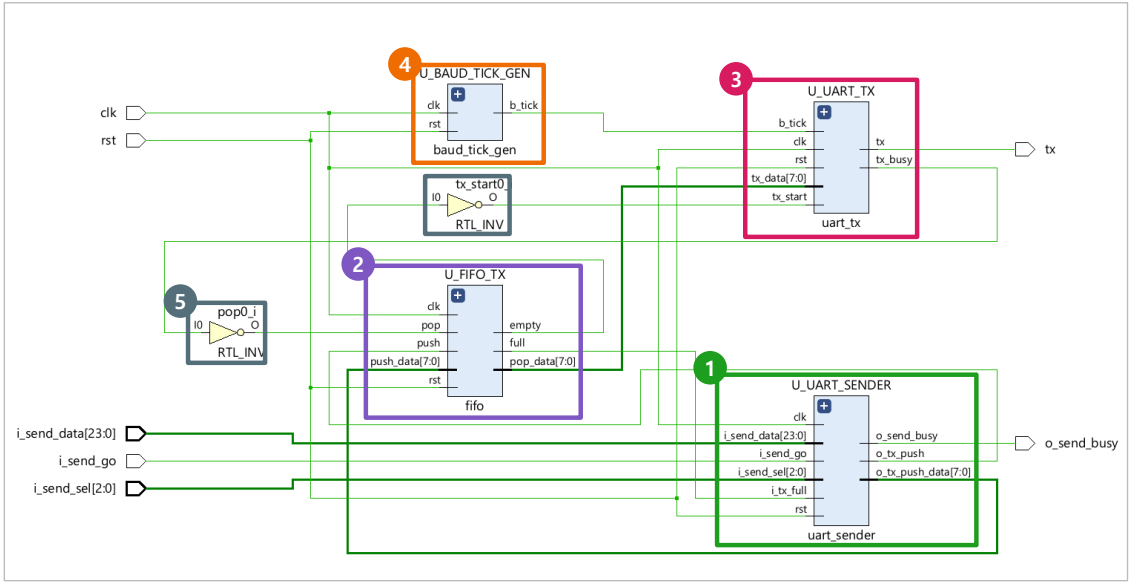
5 종 연달아 : 13 + 13 + 13 + 7 + 8
= 54 글자
→ 2 의 거듭제곱 → 깊이 64
깊이 16 대비 LUT +10 · FF +4

2 sender 자원 (구현)

LUT 136 · FF 32
(전체 LUT 1,476 · FF 738)

sender RTL ① 시험용 Top — sender → TX FIFO 64 → uart_tx

요약 **sender_tx_top : uart_sender 에 fifo · uart_tx · baud_tick_gen 연결, 사이는 NOT 2 개**



1 U_UART_SENDER

1 클럭에 1 글자 push

2 U_FIFO_TX (64)

sender → uart_tx

3 U_UART_TX

start · 8 비트 · stop

4 U_BAUD_TICK_GEN

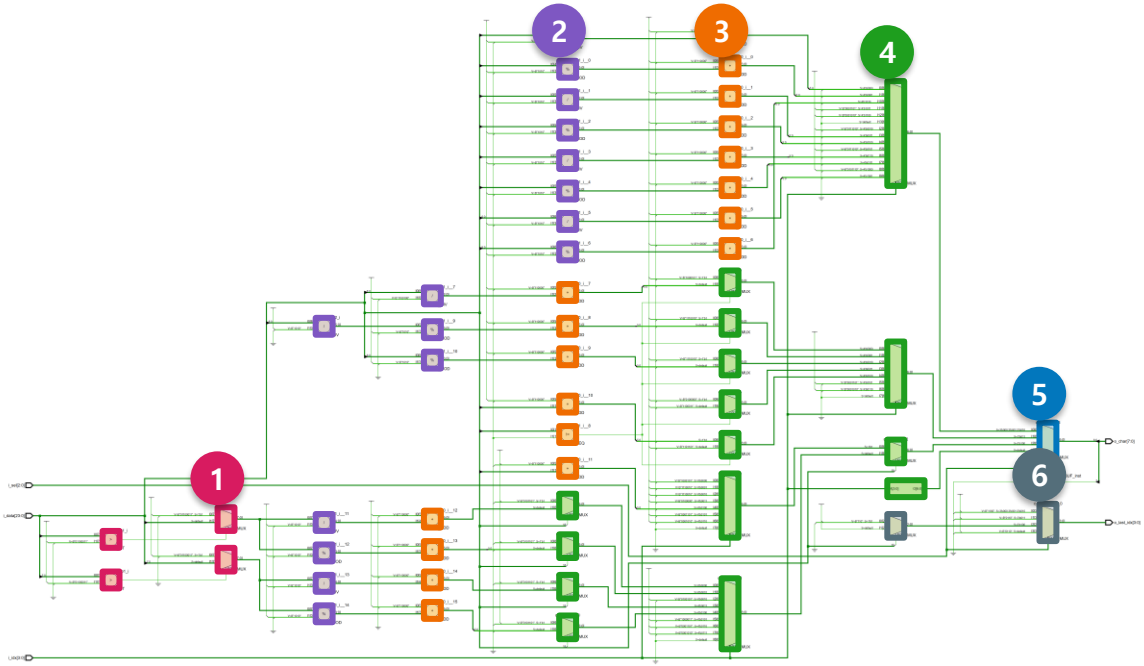
9600 × 16 tick

5 NOT 2 개

start = ~empty · pop = ~busy

sender RTL — 글자 MUX

요약 **조합회로 · 99 상한 → 자리 나누기 → 글자 → 형식 MUX → sel MUX**



1 99 상한
비교기 (> 99) · MUX → 최대 99

2 자리 나누기
÷10 · %10

3 글자로
+ "0" → 30 ~ 39

4 형식별 MUX
시간 · 거리 · 온습도 · ERROR

5 sel MUX
sel 로 한 글자 → o_char

6 o_last_idx
마지막 글자 12 · 6 · 7 · 6

Vivado RTL Schematic (sender_char_mux.v) · 색 네모 = 그 번호의 셀 · 래치 0

sender edge case ② — TX FIFO 가 꽉 차면 멈췄다가 이어서 (손실 0)

요약 3 글자 넣은 뒤 full 20 클럭 → 그 자리에서 기다렸다 이어서 → 13 글자 모두, busy 33 클럭



1 send_go
660 ns
23:59:59.99

2 full 20 클럭
700 → 900 ns

3 멈춤
c_idx 3 · push 0

4 이어서
900 ns 부터 "5"

5 판정
busy 33 = 13 + 20

결과
13 / 13 · 손실 0

UART sender 시뮬레이션 — 시나리오와 결과 한눈에

sender = 값 24 비트 + 형식 sel → 글자 줄 ("12:34:56.78" CR LF) 을 TX FIFO 에 한 글자씩 넣는 블록

검증	시나리오	확인한 것	결과	장
글자 MUX	① 스톱워치 시간 한 줄	시간 24 비트 → 13 글자	**13 / 13**	3
(조합회로)	② 거리 0 · 7 · 58 · 400	세 자리 0 채움 + cm	**28 / 28**	4
tb_sd1_char	③ 온습도 3 줄 + sel 5 · 6 · 7	45%25C · 잘못된 sel → ERROR	**45 / 45**	5
sender FSM	(1) 시간 한 줄	go → busy 13 클럭 · 1 클럭 1 글자	**err 0**	6
tb_sd2_sender	(2) ~ (4) 거리 · 온습도 · ERROR	busy = 글자 수 (7 · 8 · 7)	**err 0**	7
	(5) TX FIFO full 20 클럭	멈췄다 이어서 · 손실 0	**busy 33**	8
	(6) busy 중 send_go	무시 (ERROR 안 섞임)	**err 0**	9
	(7) 도중 데이터 변경	go 순간 값 유지	**err 0**	10
	(8) 5 종 연달아	busy 0 다음 클럭 go 받음 · 54 글자	**54 / 54**	11 · 12
	(9) (10) go 전 · LF 직전 full	첫 · 마지막 글자도 기다림	**busy 18 · 17**	13
	(11) 마지막 클럭 send_go	버려짐 → busy 0 보고 go	**err 0**	14
	(12) 도중 rst	즉시 IDLE · 다음 줄 정상	**cnt 5**	15
	(13) 랜덤 200 줄 + 무작위 full	기대값 모델과 모든 글자 비교	**err 0**	16
Top (sender	시간 한 줄 전체	busy 130 ns ↔ 선 13.49 ms	**13 / 13**	17
+ TX FIFO	시작 확대 60 ~ 265 ns	go → FIFO → uart_tx → start 비트	**35 ns**	18
+ uart_tx)	한 글자 프레임 "1"	LSB 먼저 · 비트 104.2 µs	**rcnt 1**	19
tb_sd3_top	5 종 연달아 54 글자	56.25 ms 쉬지 않고 전송	**67 / 67**	20
	5 종 busy 확대	TX FIFO 깊이 64 꽉 참 0 번	**full 0**	21

세 TB 모두 TOTAL MISMATCH = 0 (Vivado 2022.2 xsim, clk 100 MHz = 10 ns, UART 9600 bps)

파형 읽는 법 — i_send_data 24 비트는 "숫자 여러 개를 비트로 붙인 값"

파형에는 16 진수 (예 528f28) 로 보임 → 비트 구간을 나눠 10 진수로 읽으면 시간 · 거리 · 온습도

sel 0 · 1 · 2 (스톱워치 · 시계 · 타이머)



16 진	2 진 (시 | 분 | 초 | 1/100 초)	읽은 값
528f28	0101 0 | 010 100 | 0 1111 0 | 010 1000	→ **10 : 20 : 30 . 40**
645c4e	0110 0 | 100 010 | 1 1100 0 | 100 1110	→ **12 : 34 : 56 . 78**
bf7de3	1011 1 | 111 011 | 1 1101 1 | 110 0011	→ **23 : 59 : 59 . 99**

보내는 줄
시간 13 글자
12:34:56.78 CR LF
거리 7 글자
400cm CR LF
온습도 8 글자
45%25C CR LF
ERROR 7 글자
sel 5 · 6 · 7

sel 3 거리



000190 → 190_{16} = **400 cm**

sel 4 온습도

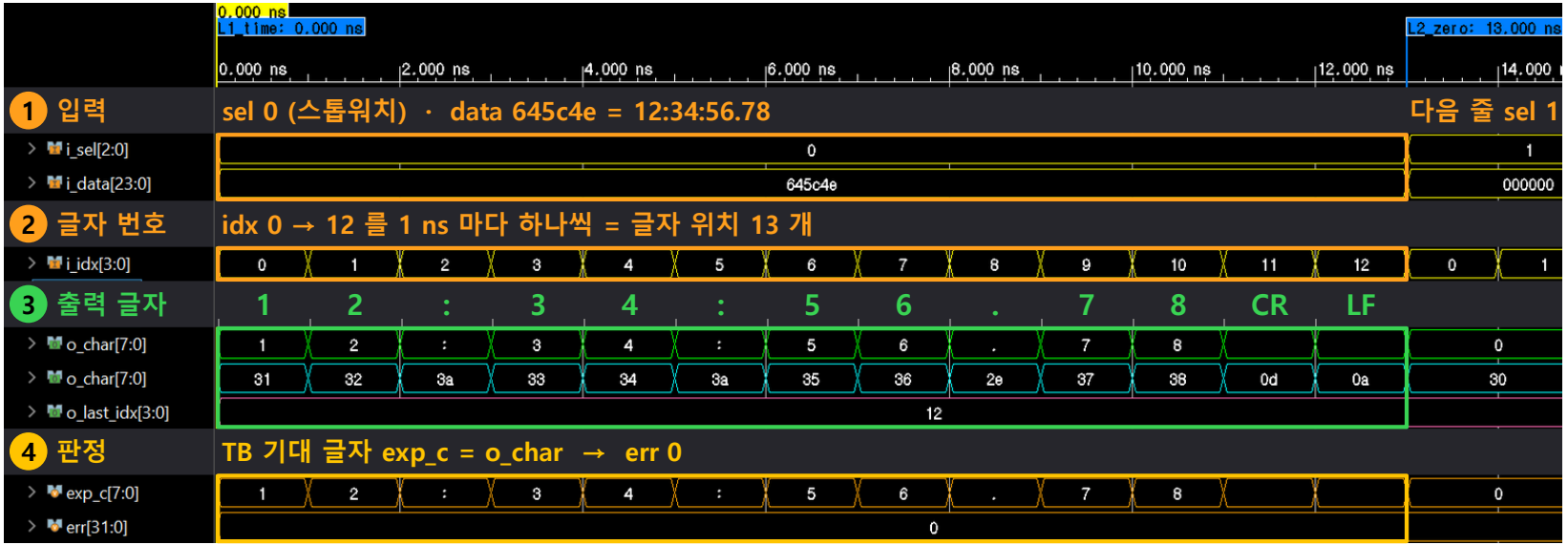


002d19 → 2d = 45 % · 19 = 25 ℃ → **45%25C**

글자 = 8'h30 + 자리 숫자 (예 "1" = 31, ":" = 3a, CR = 0d, LF = 0a) — 파형의 o_tx_push_data 줄

글자 MUX — 시간 24 비트 645c4e 를 글자 13 개 "12:34:56.78" + CR LF 로 바꿈

시나리오 sel 0 + data 645c4e 고정, idx 0 → 12 를 1 ns 마다 바꿔 글자 위치마다 글자 확인 (tb_sd1_char)



1 입력 : 시간 24 비트

645c4e = 시 5 · 분 6 · 초 6 · 1/100 초 7 비트
= 12 : 34 : 56 . 78

2 글자 번호 idx

sender FSM 이 줄 c_idx 자리.
0 → 12 = 글자 13 개

3 출력 o_char

숫자 = 8'h30 + 자리값
31 32 3a 33 34 … 0d 0a
마지막 글자 위치
o_last_idx = 12

4 판정

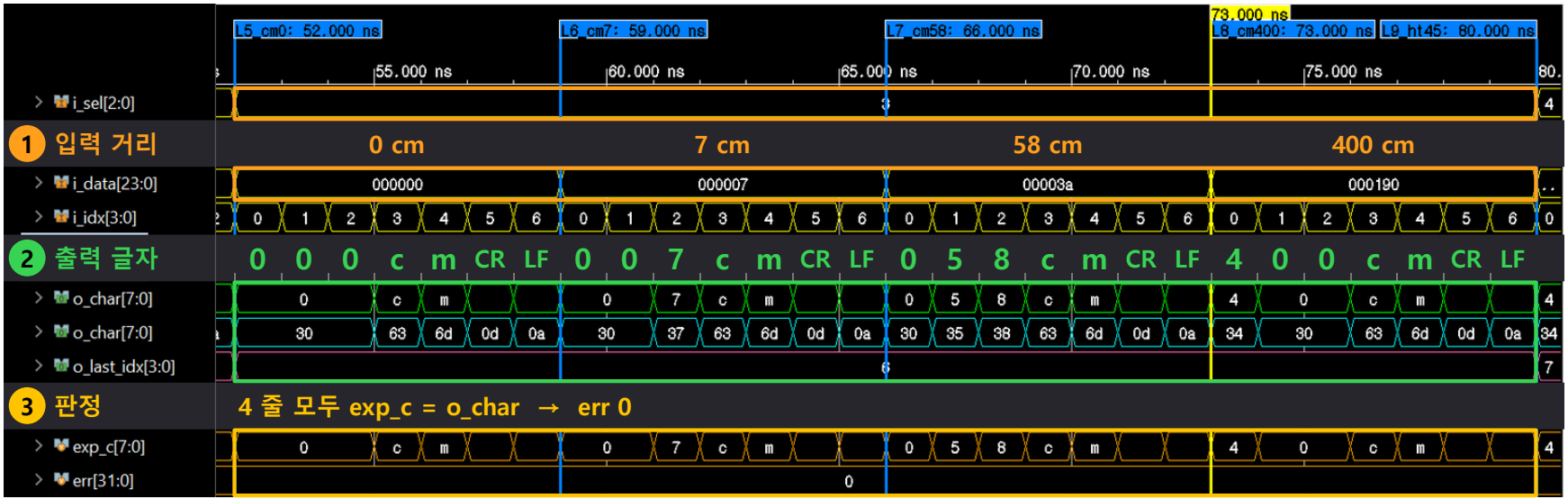
TB 가 따로 계산한 exp_c 와
글자 위치마다 비교 → err 0

결과

13 / 13 글자 일치
err 0
조합회로 : idx 가 바뀌면 같은 시각에 글자도 바뀜

글자 MUX — 거리는 늘 세 자리 + "cm" : 0 → 000cm · 7 → 007cm · 58 → 058cm · 400 → 400cm

시나리오 sel 3 (거리), data 0 · 7 · 58 · 400 (최소 · 한 자리 · 두 자리 · 최대) 한 줄씩, 줄마다 idx 0 → 6



1 입력 : 거리 9 비트

data = {15'b0, cm 9 비트}
000000 · 000007 · 00003a · 000190

2 출력 7 글자

백 · 십 · 일 자리 + c m CR LF
빈 자리는 0 → 늘 3 자리
o_last_idx = 6

3 판정

4 줄 × 7 글자를 exp_c 와 비교
→ err 0

4 파형 읽는 법

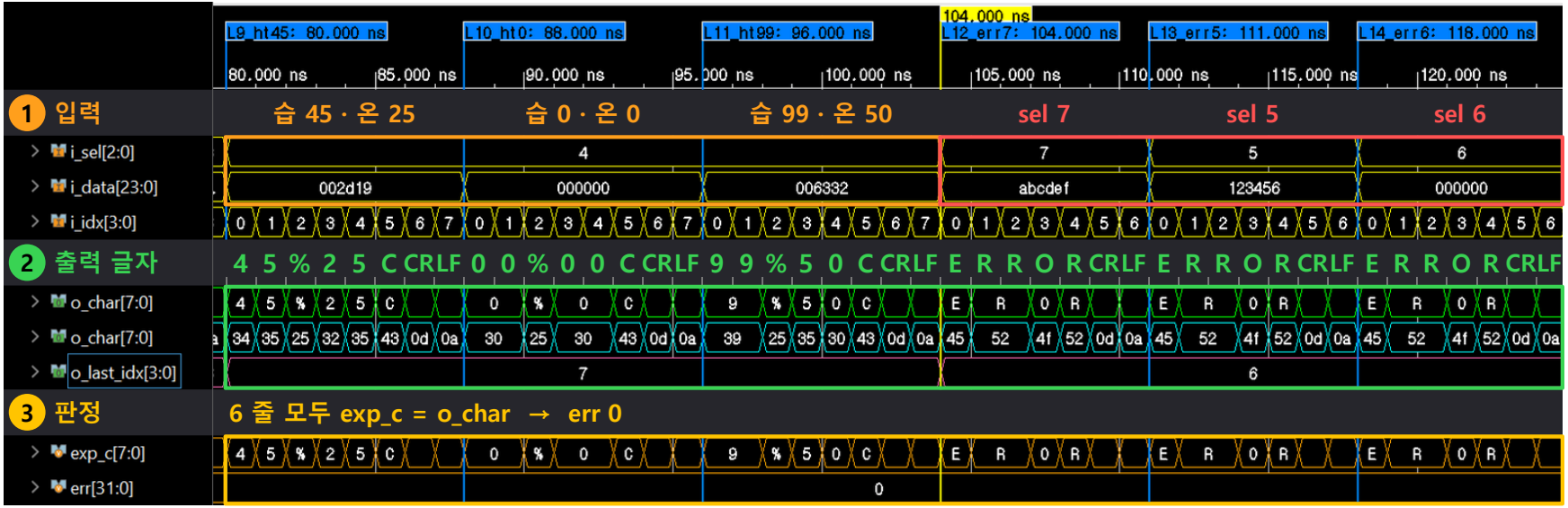
같은 값 (30 30 30) 이 이어지면
Vivado 는 한 구간으로 그림
→ 위 큰 글자가 실제 글자

결과

28 / 28 글자 일치
err 0
경계값 0 · 400 포함

글자 MUX — 온습도는 "45%25C", 안 쓰는 sel 5 · 6 · 7 은 데이터와 상관없이 "ERROR"

시나리오 sel 4 로 습 · 온 (45 · 25 / 0 · 0 / 99 · 50), 이어서 sel 7 · 5 · 6 에 아무 데이터 (abcdef · 123456 · 000000)



1 입력 : 습 · 온

data = {8'b0, 습 8, 온 8}
002d19 → 2d = 45 · 19 = 25

2 출력

온습도 8 글자 (o_last_idx 7)
ERROR 7 글자 (o_last_idx 6)

3 sel 5 · 6 · 7

data 가 abcdef · 123456 이어도
늘 E R R O R → 잘못된 sel 방어

4 판정

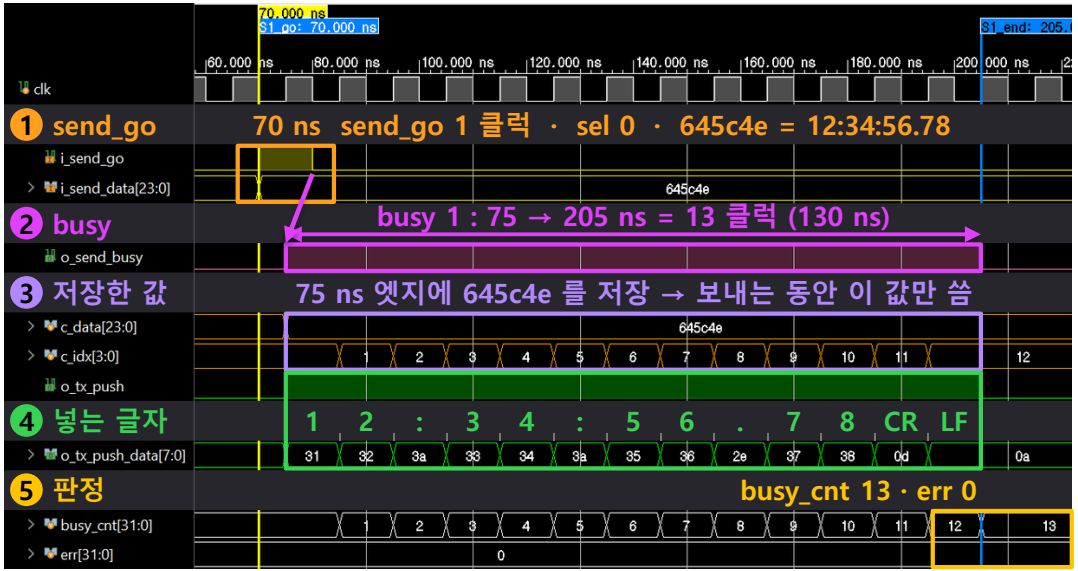
6 줄 45 글자 exp_c 비교 → err 0

결과

45 / 45 글자 일치
err 0
최소 0 · 최대 99 · 잘못된 sel 3 개 포함

sender — send_go 한 번에 글자 13 개를 1 클럭에 1 개씩 TX FIFO 에 넣음 (busy 13 클럭)

시나리오 (1) TX FIFO 비어 있음 (full 0), 70 ns 하강 엣지에 sel 0 + 12:34:56.78 로 send_go 1 클럭



1 send_go 70 ns
하강 엣지에 1 클럭 펄스
sel 0 · data 645c4e

2 busy 13 클럭
다음 상승 엣지 75 ns 에 SEND
13 글자 = 13 클럭 뒤 IDLE

3 c_data · c_idx
75 ns 에 645c4e 저장
c_idx 0 → 12 클럭마다 +1

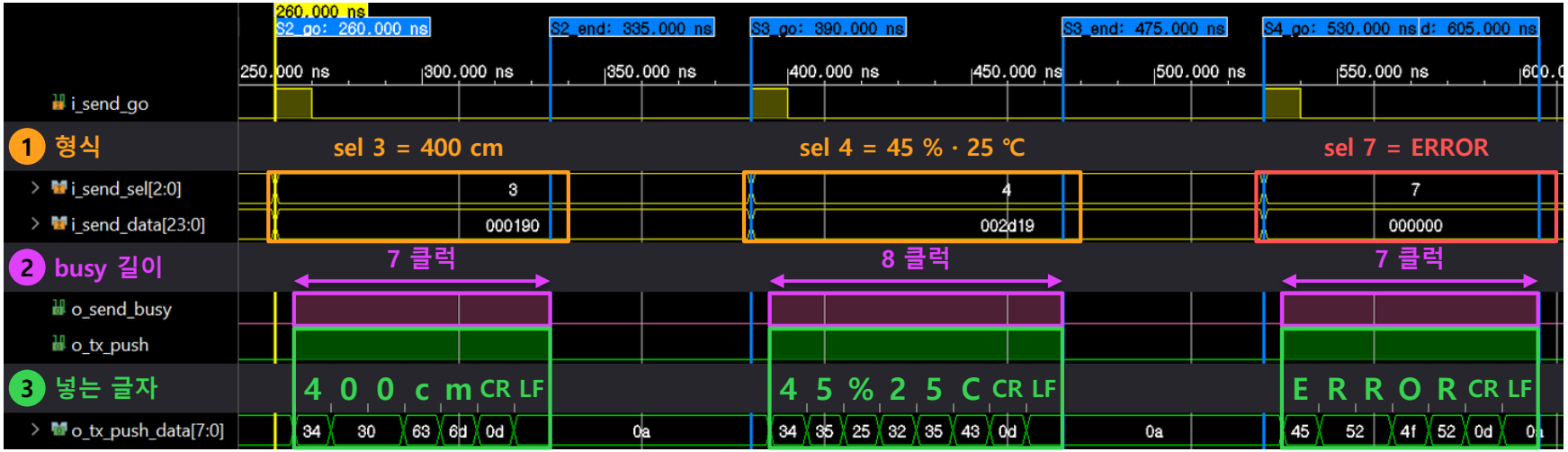
4 push · 글자
push 1 이 13 클럭 계속
31 32 3a … 0d 0a

5 판정
busy_cnt 13 받아 적은 13 글자 = 기대
err 0

결과
busy 13 클럭 = 130 ns
글자 13 / 13 일치 · err 0
go → SEND 반 클럭 (5 ns)

sender — 응답 길이만큼 busy : 거리 7 · 온습도 8 · ERROR 7 클럭

시나리오 (2) ~ (4) 거리 400 cm · 온습도 45 % 25 ℃ · ERROR 를 한 줄씩 send_go (줄 사이 쉼)



1 형식 3 종

sel 3 거리 · sel 4 온습도
sel 7 ERROR (data 무시)

2 busy 길이 = 글자 수

o_last_idx 로 줄 끝을 앎
7 · 8 · 7 클럭

3 넣는 글자

400cm · 45%25C · ERROR
+ CR LF

결과

busy 7 · 8 · 7 클럭
글자 22 / 22 일치 · err 0
고정 길이 13 이 아니라
형식마다 다름

색 입력 제어 (busy) 출력 → TX FIF 무시 · 리셋

sender — TX FIFO 가 꽉 차면 그 글자에서 멈췄다가, 풀리면 이어서 넣음 (글자 손실 0)

시나리오 (5) 23:59:59.99 를 보내다가 3 글자 ("23:") 넣은 뒤 TB 가 i_tx_full 을 20 클럭 동안 1



1 send_go 660 ns
sel 1 · bf7de3 = 23:59:59.99

2 full 20 클럭
700 → 900 ns
TX FIFO 가 꽉 찬 흉내

3 멈춤
push = SEND & ~full → 0
c_idx 3 · 글자 "5" 그대로

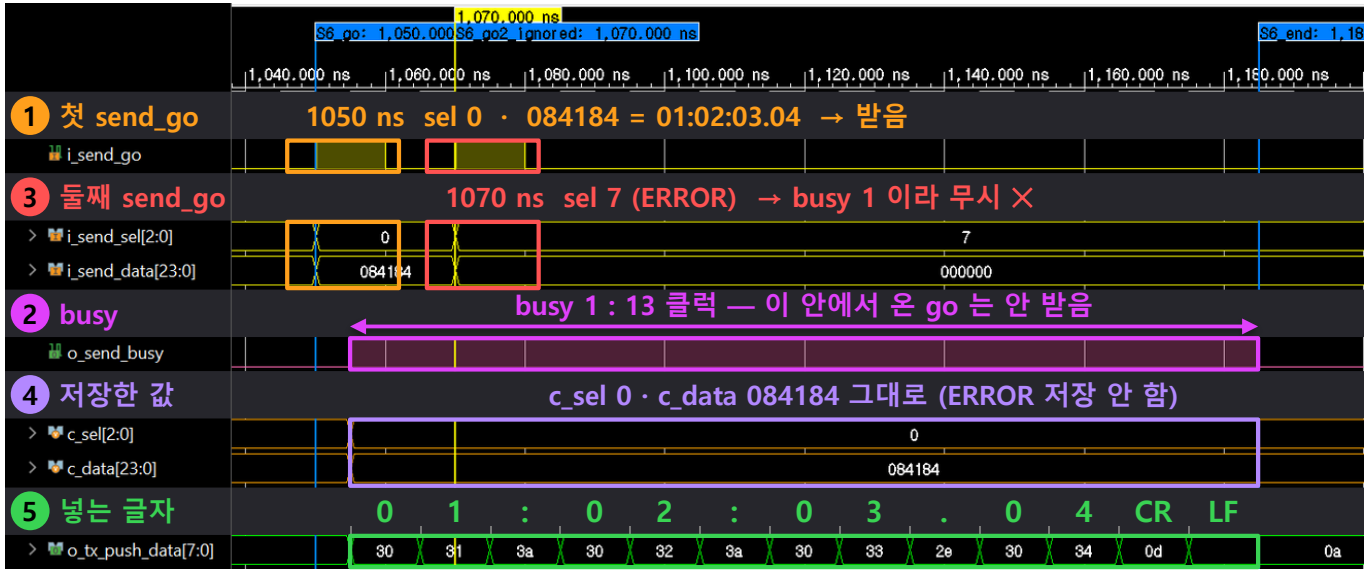
4 이어서
900 ns 뒤 5 9 : 5 9 . 9 9 CR LF 순서 그대로

5 판정
busy_cnt 33 = 13 + 20
full 중 push 0 번 · err 0

결과
busy 33 클럭 = 글자 13 + 기다림 20
글자 13 / 13 순서 그대로
full 중 push 0 (흘림 없음)

sender — 보내는 중 (busy 1) 에 온 send_go 는 형식과 상관없이 무시 (한 줄 안에 섞이지 않음)

시나리오 (6) 1050 ns 에 01:02:03.04 를 보내기 시작, 1070 ns 에 send_go 를 또 줌 — 둘째는 섞이면 바로 보이게 "E" 로 시작하는 sel 7



1 첫 go
1050 ns · sel 0 · 084184
→ 1055 ns 에 SEND

2 busy
1055 → 1185 ns
13 클럭
그대로 끝남

3 둘째 go
1070 ns · sel 7
SEND 중이라 FSM 이 안 봄

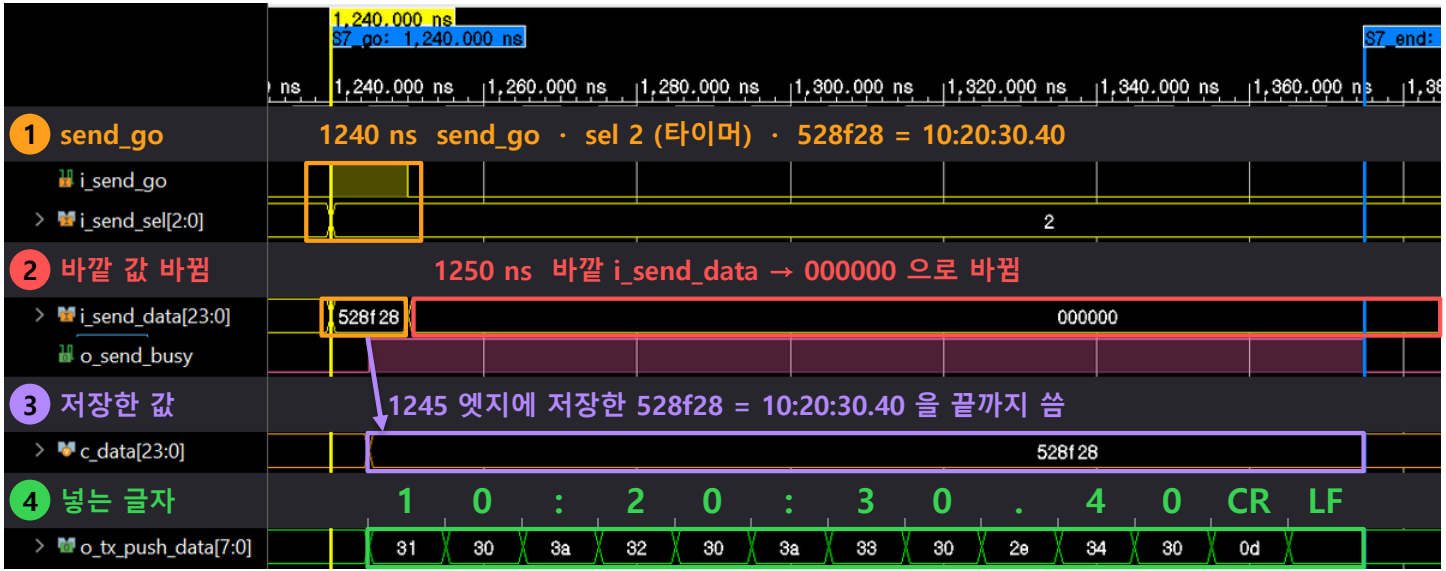
4 저장한 값
c_sel 0 · c_data 084184
바뀌지 않음

5 글자
01:02:03.04 CR LF
"E" (45) 없음

결과
둘째 go 무시
글자 13 / 13 = 01:02:03.04
→ U_CONTROL 은 busy 0 일 때만 go

sender — 보내는 도중 바깥 값이 바뀌어도 send_go 순간 값 (528f28) 을 끝까지 보냄

시나리오 (7) 1240 ns 에 10:20:30.40 으로 send_go, 1 클럭 뒤 TB 가 i_send_data 를 000000 으로 바꿈



1 send_go 순간 값
528f28 = 시 10 · 분 20 · 초 30 · 1/100 초 40

2 바깥 값 변경
1250 ns 에 000000 (보내는 중 시계 바뀜)

3 c_data
1245 ns 엣지에 저장 줄 끝까지 528f28

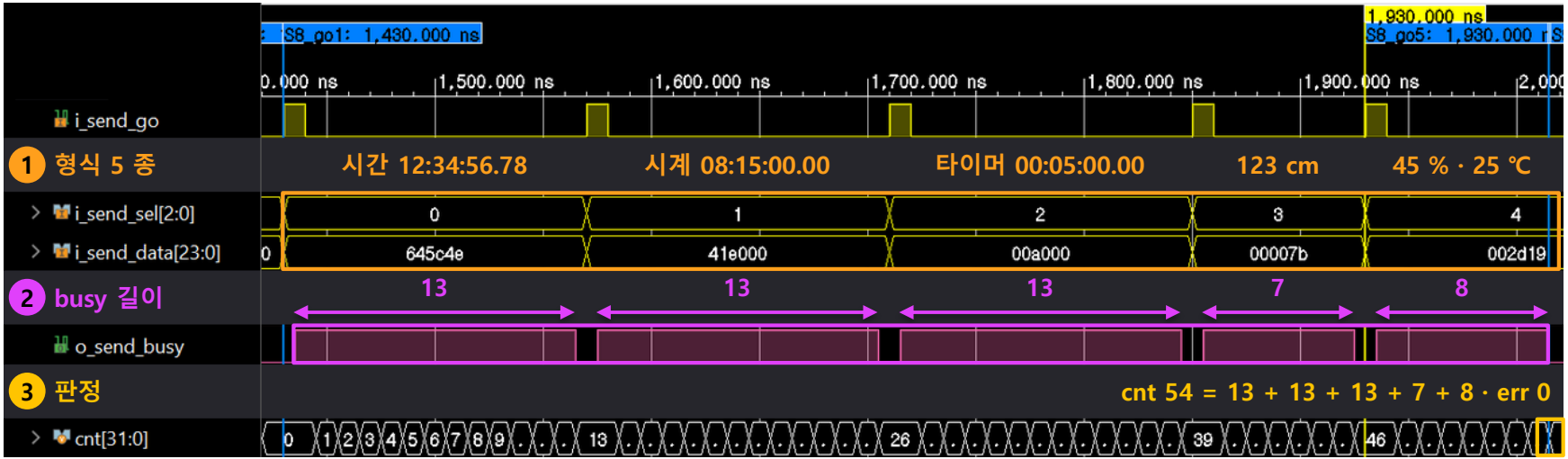
4 글자
10:20:30.40 CR LF 13 글자 그대로

결과
c_data 유지
글자 13 / 13 = 10:20:30.40
한 줄 안 값 안 섞임

색 입력 무시 · 리셋 저장한 값 출력 → TX FIFO

sender — 5 종을 연달아 : busy 가 0 이 된 다음 클럭에 바로 다음 send_go → 54 글자

시나리오 (8) /auto all 흉내 — sel 0 · 1 · 2 · 3 · 4 를 busy 가 0 되자마자 차례로 send_go



1 형식 5 종

스톱워치 · 시계 · 타이머 · 거리 · 온습도

2 busy 5 구간

13 · 13 · 13 · 7 · 8 클럭 사이 빈 클럭 1 개씩

3 판정

받아 적은 54 글자 = 기대 err 0

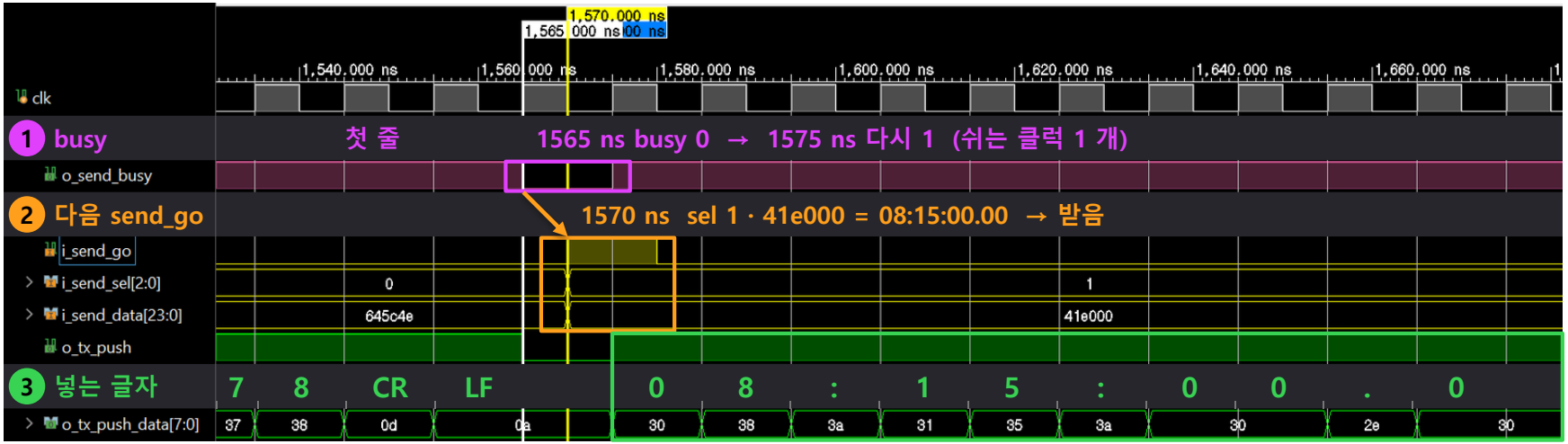
결과

54 / 54 글자 일치
1430 → 2015 ns (58.5 클럭)
TX FIFO 깊이 64 안

색 입력 제어 (busy) 판정

sender — busy 0 을 본 바로 다음 클럭의 send_go 는 받아짐 (빈 틈 1 클럭)

시나리오 (8) 확대 — 첫 줄 (스톱워치) 이 1565 ns 에 끝나고 1570 ns 하강 엣지에 시계 줄 send_go



1 1565 ns busy 0

첫 줄 LF 를 넣은 엣지에서 IDLE

2 1570 ns send_go

U_CONTROL 이 busy 0 을 보고 다음 하강 엣지에 go

3 1575 ns 다음 줄 시작

0 8 : 1 5 : 0 0 . 0 0 ...
쉬는 클럭 1 개

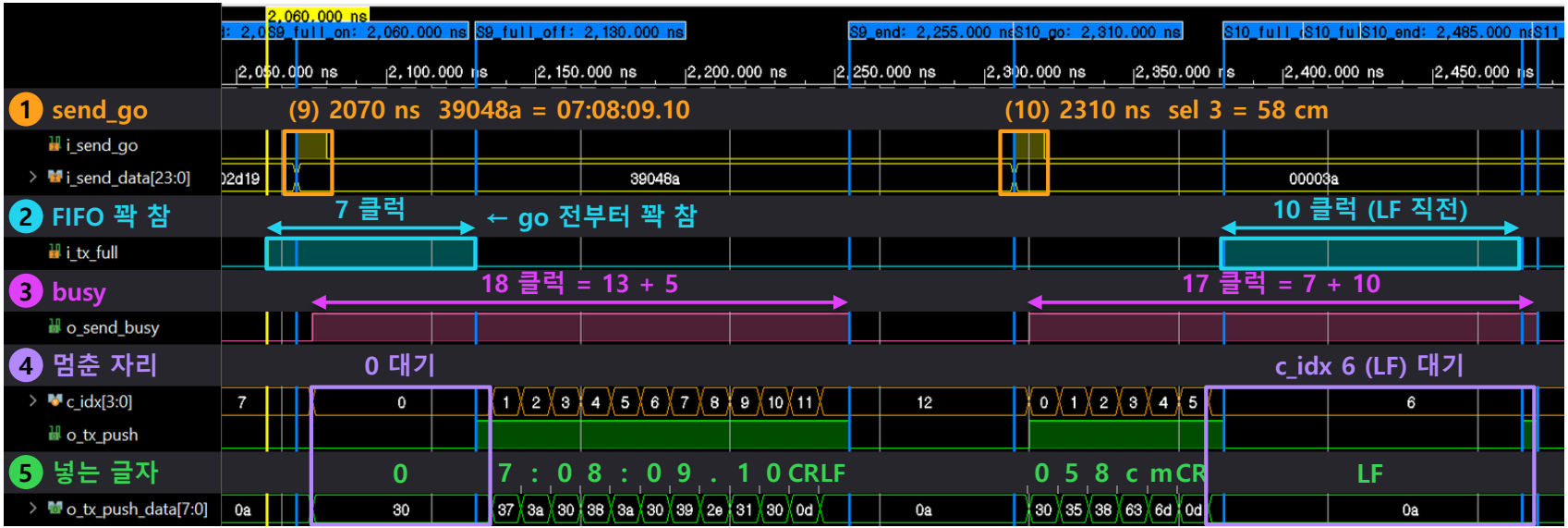
결과

busy 0 다음 클럭 go = 받음
줄 사이 빈 틈 10 ns
↔ 다음 장 : busy 1 마지막 클럭 go = 버림

색 입력 제어 (busy) 출력 → TX FIFO

sender — full 이 go 전부터 와도, 마지막 LF 직전에 와도 기다렸다가 글자 하나 안 빠짐

시나리오 (9) full 을 먼저 켠 채 send_go (5 클럭 뒤 풀림) (10) "058cm CR" 까지 넣은 뒤 LF 앞에서 full 10 클럭



1 send_go 두 번
(9) 07:08:09.10
(10) 58 cm

2 full
(9) 2060 → 2130 ns
(10) 2380 → 2480 ns

3 busy 늘어남
(9) 13 + 5 = 18 클럭
(10) 7 + 10 = 17 클럭

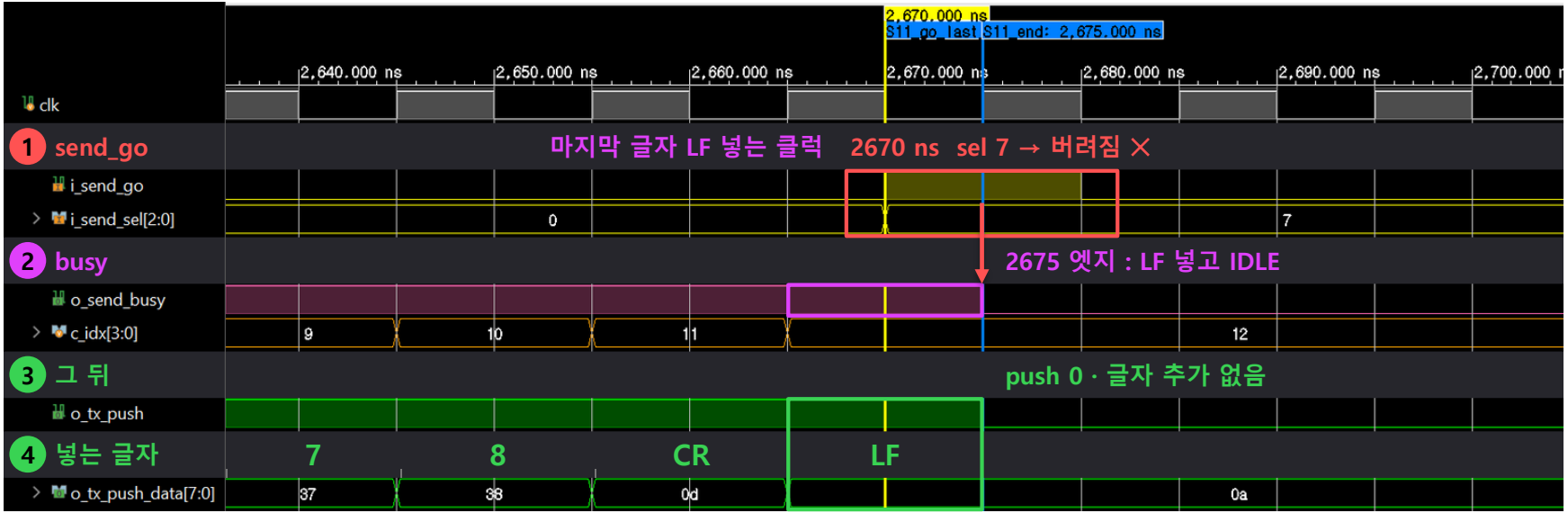
4 멈춘 자리
(9) 첫 글자 c_idx 0
(10) 마지막 LF c_idx 6

5 글자
07:08:09.10 CR LF
058cm CR LF 순서 그대로

결과
두 줄 20 / 20 글자 일치
busy = 글자 수 + full 클럭
첫 · 마지막 글자도 안 빠짐

sender — busy 1 인 마지막 클럭 (LF 넣는 엣지) 에 걸친 send_go 는 버려짐

시나리오 (11) 한 줄의 마지막 글자 LF 를 넣는 2675 ns 엣지에 sel 7 send_go (2670 ~ 2680 ns) 를 겹쳐 줌



1 2670 ns send_go

아직 busy 1 (SEND)
FSM 은 SEND 에서 go 를 안 봄

2 2675 ns 엣지

LF 를 넣고 IDLE
같은 엣지라 go 를 못 받음

3 그 뒤

push 0 · 글자 0 개
(TB cnt 13 그대로)

결과
마지막 클럭 go = 버림
규칙 : busy 0 을 본 뒤 go
(앞 장 : 그러면 다음 클럭에 받음)

색 무시 · 리셋 제어 (busy) 출력 → TX FIFO

sender — 보내는 도중 rst : 그 순간 IDLE · busy 0, 남은 글자는 안 나가고 다음 go 는 정상

시나리오 (12) 23:00:00.00 을 보내다 5 글자 ("23:00") 넣은 뒤 2800 ns 에 rst 1 클럭, 이어서 3020 ns 에 7 cm



1 send_go 2740 n
b80000 = 23:00:00.00
5 글자 넣는 중

2 rst 2800 ns
비동기 리셋
클럭 엣지 안 기다림

3 IDLE
busy 0 · c_idx 0
2800 ns 바로

4 다음 go
3020 ns 7 cm
→ 007cm CR LF
정상

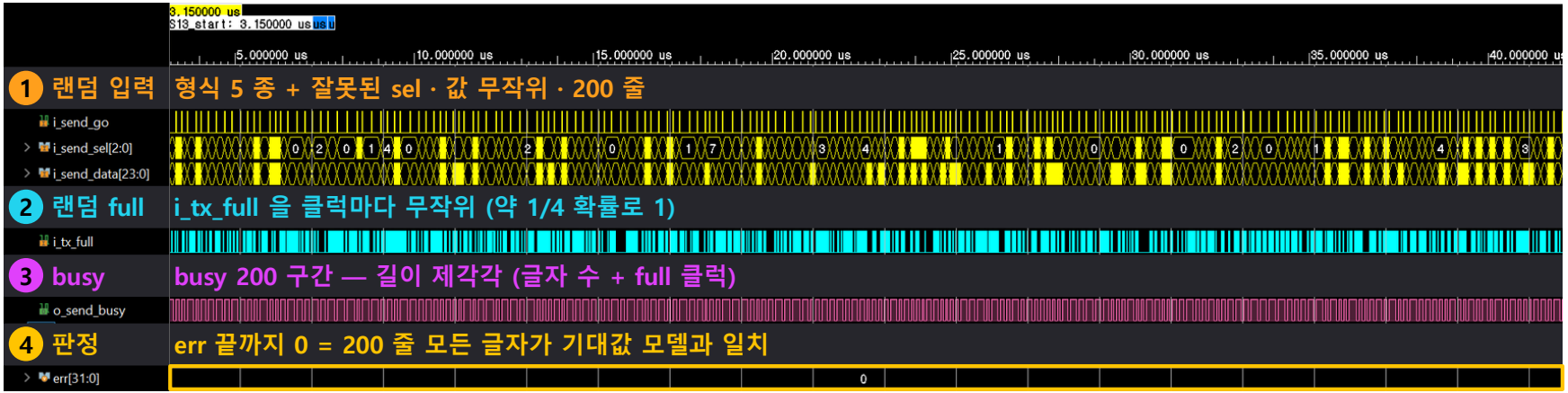
5 판정
cnt 5 그대로
err 0

결과
rst 즉시 IDLE
남은 8 글자 안 나감
rst 뒤 다음 줄 정상

색 무시 · 리셋 입력 제어 (busy) 출력 → TX FIF 판정

sender — 랜덤 200 줄 + 무작위 full : 모든 글자 일치 (err 0)

시나리오 (13) 3.15 µs 부터 형식 · 값 · full 위치를 무작위로 200 줄, 기대 글자는 TB 의 make_exp 모델이 따로 계산



1 랜덤 입력

sel 0 ~ 7, 시 · 분 · 초 · cm
· 습 · 온 범위 안 무작위

2 랜덤 full

클럭마다 약 1/4
어느 글자에서든 멈춤

3 busy

200 구간
길이 = 글자 + 기다림

4 판정

받아 적은 글자 vs
make_exp → err 0

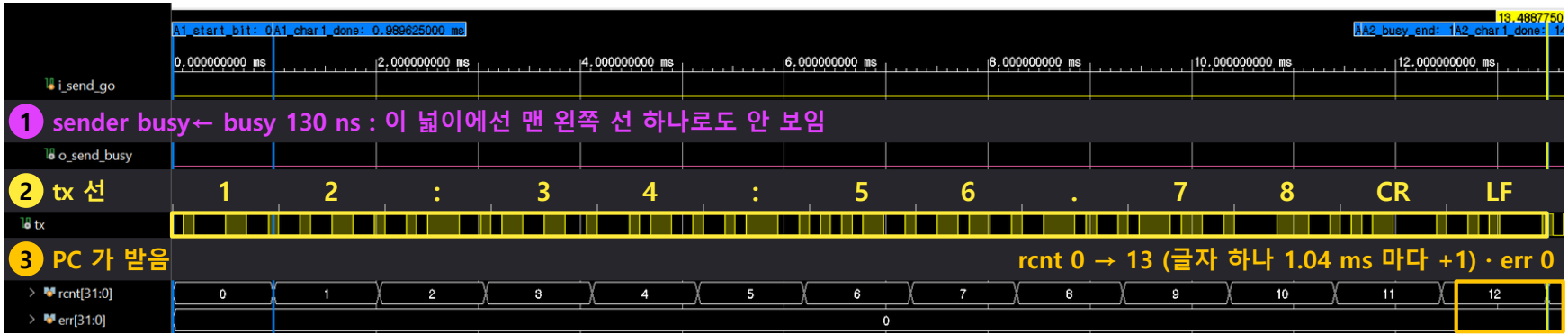
결과

200 줄 err 0
TOTAL MISMATCH 0
(1) ~ (12) 정해진 시나리오
+ (13) 랜덤

색 입력 FIFO 꽉 참 제어 (busy) 판정

Top — sender 는 130 ns 만에 끝, 선으로 나가는 13.5 ms 는 TX FIFO · uart_tx 몫

시나리오 tb_sd3_top : sender + TX FIFO 64 + uart_tx 연결, 70 ns 에 12:34:56.78 send_go, TB 가 PC 처럼 tx 선을 읽음



1 sender busy 130 ns

13 글자를 TX FIFO 에 넣으면 끝
(75 → 205 ns)

2 TX FIFO

13 글자를 받아 두고 uart_tx 가 하나씩 꺼냄

3 tx 선 13.49 ms

9600 bps · 글자 하나 10 비트
= 1.04 ms × 13

4 PC 흉내 TB

stop 비트 가운데서 받아 적음
rcnt 13 · err 0

결과

busy 130 ns ↔ 선 13.49 ms
약 10 만 배 차이
13 / 13 글자 일치

색 제어 (busy) tx 선 · PC 판정

Top — 첫 글자가 sender → TX FIFO → uart_tx 로 넘어가 105 ns 에 start 비트가 나감

시나리오 tb_sd3_top 시작 확대 : 70 ns send_go (sel 0 · 12:34:56.78) 직후 60 ~ 265 ns



1 75 ns ~ sender
1 클럭에 1 글자 push
205 ns 까지 13 글자

2 85 ns FIFO
첫 글자 저장
wptr 1 · empty 0

3 95 ns 꺼냄
uart_tx 가 pop
rptr 1

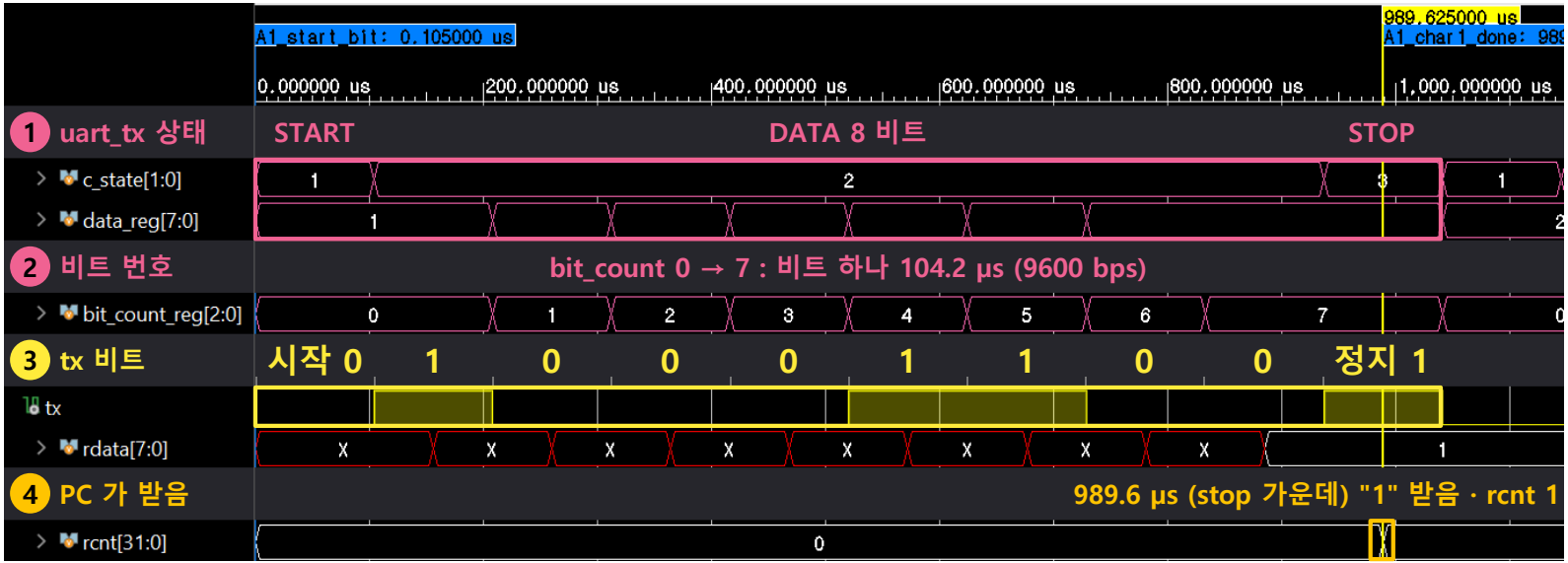
4 95 ns uart_tx
c_state START
data_reg = "1" (31)

5 105 ns tx
start 비트 0 선으로 나가기 시작

결과
go → start 비트 35 ns
205 ns : FIFO 에 12 글자 대기
(wptr 13 · rptr 1)

uart_tx — 글자 "1" (31 = 0011_0001) 을 LSB 먼저 1 0 0 0 1 1 0 0, 비트 하나 104.2 µs

시나리오 tb_sd3_top 첫 글자 프레임 확대 : 105 ns start 비트부터 stop 비트까지 (0 ~ 1.15 ms)



1 c_state

1 START → 2 DATA → 3 STOP
→ 다음 글자 START

2 data_reg · bit_count

data_reg "1" (31)
bit_count 0 → 7

3 tx 비트열

31 = 0011_0001
LSB 먼저 → 1 0 0 0 1 1 0 0

4 PC 흉내 TB

stop 비트 가운데 989.6 µs
rdata "1" · rcnt 1

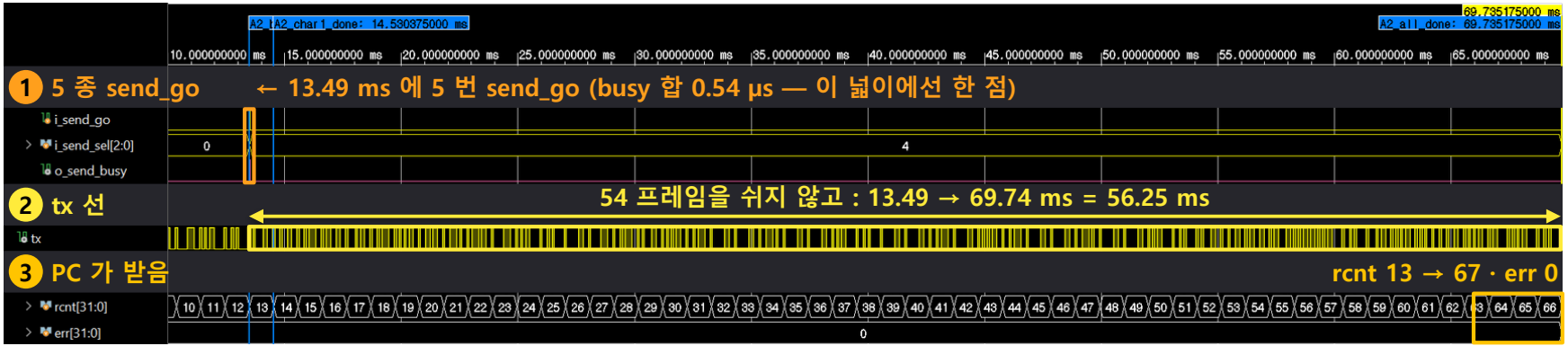
결과

한 글자 = 10 비트 = 1.04 ms
9600 bps (104.17 µs / 비트)
rdata "1" · err 0

색 uart_tx tx 선 · PC 판정

Top — 5 종 54 글자를 한꺼번에 넣어도 TX FIFO 가 받아 두고 uart_tx 가 56 ms 동안 쉬지 않고 보냄

시나리오 tb_sd3_top 둘째 부분 : 첫 줄이 끝난 13.49 ms 에 /auto all 흉내로 sel 0 ~ 4 send_go 5 번



1 5 번 go

13,488,780 → 13,489,280 ns
busy 합 54 클럭

2 TX FIFO

54 글자를 0.6 µs 만에 받음
(깊이 64 안)

3 tx 54 프레임

1.04 ms × 54 = 56.25 ms
프레임 사이 쉼 없음

4 PC 흉내 TB

받은 글자 67 개
= 첫 줄 13 + 54 · err 0

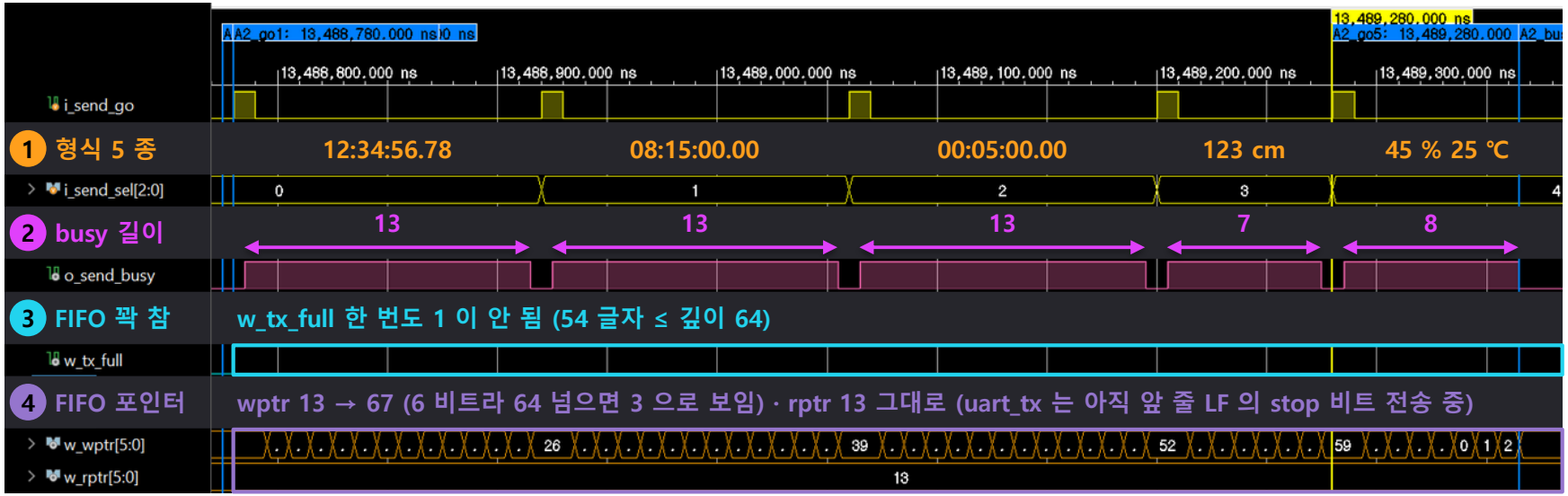
결과

67 / 67 글자 일치
sender 0.6 µs ↔ 선 56 ms
FIFO 꽉 참 0 번 (다음 장)

색 입력 tx 선 · PC 판정

Top — 5 줄 54 글자를 0.6 µs 만에 넣어도 TX FIFO (깊이 64) 는 한 번도 꽉 차지 않음

시나리오 앞 장의 13.49 ms 부분 확대 : 13,488,770 ~ 13,489,385 ns



1 5 번 go
busy 0 다음 클럭마다 다음 줄 send_go

2 busy
13 · 13 · 13 · 7 · 8 클럭 = 54 글자

3 full 0
TX FIFO 깊이 64 > 54 글자
→ sender 가 안 기다림

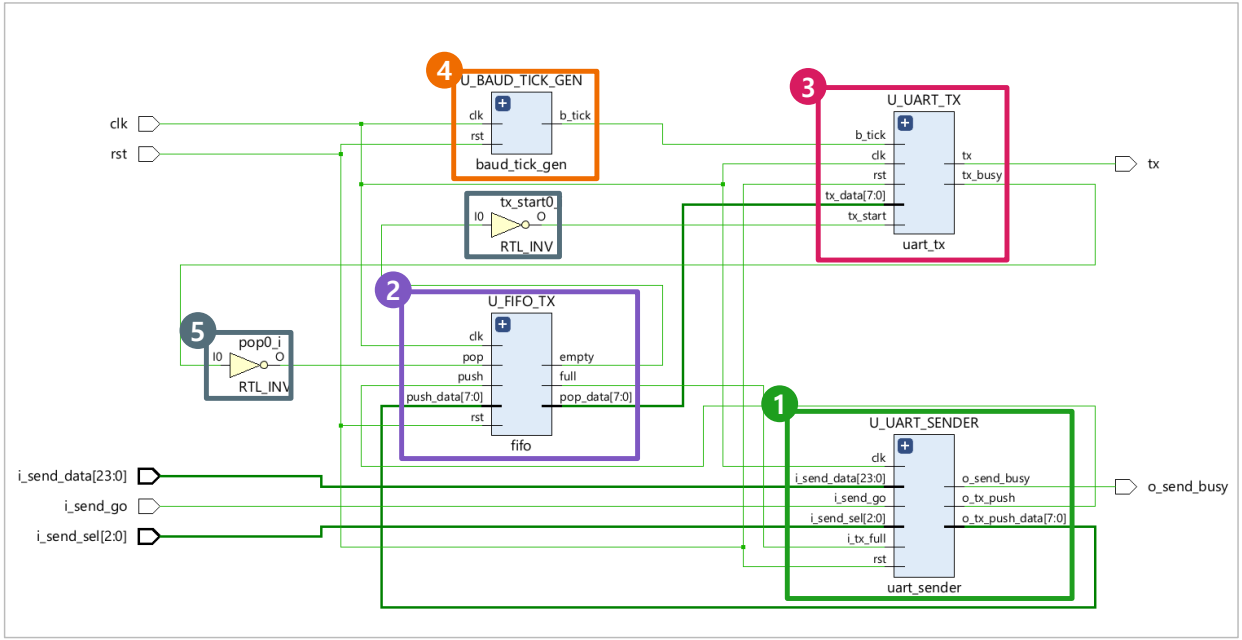
4 포인터
wptr 13 → 67 (= 3)
rptr 13 (앞 줄 stop 비트 중)

결과
full 0 번 · 손실 0
54 글자 ≤ 깊이 64
→ busy 가 짧게 끝남

색 입력 제어 (busy) FIFO 꽉 참 TX FIFO

Top RTL — sender → TX FIFO (64) → uart_tx → tx, 사이 연결은 NOT 두 개

시나리오 sender_tx_top : 내 uart_sender 에 fifo · uart_tx · baud_tick_gen 을 이어 붙인 시험용 Top (RTL Schematic)



1 U_UART_SENDER (내 설계)

send_go · sel · data → 1 클럭 1 글자
o_tx_push · o_tx_push_data
i_tx_full ← FIFO full

2 U_FIFO_TX (깊이 64)

#(.DEPTH(64)) 로 fifo 깊이만 64 로 바꿈
push ← sender · pop_data → uart_tx

3 U_UART_TX

tx_start 이면 tx_data 를
start · 8 비트 · stop 으로 tx 에

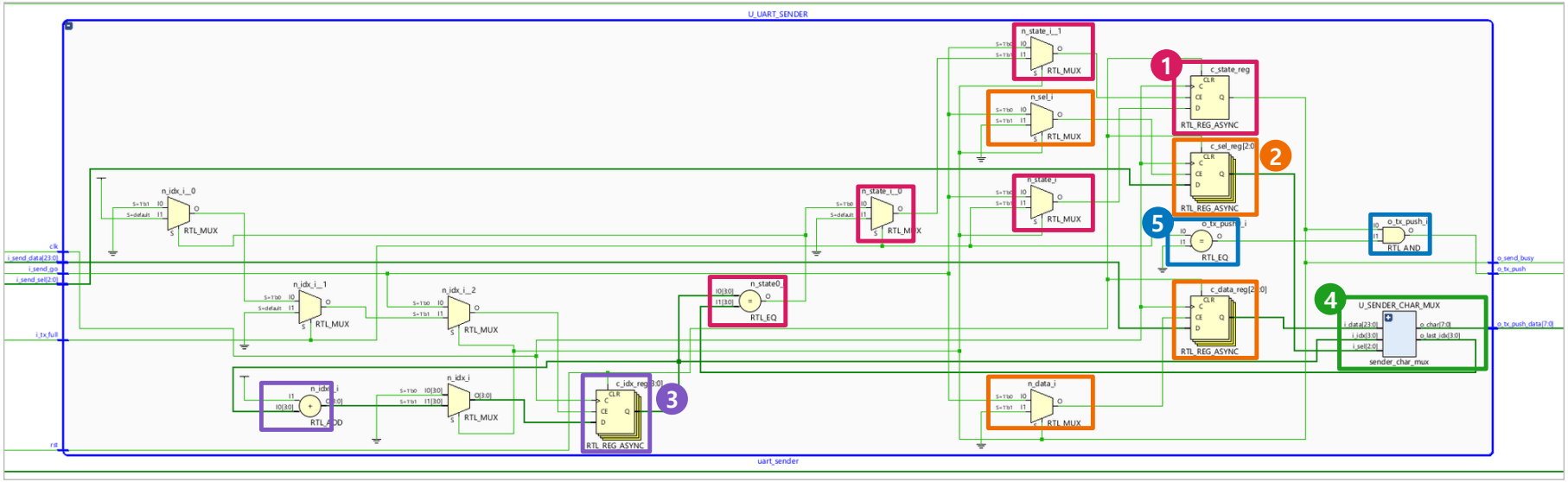
4 U_BAUD_TICK_GEN

9600 × 16 tick (b_tick)
→ uart_tx 비트 박자

5 RTL_INV 2 개 (연결)

uart_sender RTL — 상태 1 비트 (c_state) + 잡아 두는 레지스터 3 개 + 글자 MUX

시나리오 U_UART_SENDER 펼친 화면 : 레지스터는 모두 RTL_REG_ASYNC (비동기 rst), 래치 없음



1 CU : c_state (IDLE · SEN

c_state_reg 1 비트
n_state MUX + RTL_EQ
(c_idx == 마지막 글자 위치 → IDLE)

2 DP : c_sel · c_data

go 때만 새 값 (n_sel · n_data MUX)
보내는 동안 바깥 값 무시

3 DP : c_idx 카운터

RTL_ADD (+1)
4 비트 레지스터

4 U_SENDER_CHAR_MUX

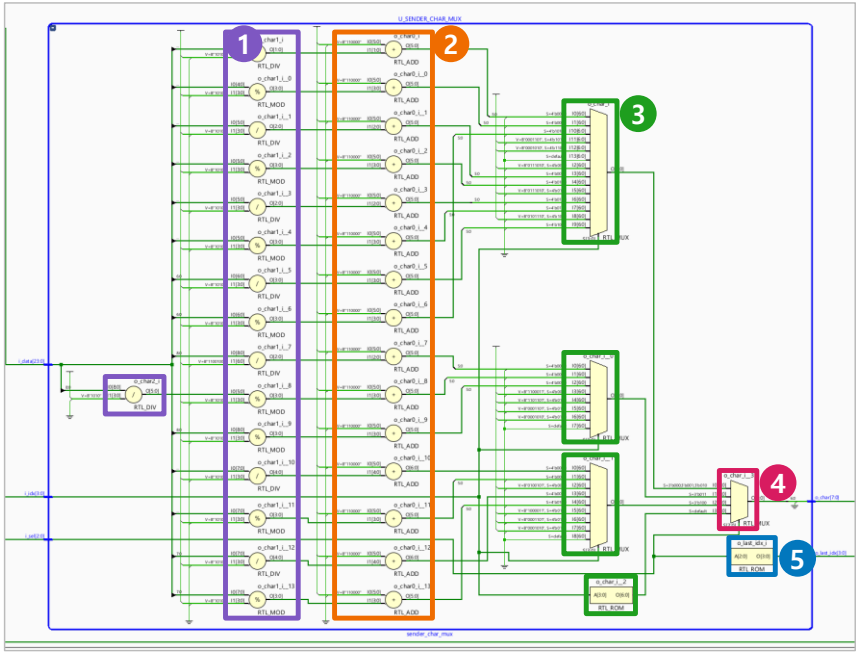
sel · data · idx →
o_char · o_last_idx

5 o_tx_push

RTL_EQ (SEND) &
~i_tx_full
= RTL_AND

글자 MUX RTL — ÷10 · %10 로 자리 나누고 +"0" 해서 형식별 MUX → sel 로 고름

시나리오 U_SENDER_CHAR_MUX 펼친 화면 : 조합회로만 (레지스터 0 개)



1 자리 나누기

RTL_DIV · RTL_MOD (÷10 · %10)
시 · 분 · 초 · 1/100 · 습 · 온
거리 9 비트는 ÷100 · ÷10 · %10

2 글자로

RTL_ADD : + 8'b110000 (= "0")
숫자 0 ~ 9 → 30 ~ 39

3 형식별 MUX (idx)

시간 13 · 거리 7 · 온습도 8 글자
ERROR 는 RTL_ROM

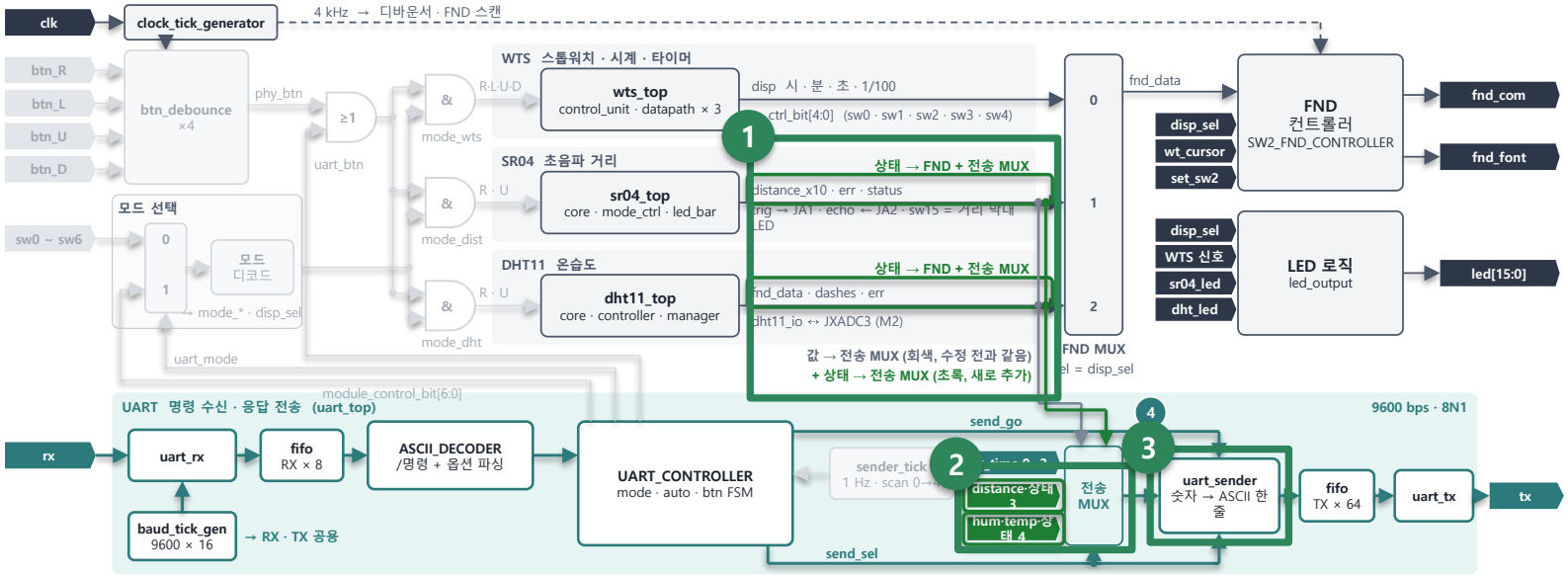
4 sel MUX

sel 로 형식 하나 고름
→ o_char

5 o_last_idx ROM

트러블 슈팅 ③ 해결 — 송신 MUX 에 상태 비트를 같이 실음

③ 해결 **24 비트의 빈 윗비트에 상태 → sender 글자 MUX 가 Err N · --%--C (블록 · 포트 그대로)**



디코더 : "/get " + Enter

컨트롤러 : /mod 모드 → send_sel (온습도 4 · 거리 3 · ...)

전송 MUX : *_time · distance · hum·temp 중 선택

1 상태 선

FND + 송신 MUX

2 송신 MUX 입력

값 + 상태 비트

3 글자 MUX

Err 1 ~ 3 · --%--C

트러블 슈팅 ③ 해결 상세 — 바꾼 코드 2 곳

③ 해결 **노란 줄 = 바뀐 줄 · 파일 2 개 (merge_final_top.v · sender_char_mux.v)**

① merge_final_top.v 송신 MUX

```
3'd3: w_send_data = {15'd0, w_distance};                    // 수정 전
3'd3: w_send_data = {13'd0, w_dist_status, w_distance};     // 수정 후
3'd4: w_send_data = {4'd0, w_dht_err, w_dht_dashes, w_dht_status,
                     w_humidity, w_temperature};            // 수정 후
```

② sender_char_mux.v 거리 (온습도도 같은 방식)

```
wire [1:0] w_dst = i_data[10:9];                    // 거리 상태
4'd0: if (w_dst != 2'b00) o_char = "E";  else o_char = 8'h30 + w_cm / 100;
  ... 1 · 2 = "r" · 3 = 공백
4'd4: if (w_dst != 2'b00) o_char = 8'h30 + w_dst;  else o_char = "m";
```

구현 LUT 1,473 → 1,489 (+16) · WNS +1.387 → +0.320 ns (위반 0) · 통합 TB 560 검사 일치

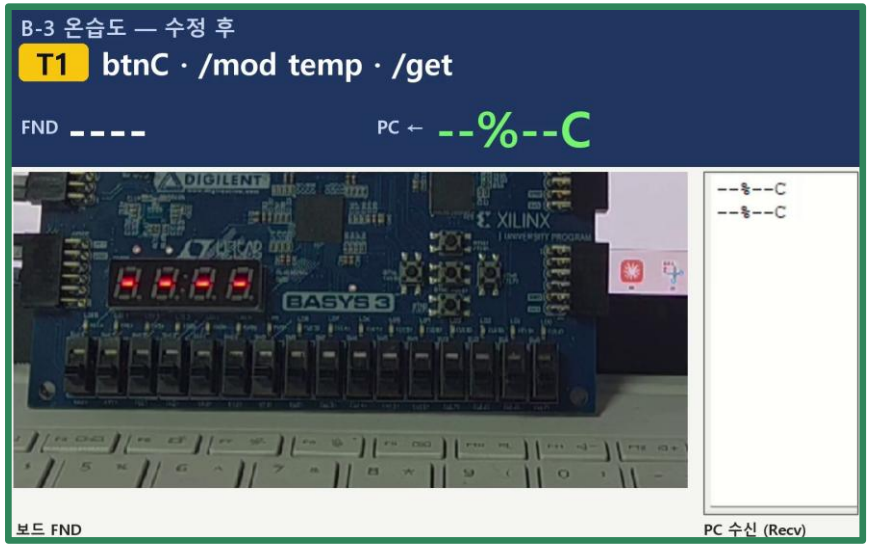
트러블 슈팅 ④ 적용 결과 — B-3 온습도 (보드)

④ 적용 결과 같은 조작에서 PC 00%00C · 32%24C → --%--C · Err 1

수정 전



수정 후



단계	조작	FND	수정 전 PC	수정 후 PC
T1	리셋 → /get	----	00%00C	--%--C
T3	DATA 선 뽑기 → /get	**Err1**	32%24C	Err 1

B-2 · B-3 수정 결과 — FND 가 오류 · 측정 전이면 PC 도 같게 (시뮬 불일치 0 · 보드 확인)

송신 데이터 24 비트의 빈 윗비트에 상태를 넣음 → sender 가 그 비트로 글자를 고름

상황	FND	수정 전 PC	수정 후 PC
거리 범위 밖 (Err2)	Err2	400cm	**Err 2**
거리 무응답 (Err1)	Err1	000cm	**Err 1**
거리 echo 고착 (Err3)	Err3	400cm (보드)	**Err 3**
온습도 첫 측정 전	----	00%00C	**--%--C**
온습도 오류 (Err1~3)	Err1~3	마지막 정상값	**Err 1~3**
정상	숫자	숫자	**숫자 (같음)**

왜 고쳤나
FND 는 Err2 인데 PC 는 400cm → PC 만 보는 쪽은 진짜 400 cm 와 범위 밖을 구분 못 함. 온습도도 측정 전 · 오류에 숫자가 가서 정상값으로 오해.

송신 데이터 24 비트 (빈 윗비트 사용)

거리 수정 전 {15'd0, 거리 9}

거리 수정 후 **{13'd0, 상태 2, 거리 9}**

온습도 수정 전 {8'd0, 습도 8, 온도 8}

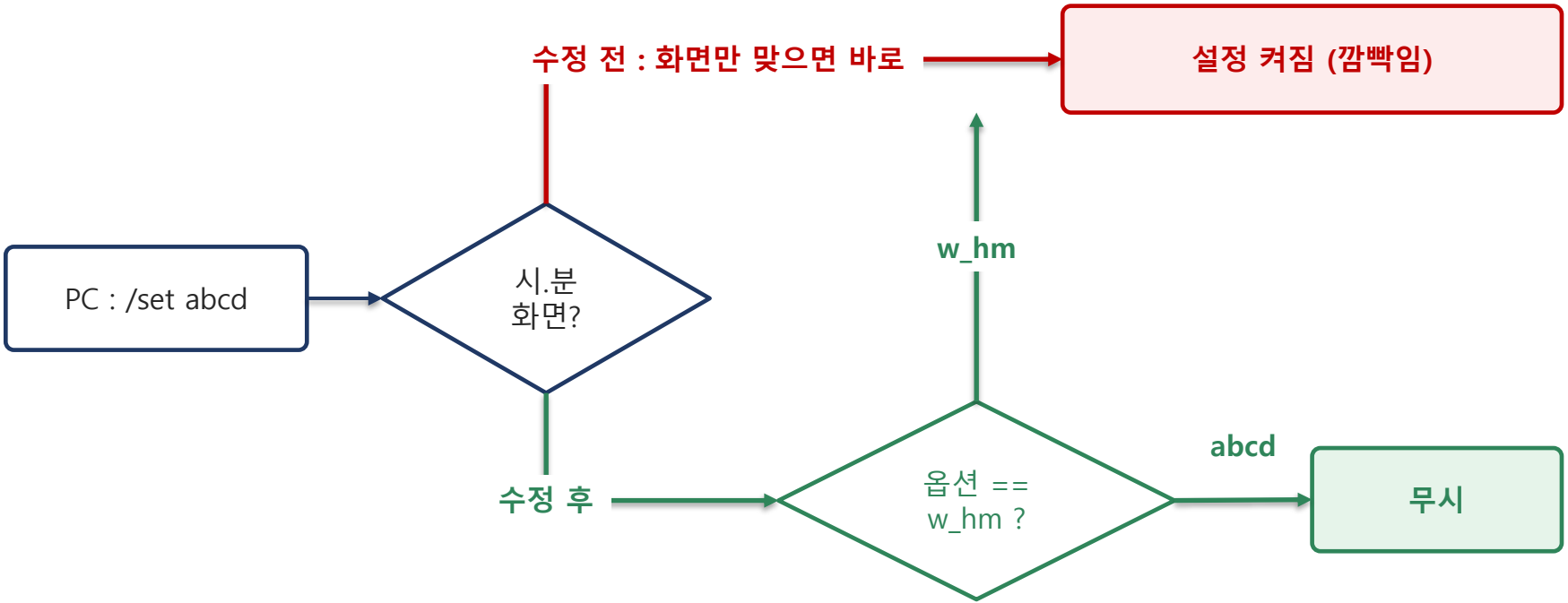
온습도 수정 후 **{4'd0, err, dashes, 상태 2, 습도 8, 온도 8}**

확인

· 통합 TB (20 시나리오 · 랜덤 명령 120) : 검사 560, 불일치 0
(같은 TB 로 수정 전도 560 · 0 → 다른 동작 안 깨짐)
· 보드 (10/31) : Err 2 · --%--C · Err 1 · Err 3 확인
· 구현 : LUT 1,473 → 1,489 (+16), FF 737 같음,
WNS +1.387 → +0.320 ns (위반 0, 최악 = 리셋 배선)

G3 — /set 뒤에 아무 4 글자나 와도 시계 설정이 켜짐

1 / 8 **담당 코드 : UART 컨트롤러**



G3 — 바뀐 코드

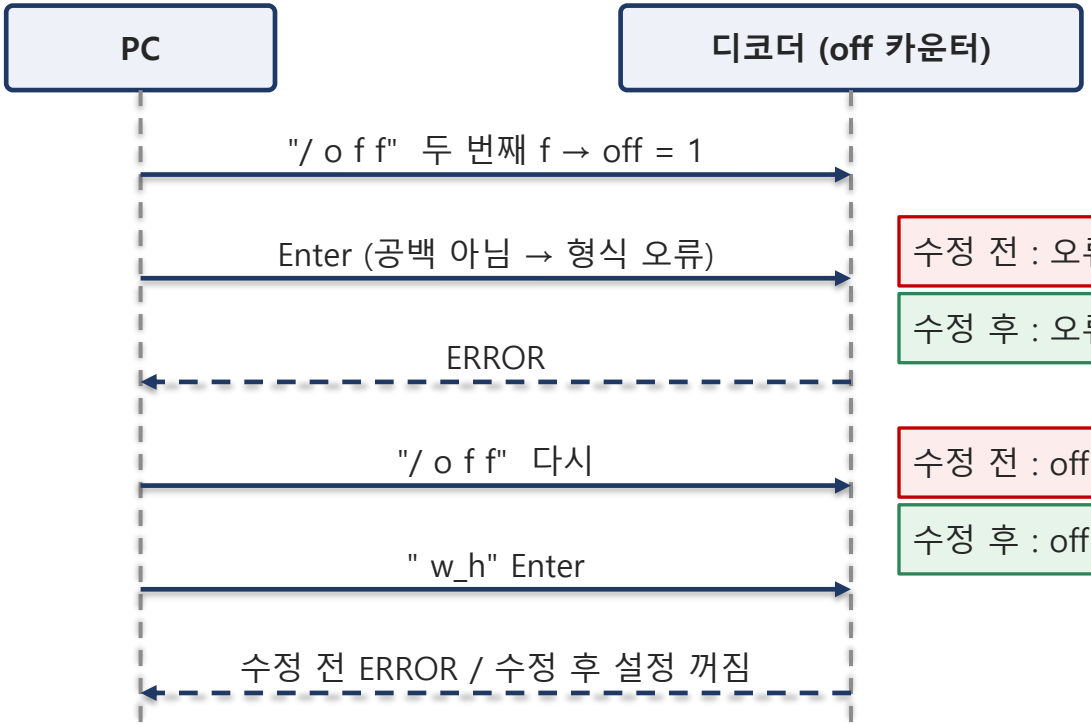
1 / 8 **담당 코드 : UART 컨트롤러**

바뀐 코드 : UART/01_UART_CONTROLLER.sv 296 줄

```
- else if(i_set && (c_module_control_bit ... == 7'b0000_010))
+ else if(i_set && (i_con_ascii == "w_hm")
+         && (c_module_control_bit ... == 7'b0000_010))
```

G7 — /off 를 한 번 잘못 보내면 그 뒤 /off w_h 도 계속 ERROR

2 / 8 **담당 코드 : 디코더**



수정 전 : 오류 때 off 안 지움 → off = 1 남음

수정 후 : 오류 때 off = 0

수정 전 : off 이미 1 → "f 또 옴" → 오류

수정 후 : off 0 → 정상

G7 — 바뀐 코드

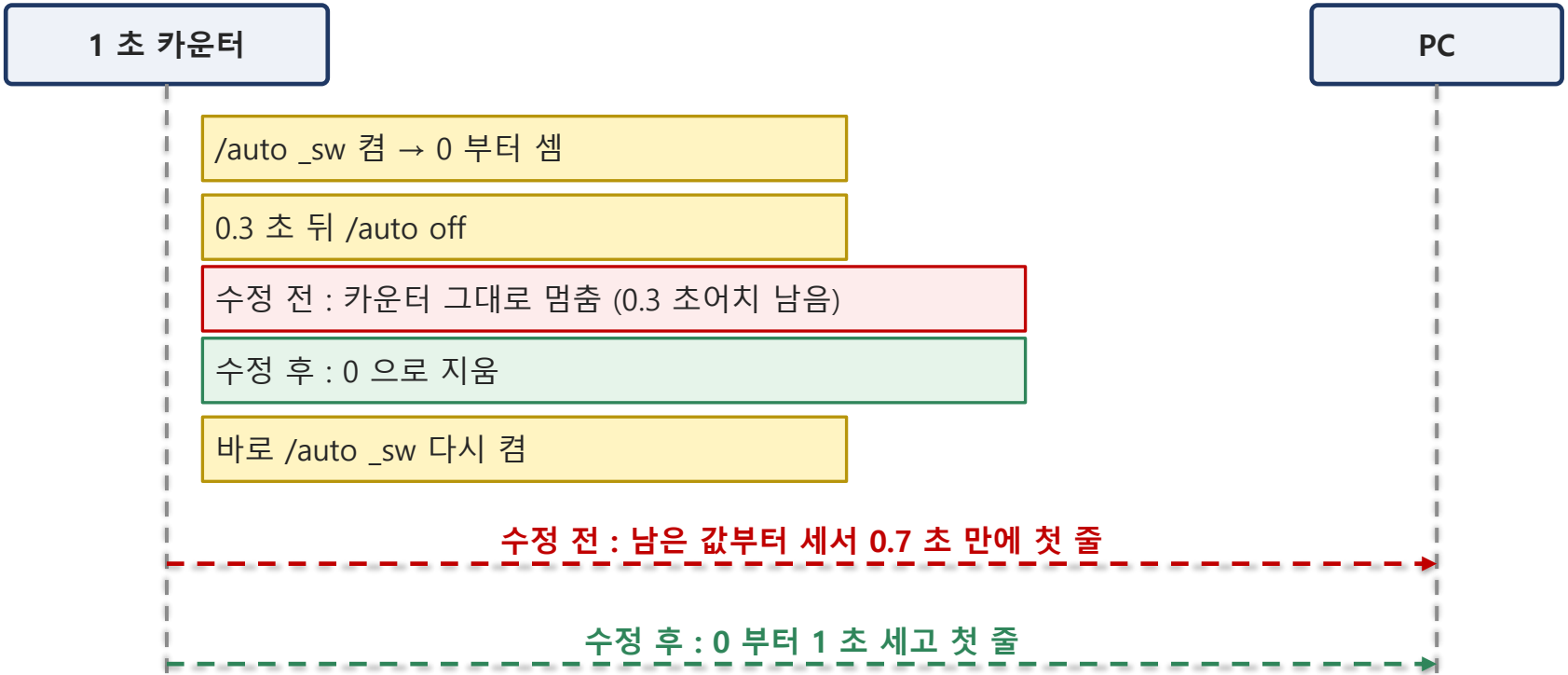
2 / 8 **담당 코드 : 디코더**

바뀐 코드 : UART/02_UART_ASCII_DECODER.sv 384 줄 (if(bad) 블록)

```
  n_error = 1;
+ n_off = 0;   // 오류 때 /off 글자 세던 값도 지움
  n_data_reg = 0;
```

H7 — /auto 를 껐다 바로 켜면 첫 줄이 1 초보다 일찍 옴

3 / 8 **담당 코드 : UART 컨트롤러**



부록 B 트러블 슈팅 2 상세

H7 — 바뀐 코드

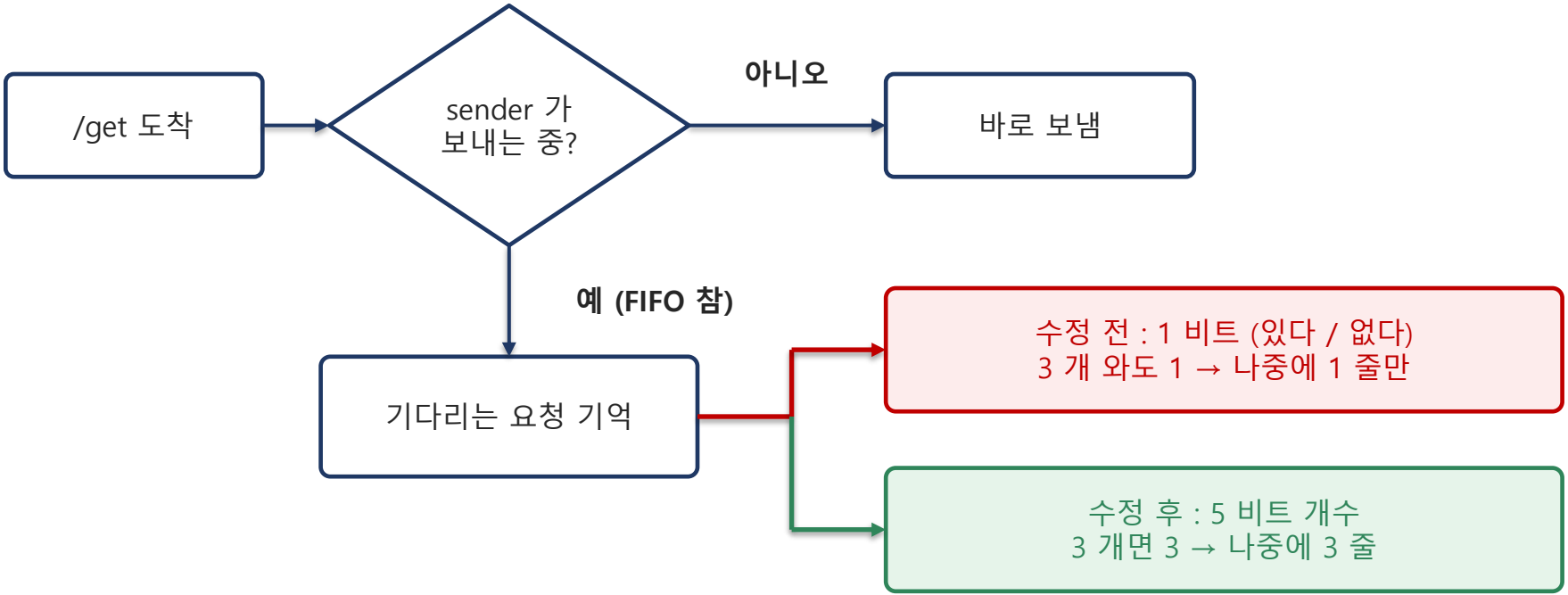
3 / 8 **담당 코드 : UART 컨트롤러**

바뀐 코드 : UART/01_UART_CONTROLLER.sv 524 줄 (sender_tick_generator, 꺼졌을 때)

```
  o_clk <= 0;
+ r_counter <= 0;   // 끌 때 카운터도 0
```

H9 — /get 을 쉬지 않고 20 번 보내면 15 줄만 옴

4 / 8 **담당 코드 : UART 컨트롤러**



H9 — 바뀐 코드

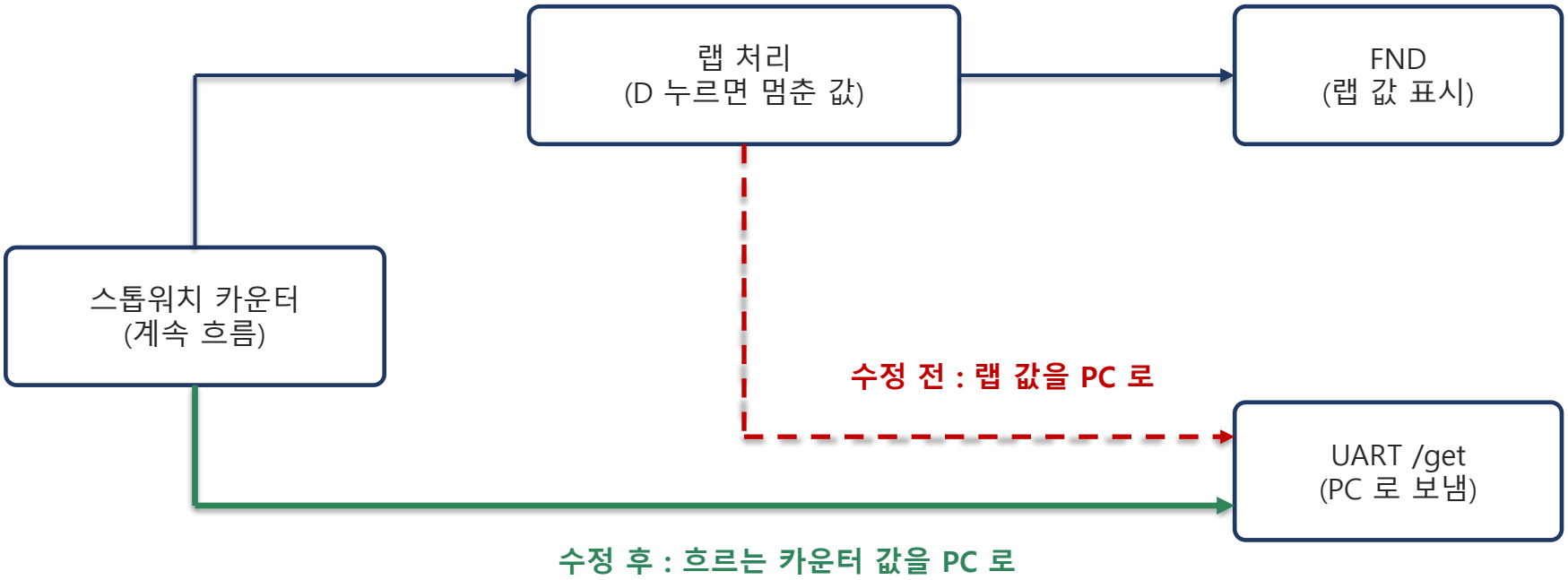
4 / 8 **담당 코드 : UART 컨트롤러**

바뀐 코드 : UART/01_UART_CONTROLLER.sv 72 · 353 ~ 357 줄

```
- reg c_send_pend;
+ reg [4:0] c_send_pend;
- n_send_pend = c_send_pend | get;          // get = 요청 1 클럭
+ n_send_pend = c_send_pend + get;
- n_send_pend = 0;                          // 보내면
+ n_send_pend = n_send_pend - 1;            // 보내면 하나만 뺌
```

D7 — 스톱워치 랩을 띄운 채 /get 하면 랩 값이 감

5 / 8 **담당 코드 : 스톱워치 (WTS)**



D7 — 바뀐 코드

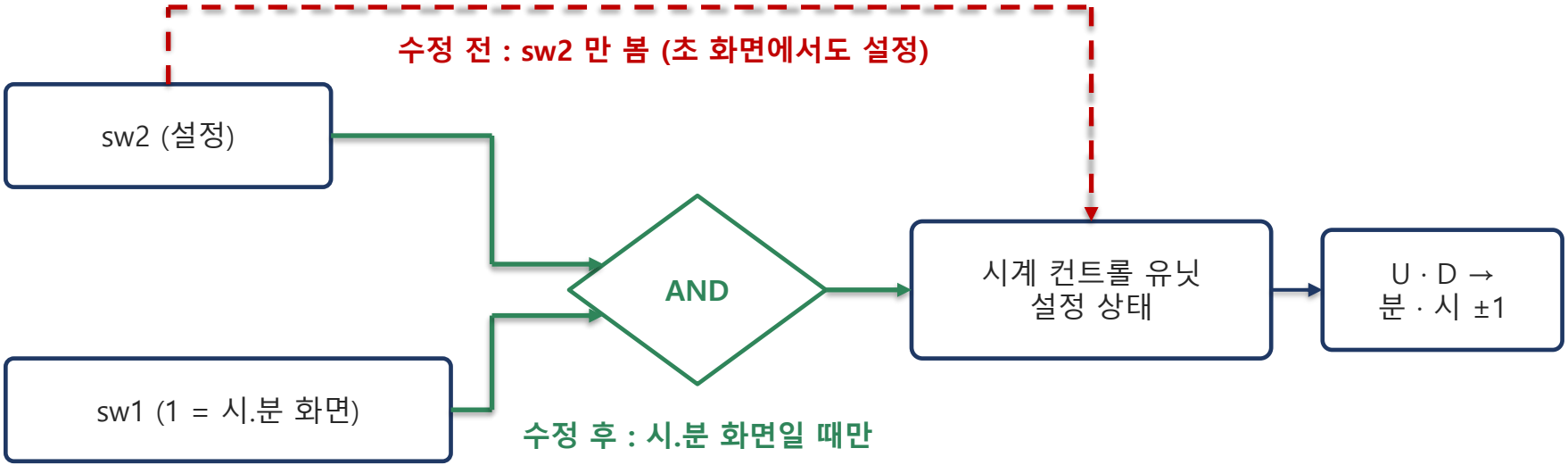
5 / 8 **담당 코드 : 스톱워치 (WTS)**

바뀐 코드 : WTS/stopwatch_datapath.v · WTS/wts_top.v

```
+ assign o_sw_run_time = {w_cnt_hour, w_cnt_min, w_cnt_sec, w_cnt_msec};
- assign o_sw_time = {w_sw_hour, w_sw_min, w_sw_sec, w_sw_msec};
+ assign o_sw_time = w_sw_run_time;   // 랩 전 카운터 값
```

E6 — 시계 초 화면에서 설정 + U → 안 보이는 분이 바뀜

6 / 8 **담당 코드 : 시계 (WTS)**



E6 — 바뀐 코드

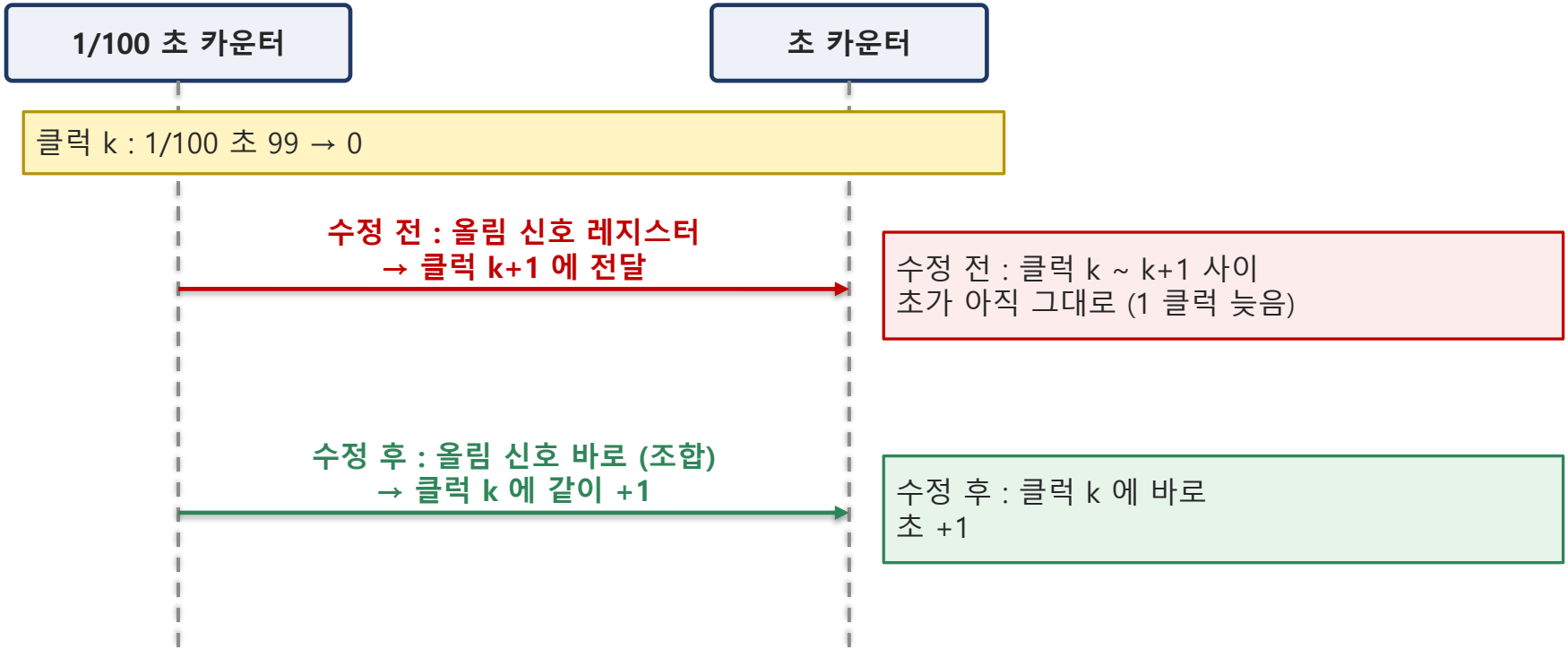
6 / 8 **담당 코드 : 시계 (WTS)**

바뀐 코드 : WTS/wts_top.v 167 줄 (U_WATCH_CONTROL_UNIT)

```
-      .i_set_sw2   (i_set_sw2),
+      .i_set_sw2   (i_set_sw2 & i_display_mode_sw1),
```

L6 — 초가 넘어가는 바로 그 클럭에 /get → 1 초 늦은 값

7 / 8 **담당 코드 : 스톱워치 카운터 (WTS)**



L6 — 바뀐 코드

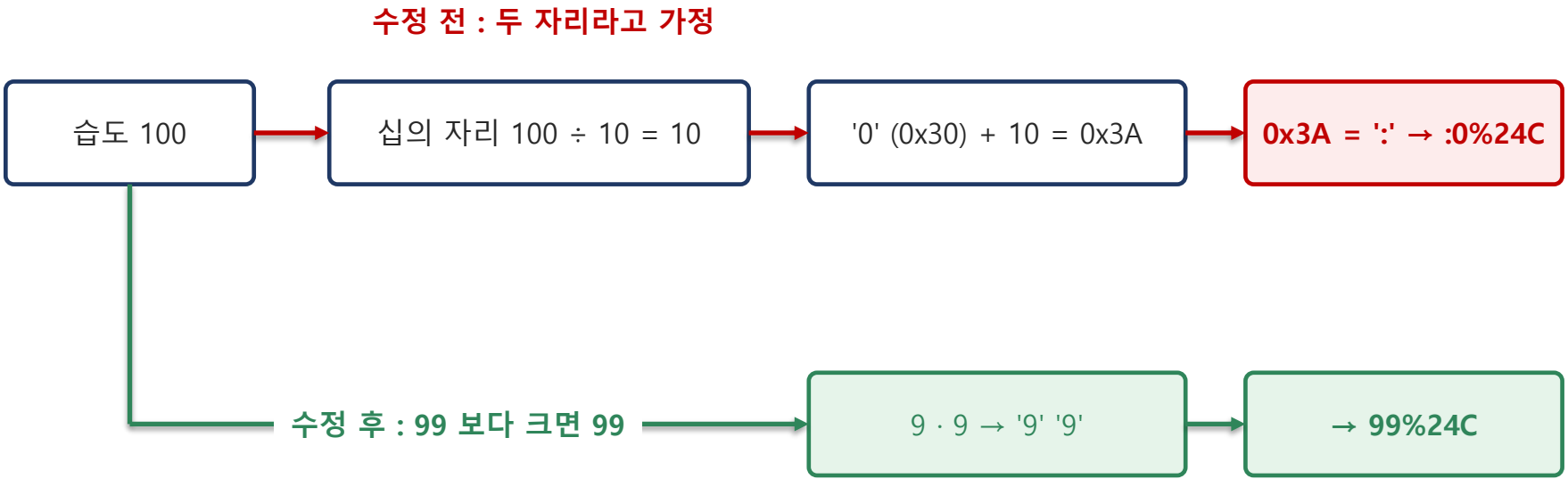
7 / 8 **담당 코드 : 스톱워치 카운터 (WTS)**

바뀐 코드 : WTS/stopwatch_datapath.v (stopwatch_tick_counter)

```
- output reg o_tick          // always 안에서 o_tick <= 1 (1 클럭 뒤)
+ output o_tick
+ assign o_tick = i_tick & (counter_reg == TIMES - 1);   // 다운은 == 0
```

J14 — 습도가 100 이면 PC 로 :0%24C

8 / 8 **담당 코드 : sender 글자 MUX (보윤)**



J14 — 바뀐 코드

8 / 8 **담당 코드 : sender 글자 MUX (보윤)**

바뀐 코드 : UART/sender_char_mux.v 22 ~ 23 줄

```
- wire [7:0] w_hum = i_data[15:8];
+ wire [7:0] w_hum = (i_data[15:8] > 8'd99) ? 8'd99 : i_data[15:8];
  (온도 w_temp 도 같은 방식)
```

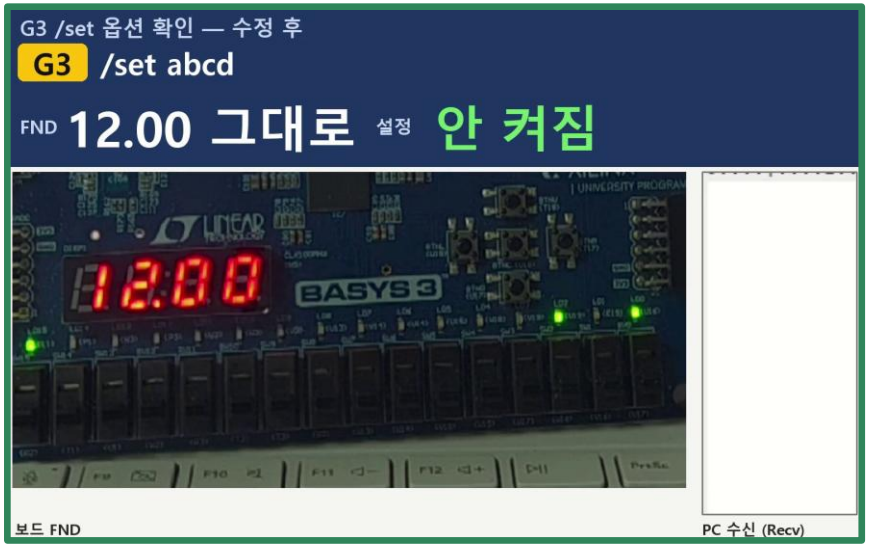
트러블 슈팅 2 ④ 보드 — G3 /set abcd

④ 적용 결과 **/set abcd 뒤 수정 전은 깜빡임 (설정 켜짐), 수정 후는 그대로**

수정 전



수정 후



	수정 전	수정 후
/set abcd 뒤	깜빡임 (설정 켜짐)	12.00 그대로

트러블 슈팅 2 ④ 보드 — G7 /off 오타 뒤 /off w_h

④ 적용 결과 **/off 오타 뒤 /off w_h : 수정 전 ERROR 또 · 계속 깜빡임 → 수정 후 설정 꺼짐**

수정 전



수정 후

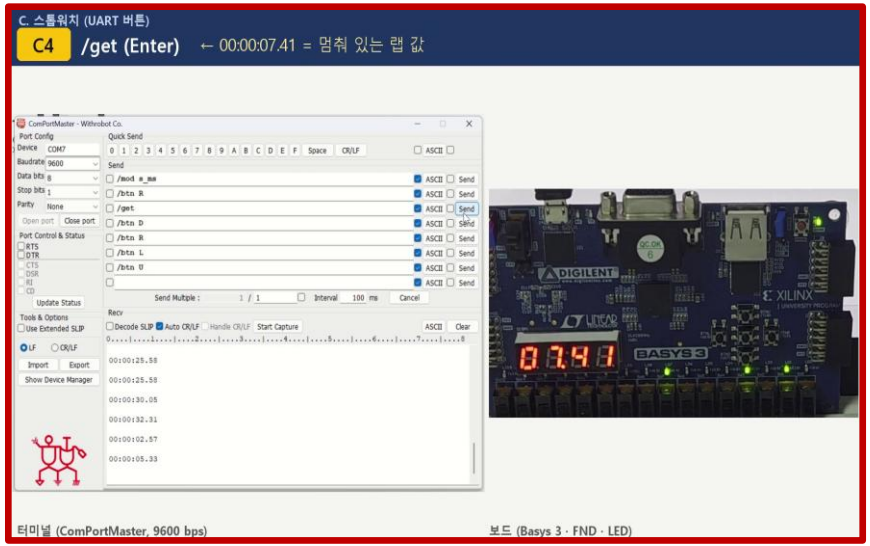


	수정 전	수정 후
/off w_h 뒤	ERROR · 계속 깜빡임	깜빡임 멈춤 · ERROR 없음

트러블 슈팅 2 ④ 보드 — D7 랩 중 /get

④ 적용 결과 **랩 표시 중 /get : 수정 전 랩 값 → 수정 후 흐르는 값 (FND 는 둘 다 랩)**

수정 전



수정 후

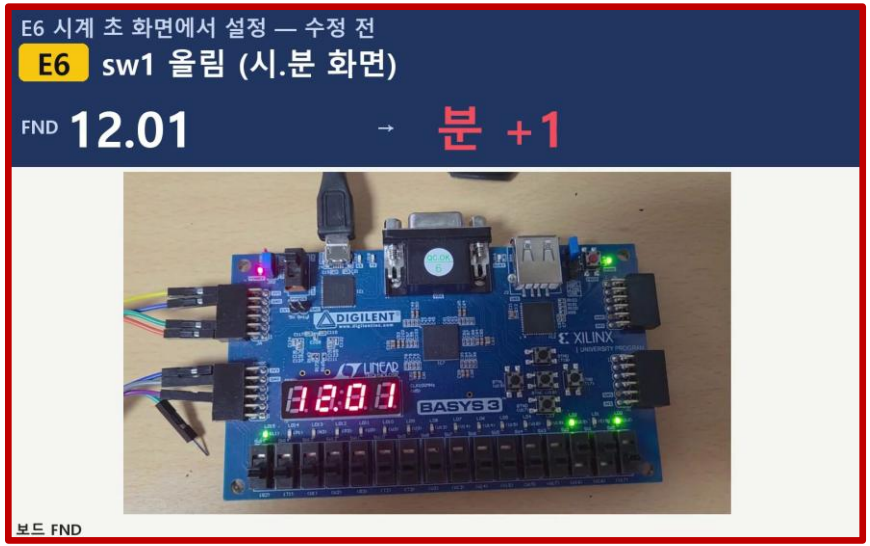


	수정 전 (시연 C)	수정 후
랩 중 /get	00:00:07.41 = 랩 값	00:00:05.90 · 07.89 (랩 03.17)

트러블 슈팅 2 ④ 보드 — E6 초 화면에서 설정

④ 적용 결과 **초 화면 · sw2 · btnU 뒤 시.분 화면 : 12.01 → 12.00 (분 그대로)**

수정 전



수정 후



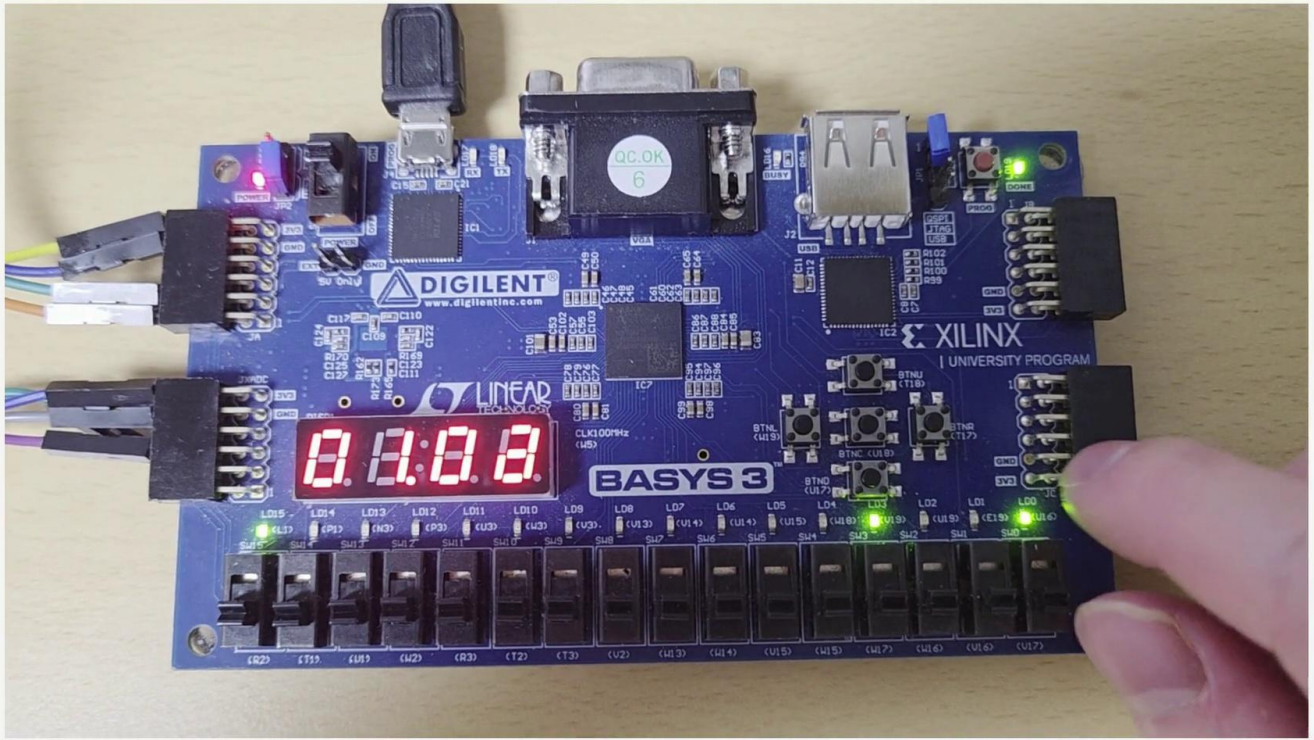
	수정 전	수정 후
sw1 올린 뒤 FND	12.01 (분 +1)	12.00 (그대로)

부록 C 시연 영상 전체

시연 A 스위치 모드

A. 스위치 모드 (UART 명령 보내기 전) — 리셋 직후

A1 **btnC (리셋)** FND 시계 00.00 부터 (초.1/100 초)

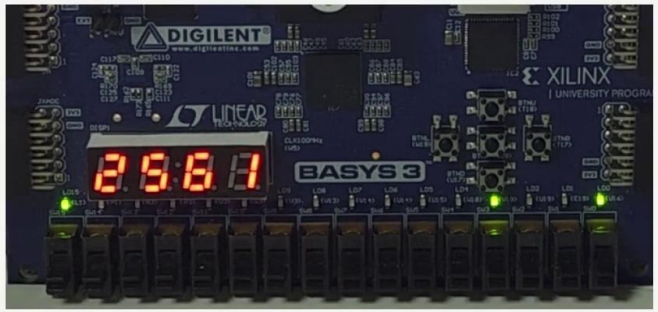


부록 C 시연 영상 전체

시연 B UART 로 시계

B. UART 로 화면 바꾸기 · 시계

B **시작 : 시계 초.1/100 화면** 이제부터 UART 명령으로 제어



터미널 (ComPortMaster, 9600 bps)

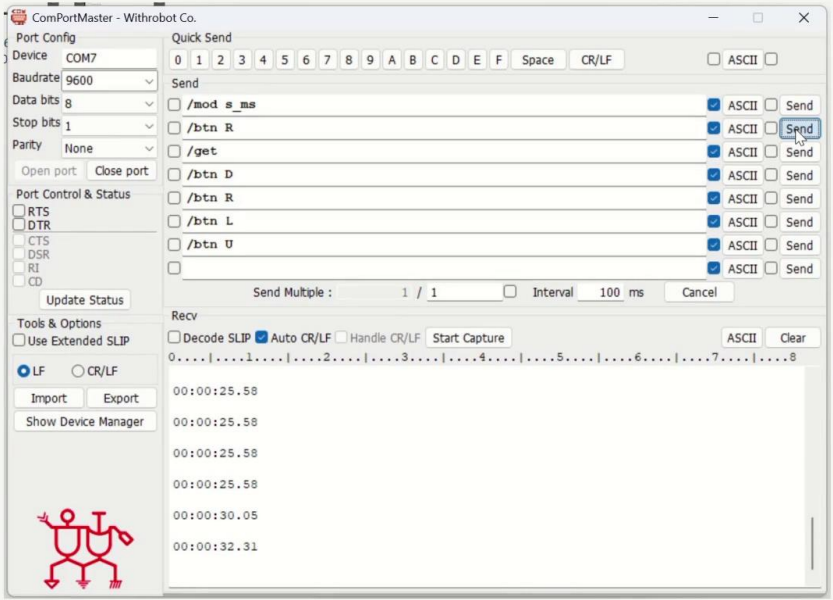
보드 (Basys 3 · FND · LED)

부록 C 시연 영상 전체

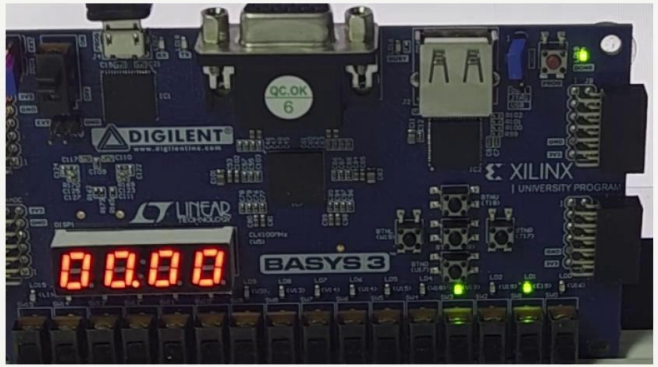
시연 C 스톱워치

C. 스톱워치 (UART 버튼)

C1 **/mod s_ms** FND 스톱워치 화면 (리셋 뒤 처음이면 00.00) · LD1



터미널 (ComPortMaster, 9600 bps)



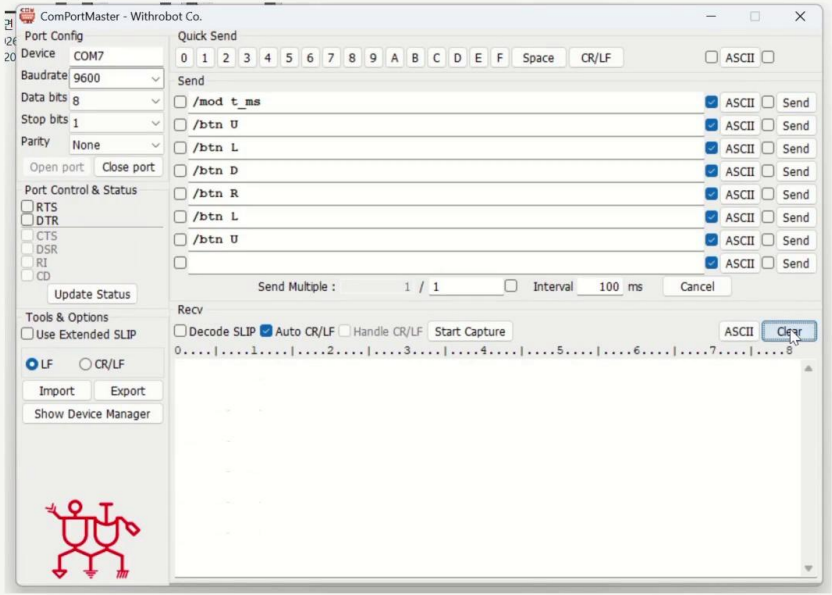
보드 (Basys 3 · FND · LED)

부록 C 시연 영상 전체

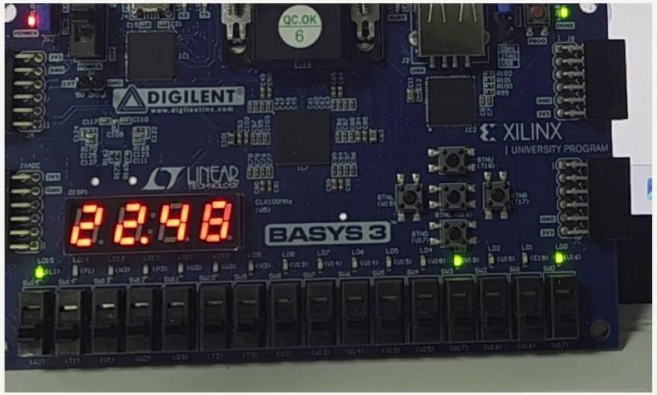
시연 D 타이머

D. 타이머

D **시작 : 앞 섹션 화면 (스톱워치)** 이제 타이머 화면으로



터미널 (ComPortMaster, 9600 bps)



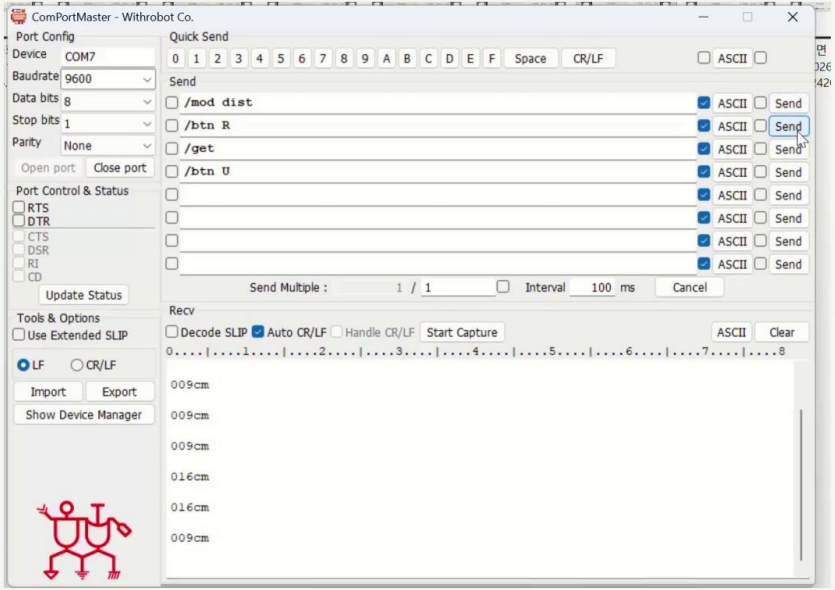
보드 (Basys 3 · FND · LED)

부록 C 시연 영상 전체

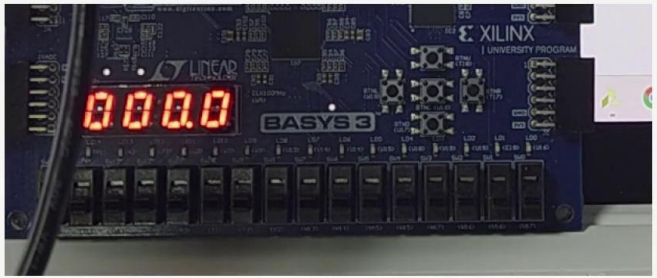
시연 E 거리 (SR04)

E. 거리 (SR04)

E1 /mod dist FND 거리 화면 000.0 (측정 전)



터미널 (ComPortMaster, 9600 bps)



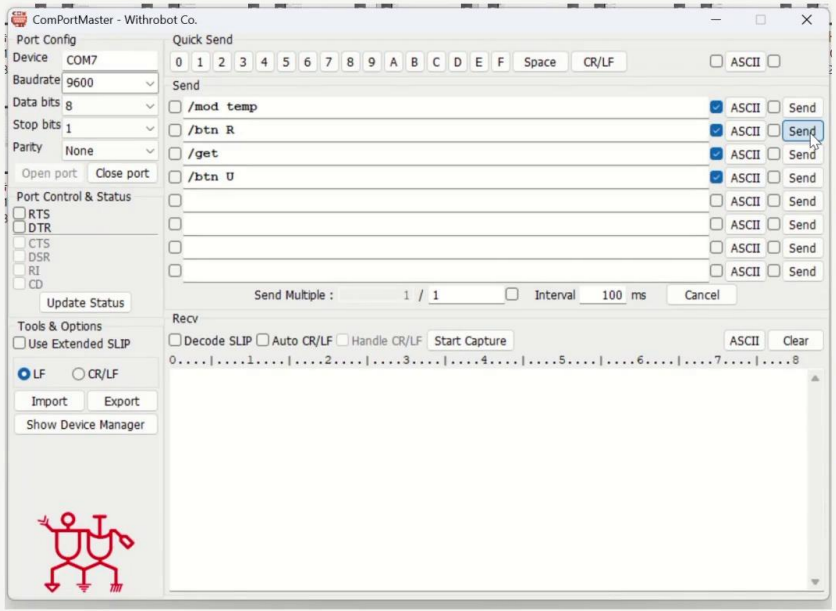
보드 (Basys 3 · FND · LED)

부록 C 시연 영상 전체

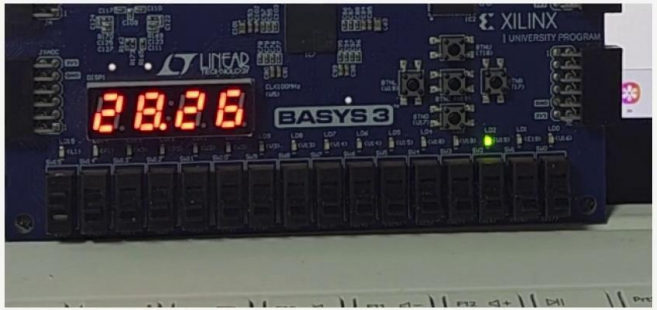
시연 F 온습도 (DHT11)

F. 온습도 (DHT11)

F2 **/btn R** 1 ~ 2 초 안에 FND 습.온 28.26 (습도 28 %, 26 ℃), LD2



터미널 (ComPortMaster, 9600 bps)



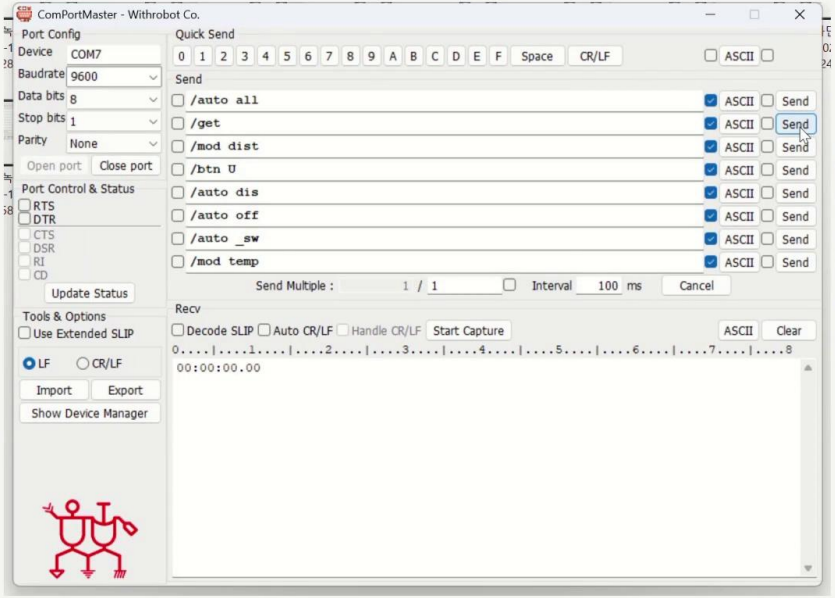
보드 (Basys 3 · FND · LED)

부록 C 시연 영상 전체

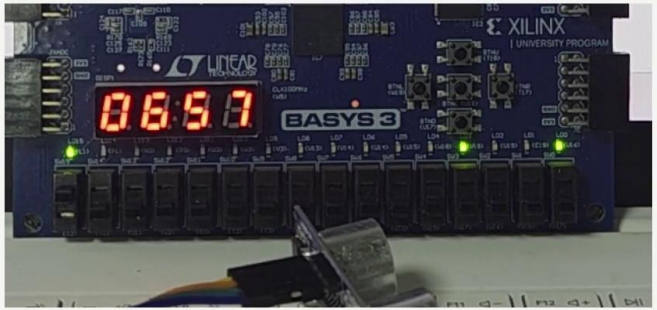
시연 G 자동 전송 /auto

G. 자동 전송 /auto

G1 **/auto all** 1 초마다 한 줄 : 스톱워치 → 타이머 → 시계 → 거리 → 온습도 → 다시



터미널 (ComPortMaster, 9600 bps)



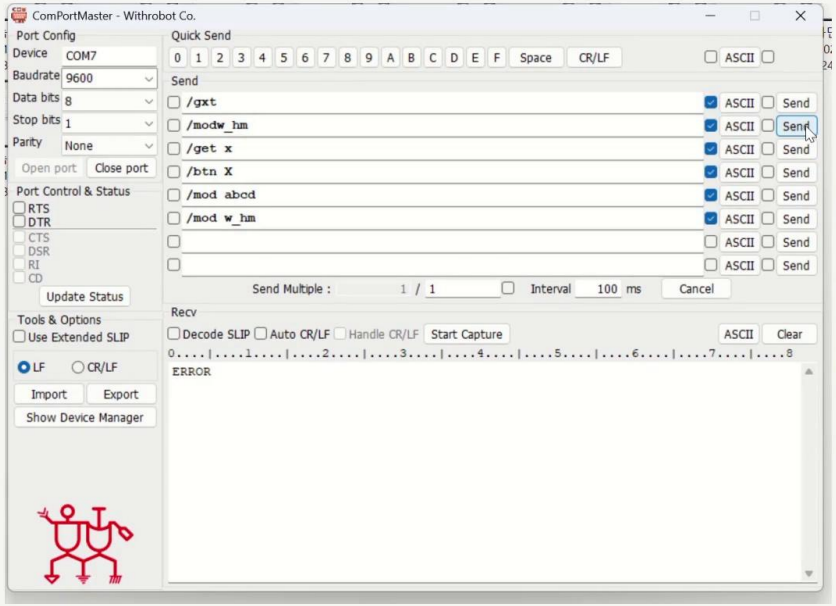
보드 (Basys 3 · FND · LED)

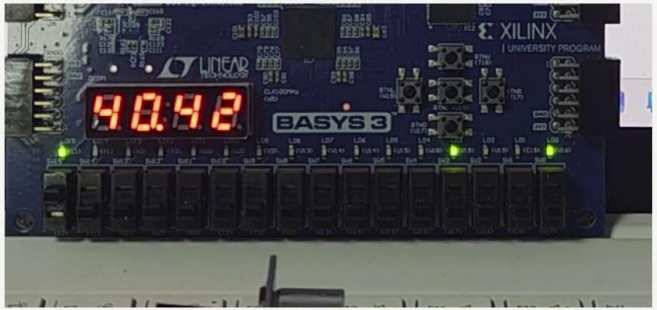
부록 C 시연 영상 전체

시연 H 잘못된 명령 → ERROR

H. 잘못된 명령 → ERROR

H1 /gxt ← ERROR (모르는 명령)





터미널 (ComPortMaster, 9600 bps)

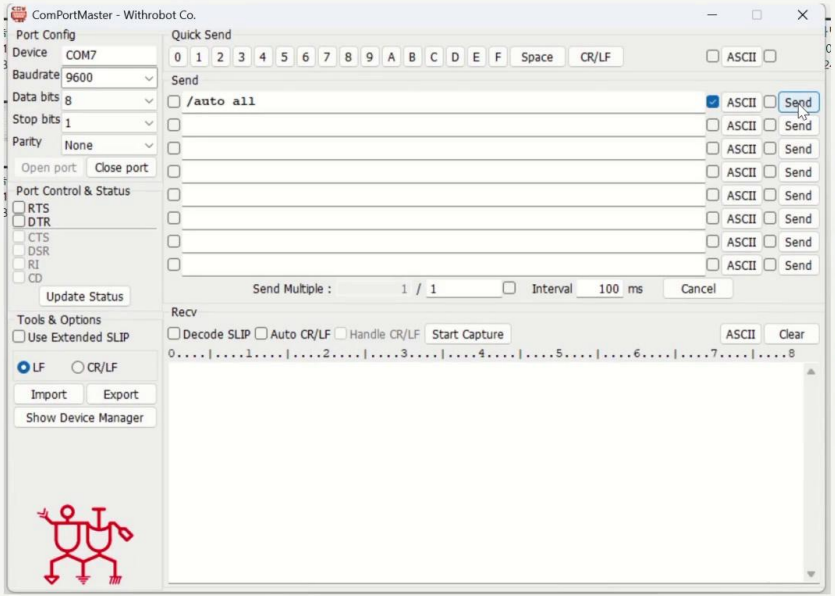
보드 (Basys 3 · FND · LED)

부록 C 시연 영상 전체

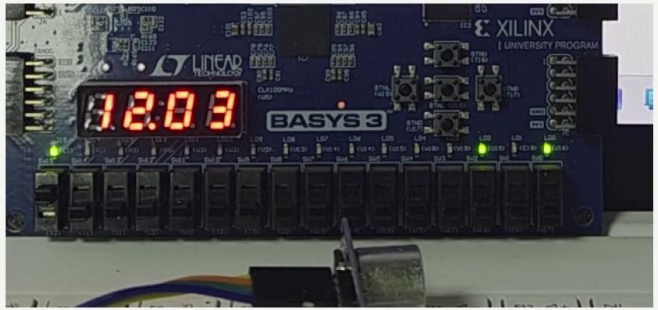
시연 I 리셋

I. 리셋

I1 **/auto all** 1 초마다 한 줄 : 스톱워치 → 타이머 → 시계 → 거리 → 온습도 → 다시



터미널 (ComPortMaster, 9600 bps)



보드 (Basys 3 · FND · LED)